

History of SQL

SQL traces its origin to Dr. E. F. Codd's 1970 paper, "A Relational Model of Data for Large Shared Data Banks," published in the Communications of the ACM. Codd's relational model became the theoretical foundation for relational database management systems (RDBMS). Building on this model, IBM researchers Donald D. Chamberlin and Raymond F. Boyce developed a query language originally called SEQUEL (Structured English Query Language) in the early 1970s. The name was later shortened to SQL following a trademark conflict with the UK aircraft manufacturer Hawker Siddeley, which held the trademark "SEQUEL." SQL is still commonly pronounced "sequel" today.

Key Milestones in SQL History

- Early 1970s: Donald D. Chamberlin and Raymond F. Boyce at IBM develop SEQUEL, based on Codd's relational model.
- Name change: SEQUEL is renamed SQL due to a trademark dispute with the Hawker Siddeley aircraft company.
- 1979: Relational Software, Inc. (later renamed Oracle) releases the first commercially available implementation of SQL (Oracle V2).
- 1986: The American National Standards Institute (ANSI) formally adopts SQL as a standard.
- 1987: The International Organization for Standardization (ISO) adopts SQL as an international standard.

Note: SQL is often described as introduced in 1974, referring to the Chamberlin & Boyce SEQUEL paper; the underlying relational model it implements was published by Codd in 1970.

History of Microsoft SQL Server

Microsoft SQL Server originated from a 1988 partnership between Microsoft, Sybase, and Ashton-Tate to port Sybase's relational database engine to the OS/2 platform. The first version, SQL Server 1.0, was a 16-bit relational database management system released on April 24, 1989, for OS/2 — essentially equivalent to Sybase SQL Server on Unix and VMS at the time.

Key Early Milestones

- 1988: Project begins as a three-way collaboration (Microsoft, Sybase, Ashton-Tate) to port Sybase's database engine to OS/2.
- 1989: Commercial launch of SQL Server 1.0 for OS/2 (16-bit).
- 1992: SQL Server 4.2 ships, bundled with OS/2 version 1.3.
- 1993: Version 4.21 is released for Windows NT alongside Windows NT 3.1, marking SQL Server's move toward Microsoft's own 32-bit Windows platform.
- 1993–94: The Microsoft–Sybase co-development agreement lapses; Microsoft licenses the source code and continues development independently, while retaining the "SQL Server" name.
- 1996: Sybase renames its own product Adaptive Server Enterprise (ASE) to distinguish it from Microsoft SQL Server.

Core Differences: 1989 vs. 2022

Feature	SQL Server 1.0 (1989)	SQL Server 2022
Architecture	16-bit, OS/2	64-bit, Multi-OS (Windows & Linux)
Primary Codebase	Sybase-derived (C)	Native Microsoft (C++)
Connectivity	Local network only	Cloud-connected (Azure integration)
Scale Limits	Very limited (departmental)	Up to 524 PB (Enterprise edition)
Security	Basic permissions	SQL Ledger (blockchain-style), Always Encrypted

Major Evolutionary Leaps

- Platform independence: SQL Server 1.0 was tied to OS/2; modern versions run on Windows, Linux, and Docker containers.
- Intelligent performance: SQL Server 2022 includes Intelligent Query Processing (IQP), which adapts and tunes query execution based on past runs.
- Azure Synapse Link: provides near real-time analytics on operational data without impacting transactional performance.
- Managed Instance Link: enables disaster recovery by failing over to Azure SQL Managed Instance.
- Modern data types: SQL Server 2022 supports JSON, XML, Graph data, and Spatial data, well beyond the flat tables of 1989.
- Analytics & AI: built-in support for Python, R, and Java enables machine learning directly within the database engine — a capability that did not exist in the original 1.0 release.

Types of Databases:

Modern data platforms span several categories, each suited to different workloads — from structured transactional systems to flexible, large-scale, or real-time stores.

Relational Databases (RDBMS)

Structured data stored in tables of rows and columns, queried and managed using SQL.

Popular platforms: Microsoft SQL Server, MySQL, PostgreSQL, Oracle Database, Teradata, IBM Db2.

Tool Type	Examples
Native DB Tools	SQL Server Management Studio, MySQL Workbench, pgAdmin
Cross-Platform SQL Tools	DBeaver, Toad
BI Tools	Power BI, Tableau
Programming	Python (pyodbc, psycopg2), .NET, JDBC

NoSQL Databases

Designed for unstructured or semi-structured data; not strictly table-based.

Type	Example Databases
Document DB	MongoDB, CouchDB
Key-Value	Redis, Amazon DynamoDB
Column-family	Apache Cassandra, HBase
Graph DB	Neo4j

Tools to access:

- MongoDB Compass (MongoDB GUI)
- Cassandra Query Language (CQL)
- Redis CLI
- Python / Node.js drivers

- BI tools (Power BI supports MongoDB and others via connectors)

Data Warehouses

Optimized for analytics (OLAP workloads) rather than transactional processing.

Popular platforms: Snowflake, Amazon Redshift, Google BigQuery, Azure Synapse Analytics, Teradata.

Access tools:

- Web-based SQL editors
- Power BI / Tableau
- dbt
- Python (Snowflake connector, etc.)
- Azure Data Studio

Cloud-Native Databases

Fully managed database services operated by a cloud provider.

- Azure SQL Database
- Amazon RDS
- Google Cloud SQL
- Cosmos DB

Big Data / Distributed Systems

Used for very large-scale data processing across distributed clusters.

- Apache Hadoop
- Apache Spark
- Databricks

Accessed via:

- Spark SQL
- Notebooks (Databricks, Jupyter)
- Power BI connectors
- PySpark

In-Memory Databases

Optimized for very fast processing and real-time analytics by keeping data in memory.

- SAP HANA
- Redis

Quick Comparison

Category	Best For	Example Use Case
RDBMS	Transactions	Banking system
NoSQL	Flexible schema	E-commerce catalog
Data Warehouse	Analytics	BI dashboards
Big Data	Massive datasets	IoT, logs
In-Memory	Real-time	Trading systems

DBMS vs RDBMS

A DBMS (Database Management System) is general software for storing and managing data, while an RDBMS (Relational Database Management System) is a more advanced type of DBMS built on the relational model, enforcing structure and relationships between data.

Feature	DBMS	RDBMS
Definition	Software used to store and manage data	Advanced type of DBMS based on the relational model
Data Storage	Data stored as files	Data stored in tables (rows & columns)
Relationship Between Data	No relationship enforced	Relationships maintained using Primary & Foreign Keys
Normalization	Not mandatory	Follows normalization rules
Redundancy	Higher redundancy	Reduced redundancy
Data Integrity	Low to moderate	High (constraints enforced)
Constraints Support	Limited	Supports Primary Key, Foreign Key, Unique, Check, Not Null
Security	Basic security	Advanced security & role-based access
Users Supported	Typically single-user systems	Multi-user systems
Scalability	Small applications	Large enterprise-level applications
Transactions (ACID)	Not fully supported	Fully supports ACID properties
Examples	dBase, FoxPro	Microsoft SQL Server, MySQL, PostgreSQL, Oracle Database

SQL Server Architecture

Overview

SQL Server's architecture is organized into a small number of cooperating layers: a client layer that issues requests, a protocol layer that transports them, a relational engine that plans and drives execution, and a storage engine that physically manages data, transactions, and concurrency. Underneath the relational and storage engines sits SQLOS, a thin scheduling and memory-management layer that both engines share. The table below summarizes each layer before the detailed breakdown.

Layer	Primary Responsibility	Key Components
1. Client Layer	Issue T-SQL requests, render results	SSMS, Power BI, custom apps
2. Protocol Layer	Transport requests to/from the server	TDS over TCP/IP, Named Pipes, Shared Memory
Security Layer	Authenticate logins, authorize access	Windows/SQL Auth, roles, permissions
3. Relational Engine	Parse, optimize, and execute queries	Parser, Optimizer, Executor, Plan Cache
SQLOS	Scheduling, memory management, latching	Schedulers, Buffer Pool memory broker
4. Storage Engine	Read/write data, manage transactions & locks	Buffer Manager, Log Manager, Lock Manager, Access Methods
5. Database Structure	Physical organization of data on disk	8 KB pages, extents, .mdf/.ndf/.ldf files, B-Tree indexes

1. Client Layer

Applications and tools that originate T-SQL requests. This includes SQL Server Management Studio (SSMS), Power BI, reporting tools, and custom applications built on ODBC, OLE DB, ADO.NET, or JDBC drivers. The client layer is only responsible for composing the request and rendering the returned result set — no query logic executes here.

2. Protocol Layer

Handles communication between the client and SQL Server using SQL Server Network Interface (SNI). The dominant wire protocol is TDS (Tabular Data Stream), which can run over TCP/IP (the default for network connections), Named Pipes, or Shared Memory (for local connections). This layer receives the incoming request and hands it to the Relational Engine; on the way out, it packages result sets and error messages back to the client.

Security & Authentication (supporting layer)

Not called out in the original outline but essential to the real flow: before any query is compiled, SQL Server authenticates the login (Windows Authentication or SQL Server Authentication) and authorizes the requested operation against database roles and permissions. This happens immediately after the protocol layer and before the Relational Engine begins parsing.

3. Relational Engine (Query Processor)

The 'brain' of SQL Server. It understands the query, decides how best to execute it, and drives the Storage Engine to retrieve or modify data.

Query Parser

- Checks T-SQL syntax and validates that referenced objects (tables, columns, views) exist.
- Produces a parse tree, then the Algebrizer resolves names and data types into a logical tree used downstream.

Query Optimizer

- Generates multiple candidate execution plans (join orders, index choices, seek vs. scan).
- Uses table/index statistics to cost each plan and selects the most efficient one.
- First checks the Plan Cache for a reusable compiled plan before spending time compiling a new one.

Query Executor

- Executes the plan chosen by the optimizer, operator by operator (scans, joins, sorts, aggregates).
- Requests the actual data pages/rows it needs from the Storage Engine as execution proceeds.

Plan Cache (supporting component)

Added detail: SQL Server caches compiled execution plans in memory so that repeated executions of the same (or parameterized) query skip re-parsing and re-optimizing, which is a major performance factor in real workloads.

SQLOS (supporting layer)

Added detail: SQLOS is a thin layer beneath both the Relational Engine and the Storage Engine. It is SQL Server's own abstraction over the operating system — it schedules worker threads across CPU schedulers, brokers memory (including the Buffer Pool) between components, and manages low-level synchronization primitives (latches) and exception handling. Both engines rely on SQLOS rather than calling the OS directly, which is part of why SQL Server behaves consistently across hardware.

4. Storage Engine (Data Manager)

Physically reads and writes data on disk and in memory, and enforces the guarantees (durability, isolation, consistency) that make the data trustworthy under concurrent use.

Buffer Manager

- Maintains the Buffer Pool, an in-memory cache of 8 KB data pages, to avoid repeated disk reads.
- The Lazy Writer background process periodically flushes least-recently-used dirty pages back to disk under memory pressure.

Transaction Log Manager

- Implements Write-Ahead Logging (WAL): every change is recorded in the transaction log before the data page itself is modified on disk.
- Guarantees durability and enables crash recovery (rollback of incomplete transactions, roll-forward of committed ones).

Lock Manager

- Controls concurrent access with row-, page-, and table-level locks.
- Enforces the session's isolation level and detects/resolves deadlocks between competing transactions.

Data Files / Access Methods

- Stores actual table and index data in primary (.mdf) and secondary (.ndf) files; log records are stored in .ldf files.
- Access Methods manage how tables and indexes (B-Trees) are physically read and updated.
- Added detail: the Checkpoint process periodically writes all dirty buffer pages to disk, shortening the work needed during crash recovery.

System Databases (supporting detail)

Added detail: four built-in databases support every request across the instance:

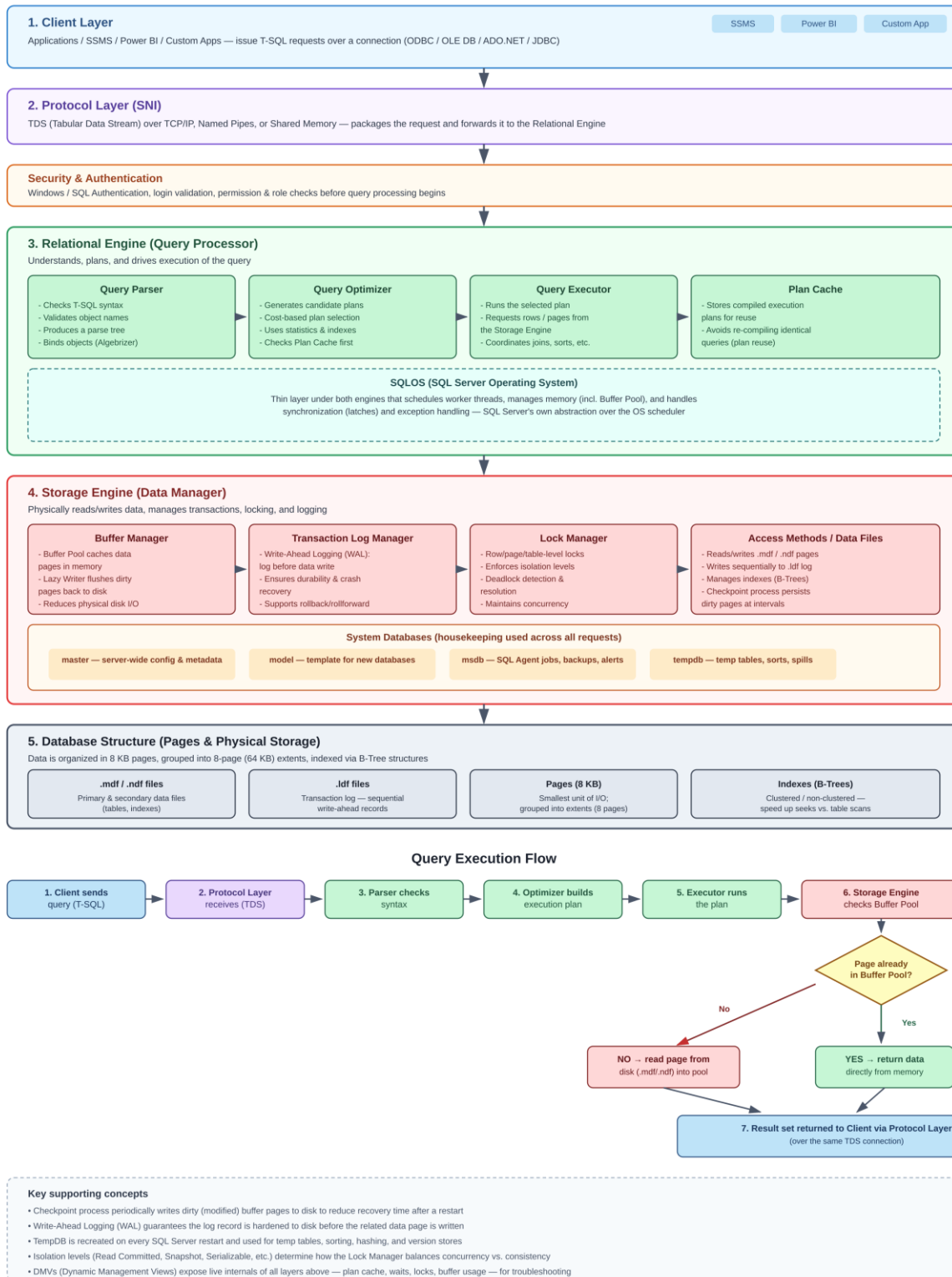
- master — server-wide configuration and metadata about every other database.
- model — the template copied to create any new user database.
- msdb — SQL Server Agent jobs, schedules, backup/restore history, alerts.
- tempdb — temporary tables, sort/hash work areas, and row-versioning; recreated on every restart.

5. Database Structure (Pages & Storage)

- Data is stored in 8 KB pages, the smallest unit of I/O in SQL Server.
- Pages are grouped into extents — 8 contiguous pages (64 KB) each — allocated as either mixed or uniform extents.
- Tables and indexes are organized as B-Trees (clustered index = data pages in the tree; non-clustered index = a separate lookup structure) for efficient seeks.
- TempDB, user data files, and log files are all structured using this same page/extent model, though TempDB is optimized for high-churn temporary work.

SQL Server Architecture

End-to-end query flow from client request to data retrieval



What is a Database:

A database in Microsoft SQL Server is a structured collection of data and objects, such as tables, views, and stored procedures, organized and stored electronically within a relational database management system (RDBMS). It serves as a centralized repository for storing, managing, retrieving, and manipulating information efficiently and securely

1. Data Storage (Tables)

- Data is stored in tables (rows & columns)
- Example:
 - Employee table
 - Orders table

2. Database Objects

A database contains multiple objects:

- Tables → Store data
- Views → Virtual tables (saved queries)
- Stored Procedures → Predefined logic
- Functions → Reusable computations
- Indexes → Improve performance

3. Schema Organization

- Data is organized into schemas (like folders)
- Example:
 - `dbo.Employee`
 - `sales.Orders`

4. Transaction Management

- Ensures data consistency and reliability
- Supports ACID properties:
 - A - Atomicity
 - C - Consistency
 - I - Isolation
 - D - Durability

5. Security

- Controls who can access data
- Users, roles, permissions

ACID:

Atomicity:

It ensures that a transaction is treated as a single, indivisible unit of work. Either all of the operations within the transaction succeed, or none of them do. If a failure occurs mid-transaction (e.g., an error or power outage), any changes made are automatically rolled back, returning the database to its state before the transaction started.

In simple words, All or nothing execution.

If any part of the transaction fails → entire transaction is rolled back

No partial updates allowed.

Consistency:

It guarantees that a transaction brings the database from one valid state to another valid state. It enforces predefined rules and integrity constraints (like PRIMARY KEY, FOREIGN KEY, and CHECK constraints). If a transaction violates any of these rules, it is aborted, preventing the entry of invalid data and maintaining the database's integrity.

In simple words, only valid data is allowed.

Data must satisfy:

- Constraints (Primary Key, Foreign Key)

- Data types
- Business rules

Isolation:

It ensures that concurrent transactions operate independently without interfering with each other. Each transaction runs as if it were the only one executing at that time. SQL Server uses locking mechanisms and provides different isolation levels (such as READ COMMITTED, REPEATABLE READ, and SERIALIZABLE) to manage how transactions see and interact with data being modified by others.

In simple words, Transactions do not affect each other

Each transaction works as if it is alone in the system

Prevents issues like:

- Dirty reads
- Non-repeatable reads
- Phantom reads

Durability:

It guarantees that once a transaction has been successfully committed, its changes are permanent and will not be lost, even in the event of a system failure, crash, or power outage. This is typically achieved by recording the changes in a transaction log and persisting them to non-volatile storage (disk)

In simple words, committed data is permanent.

Data is securely stored on disk and protected from loss

Even after crash → data remains

Common Use Cases of ACID Guarantees:

- ✓ Banking and financial transfers, where a debit from one account and a credit to another must both succeed or both fail (Atomicity).
- ✓ E-commerce order placement, where inventory deduction and order creation must remain consistent (Consistency).
- ✓ Ticket booking systems, where two customers booking the same seat concurrently must not corrupt each other's transaction (Isolation).
- ✓ Any system of record that must retain committed data after a server crash or power failure (Durability).

Types of system defined databases:

Master

Modeldb

MsdB

Tempdb

master: The most critical database, as it records all system-level information for the SQL Server instance. It stores metadata about all other databases, login accounts, and server configuration settings. SQL Server cannot start if the master database is unavailable.

Detailed Explanation

- ✓ It acts as the **central repository** for the entire SQL Server instance.
- ✓ Every database, login, and system configuration is **registered and tracked here**.
- ✓ SQL Server reads master DB **during startup** to initialize the system.

What it stores

- List of all databases (sys.databases)
- Server-level logins (sys.logins)
- Configuration settings (sp_configure values)
- Linked servers
- Endpoints and system-wide settings

model: This acts as a template for any new user database created on the SQL Server instance. The entire content and database options of model are copied to the new database. The tempdb database is also created using the model as a template every time the SQL Server service restarts.

Detailed Explanation

- ✓ Whenever a new database is created, SQL Server **copies the structure and settings** from the model database.
- ✓ It ensures **consistency and default configuration** across all databases.

What it stores

- Default database settings (recovery model, collation, file size)
- User-defined objects (tables, views, procedures if added manually)

msdb: Used by the SQL Server Agent for scheduling alerts, jobs, and maintenance plans. It also stores a complete online backup and restore history, Database Mail configuration, and other service broker information.

Detailed Explanation

- ✓ It supports **job scheduling and automation processes** inside SQL Server.
- ✓ All scheduled tasks (like ETL jobs, backups, maintenance) are managed through msdb.

What it stores

- SQL Server Agent jobs (sysjobs)
- Job execution history (sysjobhistory)
- Backup & restore history (backupset, backupmediafamily)
- Alerts and operators
- Database Mail configuration
- Maintenance plans

tempdb: A global, temporary workspace used by all users and processes to store temporary objects (like temporary tables and stored procedures) and intermediate result sets for operations such as sorting or hash joins. It is recreated from a clean copy of the model database every time the SQL Server service is started. It can be a performance bottleneck if not managed properly, and it's often recommended to place it on a high-performance drive

Detailed Explanation

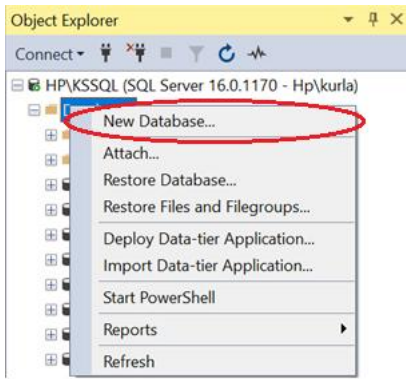
- ✓ It acts as a **workspace** for SQL Server during query execution.
- ✓ It is recreated **fresh every time SQL Server restarts**, meaning all data inside is temporary.

What it stores

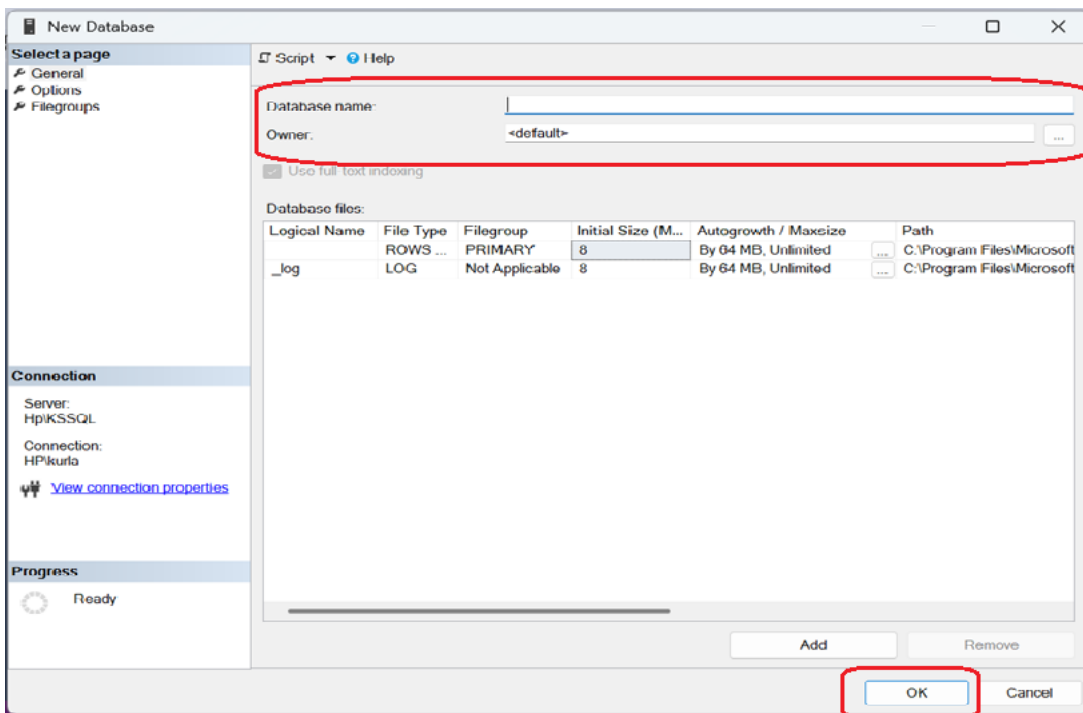
- Temporary tables (#temp, ##temp)
- Table variables
- Intermediate query results (sorting, joins, grouping)
- Index operations (rebuild, sort)
- Row versioning (snapshot isolation, triggers)

Create a New Database (Using UI):

It is quite simple to create a new database in Microsoft SQL Server. Just right-click on the "Databases" node and select "New Database..."



There are lots of settings you may set regarding your database, but the only information you must fill in is the name of your database and click ok.



Create a new database by using query:

Syntax:

CREATE DATABASE DATABASE NAME

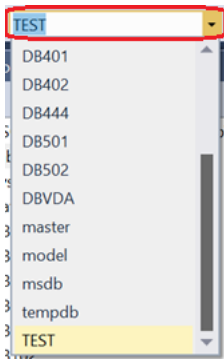
Below is the query to create database:

CREATE DATABASE TEST

How to use database:

Method 01:

Select the database from the dropdown menu as shown in below picture



Method 02:

Write a query to use database:

```
USE TEST
```

```
GO
```

Introduction to Datatypes in SQL Server:

A data type in SQL Server is an attribute that specifies the type of data a column, variable, or parameter can hold, such as integer data, character data, monetary data, or date and time data. Choosing the correct data type is crucial for maintaining data integrity, optimizing storage space, and ensuring efficient query performance:

In simple terms:

A data type tells SQL Server what kind of value is allowed in a column or variable or parameter.

Why Do We Use Data Types?

Data types are essential because they:

1. Ensure Data Accuracy

- Prevent invalid data entry
- Example: You cannot store text in an INT column

2. Optimize Storage

- Each data type uses specific memory
- Choosing correct type → saves disk space

3. Improve Performance

- Smaller and appropriate data types → faster queries
- Efficient indexing and processing

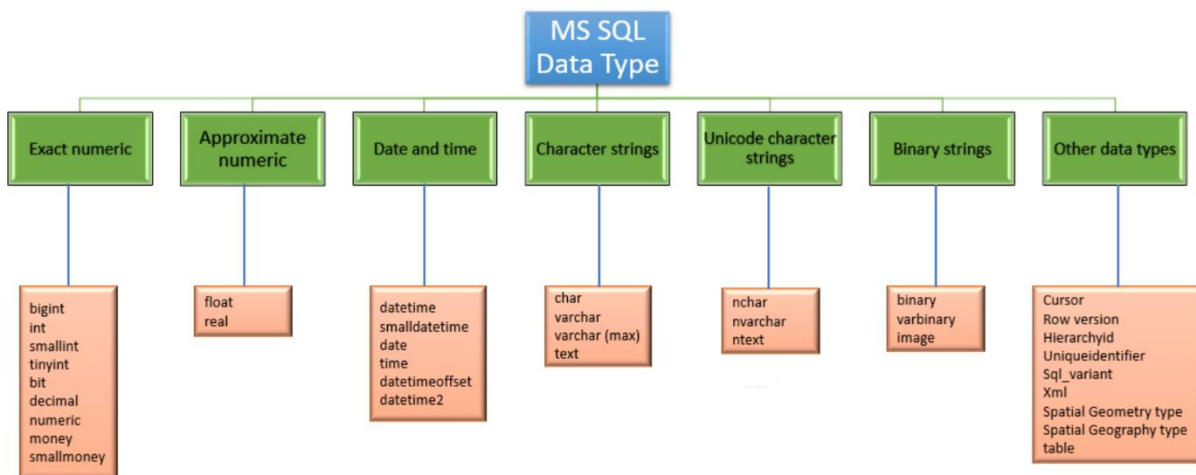
4. Maintain Data Integrity

- Enforces rules on data
- Example: Date column only stores valid dates

5. Enable Proper Operations

- SQL operations depend on data types
- Example:
 - You can **add numbers**
 - You cannot **add text to date**

Below are the different type of data types used inn SQL Server:



Here is the detailed information of each datatype, with description, lower limit and upper limit along with the memory utilised by each data type in SQL server.

Introduction to Constraints in SQL Server:

What are Constraints:

Constraints are rules enforced on table columns to maintain data integrity, accuracy, and consistency.

In Microsoft SQL Server, constraints are rules applied on table columns to ensure that the data stored is valid, accurate, and consistent.

👉 Think of constraints as data validation rules at the database level.

Uses of Constraints:

Constraints help to:

- Prevent invalid data entry
- Enforce business rules (e.g., salary > 0)
- Maintain relationships between tables
- Ensure uniqueness (e.g., email, ID)
- Avoid NULLs where data is mandatory

Types of Constraints

1. PRIMARY KEY

Definition:

A column (or combination) that uniquely identifies each row in a table.

- No duplicates
- No NULL values

Example:

EMP_ID INT CONSTRAINT PK_EMP PRIMARY KEY

Use Case:

- Each employee must have a unique ID
- Used as a reference in other table

2. NOT NULL

Definition:

Ensures a column cannot store NULL values

Example:

```
EMP_NAME VARCHAR(100) NOT NULL
```

Use Case:

Employee name must always be present
Joining date (EMP_DOJ) should not be empty

3. UNIQUE

Definition:

Ensures all values in a column are distinct (no duplicates)

Example:

```
EMP_MAIL VARCHAR(150) CONSTRAINT UQ_EMP_MAIL UNIQUE
```

Use Case:

No two employees should have the same email
Mobile numbers should be unique

4. CHECK

Definition:

Restricts values based on a condition

Example:

```
EMP_GENDER CHAR(1)  
CONSTRAINT CHK_GENDER CHECK (EMP_GENDER IN ('M','F','O'))  
EMP_SAL DECIMAL(10,2)  
CONSTRAINT CHK_SAL CHECK (EMP_SAL > 0)
```

Use Case:

Gender must be valid (M/F/O)
Salary must be greater than zero

5. DEFAULT

Definition:

Provides a default value if no value is inserted

Example:

```
EMP_LOC VARCHAR(100) DEFAULT 'INDIA'
```

Use Case:

If location is not provided → automatically set to INDIA

6. FOREIGN KEY

Definition:

Creates a relationship between tables (or same table)

Example (Self-reference):

```
CONSTRAINT FK_EMP_MANAGER  
FOREIGN KEY (MANAGER_ID) REFERENCES EMPLOYEE(EMP_ID)
```

Use Case:

MANAGER_ID must exist as an EMP_ID
Maintains manager–employee hierarchy

Create Table with All Constraints:

Example: Create a EMP table with all constraints:

```
CREATE TABLE EMP_INFO
```

```
(
```

```
EMP_ID INT CONSTRAINT EMP_PK PRIMARY KEY,
```

```

EMP_NAME      VARCHAR(100) CONSTRAINT EMP_NN_1 NOT NULL,
EMP_GENDER    CHAR(1)  CONSTRAINT EMP_CHK CHECK (EMP_GENDER = 'M' OR EMP_GENDER = 'F'),
EMP_LOC       VARCHAR(10) CONSTRAINT EMP_NN_2 NOT NULL,
EMP_DOJ       DATE     CONSTRAINT EMP_NN_2 NOT NULL,
EMP_SAL       INT      CONSTRAINT EMP_DEFAULT_1 DEFAULT(10000),
EMP_MAIL      VARCHAR(150) CONSTRAINT EMP_UNIQUE UNIQUE,
MANAGER_ID    INT      CONSTRAINT EMP_FK FOREIGN KEY REFERENCES EMP_INFO(EMP_ID),
EMP_MOBILE    BIGINT   CONSTRAINT EMP_UNIQUE_1 UNIQUE
)

```

Add Constraint to Existing Table:

Example: Add CHECK constraint

```

ALTER TABLE [dbo].[EMP]
ADD CONSTRAINT CHK_SALARY CHECK (EMP_SAL >= 1000);

```

Example: Add UNIQUE constraint

```

ALTER TABLE [dbo].[EMP]
ADD CONSTRAINT UQ_EMAIL UNIQUE (EMP_MAIL);

```

Example: Add FOREIGN KEY

```

ALTER TABLE [dbo].[EMP]
ADD CONSTRAINT FK_MANAGER
FOREIGN KEY (MANAGER_ID) REFERENCES [dbo].[EMP] (EMP_ID);

```

Drop Existing Constraint

Step 1: Find constraint name

```

SELECT name
FROM sys.objects
WHERE type = 'F' OR type = 'PK' OR type = 'UQ';

```

Step 2: Drop constraint

```

ALTER TABLE [dbo].[EMP]
DROP CONSTRAINT FK_MANAGER;

```

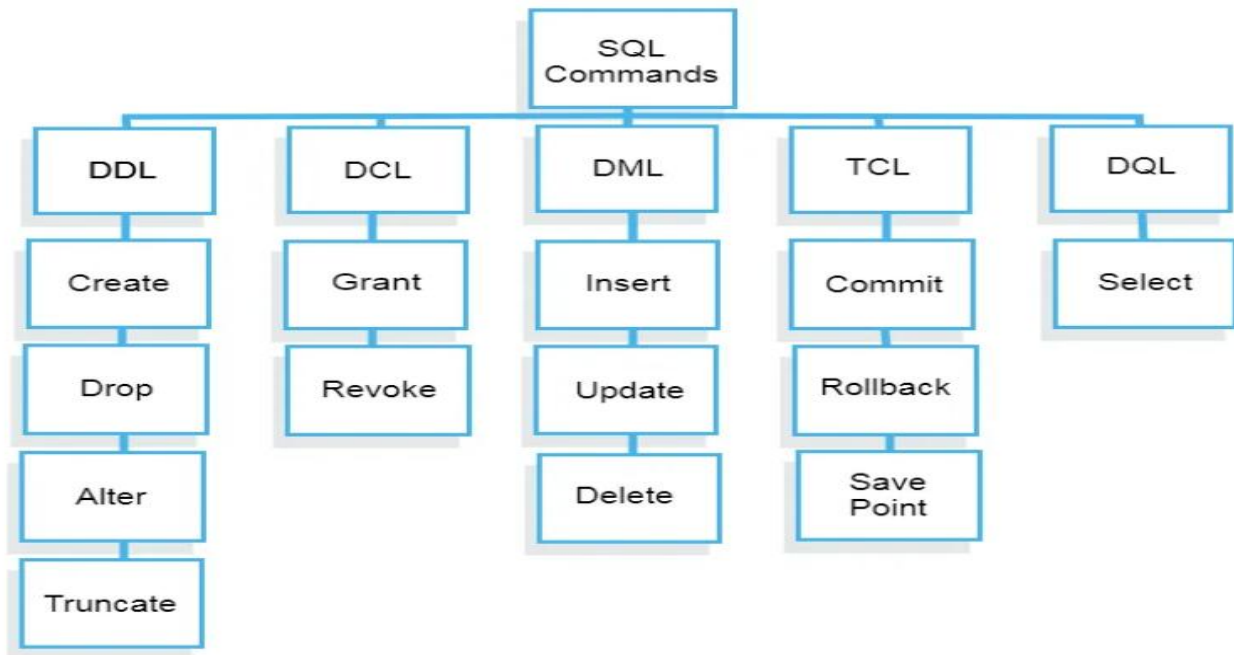
Important Notes

- Always **name your constraints** (easy to manage later)
- Existing data must satisfy constraints before adding them
- A table can have:
 - Only **one PRIMARY KEY**
 - Multiple UNIQUE / CHECK constraints

Introduction to the Different SQL commands:

What Are SQL Commands:

SQL provides a comprehensive sql commands list to communicate with databases. We can think of SQL as an instruction set passed to the database. These instructions, known as SQL language commands, enable us to perform a wide range of actions. For instance, we can use SQL commands to create a database structure, create a table or temporary table, populate the database, retrieve specific information, modify data, and control access and security.



DDL (Data Definition Language) Commands

DDL consists of database-level commands to alter database structures. These DDL commands define, modify, and delete database tables, views, indexes, and database schemas. Moreover, the DDL commands are auto-committed, which ensures that changes are permanently saved in the database and cannot be rolled back to the previous change.

CREATE

This command creates new database objects. An object can be a Table or a Database as below.

Create Database:

Syntax:

```
CREATE DATABASE DATABASE NAME
```

Below is the query to create database:

```
CREATE DATABASE TEST
```

This SQL statement creates a new database **TEST db**.

Create Table:

Syntax:

```
CREATE TABLE TABLE NAME
```

Below is the query to create CUSTOMER Table:

```
USE TEST
```

```
GO
```

```
CREATE TABLE CUSTOMERS
```

```
(
```

```
CUSTOMER
```

```
C_NAME VARCHAR (150),
```

```
C_MOBILE BIGINT,
```

```
C_ADDRESS          VARCHAR (255),
C_DOB              DATE
)
```

This SQL statement creates a new table **CUSTOMERS** in **TEST db**.

Below is the query to create EMP Table:

```
USE TEST
GO
CREATE TABLE EMP
(
EMP_ID              INT,
EMP_NAME            VARCHAR (100),
EMP_GENDER          CHAR (1),
EMP_LOC             VARCHAR (10),
EMP_DOJ             DATE,
EMP_SAL             INT,
EMP_MAIL            VARCHAR (150),
MANAGER_ID          INT,
EMP_MOBILE          BIGINT
)
```

This SQL statement creates a new table **EMP** in **TEST db**.

ALTER

This command modifies the structure of an existing object by adding, altering, or deleting table columns, changing data types, or renaming objects.

It Allows you to:

- ✓ Add new columns or constraints
- ✓ Remove columns or constraints
- ✓ Modify existing columns

Alter Add:

Is used to add new columns or constraints (PRIMARY KEY, FOREIGN KEY, DEFAULT, CHECK) to an existing table.

Add Column:

Syntax:

```
ALTER TABLE TABLENAME ADD COLUMN NAME DATATYPE;
```

Example:

Add Single Column

```
ALTER TABLE [dbo].[EMP] ADD SAL_GRADE VARCHAR (20)
```

```
ALTER TABLE [dbo].[EMP] ADD EMP_ROLE VARCHAR (20)
```

Add Multiple Columns:

```
ALTER TABLE [dbo].[EMP] ADD DEP_ID INT, DEP_NAME VARCHAR (30)
```

Add Constraints:

Syntax:

ALTER TABLE TABLENAME **ADD CONSTRAINT** CONSTRAINT NAME **DEFAULT** 0 **FOR** COLUMN NAME;

Example:

ALTER TABLE [dbo].[EMP] **ADD CONSTRAINT** DF_Salary **DEFAULT** 0 **FOR** [EMP_SAL];

ALTER TABLE [dbo].[EMP] **ADD CONSTRAINT** PK_EMP **PRIMARY KEY** [EMP_ID];

Alter Drop:

Is used to **remove columns or constraints** from an existing table.

Syntax:

ALTER TABLE TABLENAME **DROP COLUMN** COLUMN NAME;

Drop Single Column:

ALTER TABLE [dbo].[EMP] **DROP COLUMN** [EMP_ROLE]

Drop Multiple Column:

ALTER TABLE [dbo].[EMP] **DROP COLUMN** [EMP_ROLE], SAL_GRADE

Drop Constraint:

ALTER TABLE [dbo].[EMP] **DROP CONSTRAINT** DF_Salary

Note:

Data in dropped column is permanently lost

Cannot drop column if:

- It is part of a constraint
- Used in index (without removing dependency)

Alter Alter:

is used to modify an existing column's data type, size, or nullability.

Syntax:

ALTER TABLE TABLENAME **ALTER COLUMN** COLUMN NAME NEW DATATYPE;

Example:

ALTER TABLE [dbo].[EMP] **ALTER COLUMN** [EMP_LOC] **VARCHAR** (15)

ALTER TABLE [dbo].[EMP] **ALTER COLUMN** [EMP_SAL] **DECIMAL** (10,2)

ALTER TABLE [dbo].[EMP] **ALTER COLUMN** [EMP_NAME] **VARCHAR** (150) **NOT NULL**;

DROP

The DROP command is a DDL (Data Definition Language) command used to permanently remove an entire database object from SQL Server, including both its structure and data.

When you use DROP, you are **completely deleting the object itself**, not just the data inside it.

- The object **no longer exists**
- All associated data is **lost permanently**
- Cannot be recovered unless backup exists

Syntax:

DROP OBJECT OBJECT NAME;

Example:

Drop Database:
`DROP DATABASE TEST;`

Drop Table:
`DROP TABLE [dbo].[EMP];`

Drop View:
`DROP VIEW vw_EMP;`

Key Points

- ✓ Cannot be rolled back (auto-commit)
- ✓ Removes both **data + structure**
- ✓ Requires proper permissions
- ✓ Recovery only possible via **backup**

TRUNCATE:

This deletes all the existing data from a table while keeping the original table structure. TRUNCATE is typically faster than DELETE as it doesn't log individual row deletions.

Syntax:

`TRUNCATE TABLE TABLENAME;`

Example:

`TRUNCATE TABLE [dbo].[EMP];`

This removes:

All records from the table

This keeps:

Table structure
Column definitions
Constraints

Key Points:

- ✓ Cannot use WHERE clause (removes all rows only)
- ✓ Cannot be rolled back (in most typical usage)
- ✓ Faster than DELETE (minimal logging)
- ✓ Reset's **identity column** to initial value

Cannot truncate table if:

It has **foreign key constraints**
It is referenced by another table

DML (Data Manipulation Language) Commands

DML consists of SQL main commands to manipulate the data present in the database. For instance, these SQL command lists include commands to insert, modify, and delete data. The DML commands are not auto-committed, ensuring that changes are not permanently saved in the database and we can roll back to the previous state. For instance, we can retrieve back the deleted row by using the **ROLLBACK** statement.

INSERT

Adds new rows (records) of data into a table.

Syntax:

Insert single row:

```
INSERT INTO TABLENAME (COLUMN1, COLUMN2, COLUMN3)
VALUES (VALUE 1, VALUE 2, VALUE 3);
```

Example: The below command adds a new row to the EMP table.

```
INSERT INTO [dbo].[EMP] VALUES (101,'Ashok','M','HYD','2020-01-01',35000, 'Ashok@microsoft.com',101,9905980515)
```

Insert multiple rows:

Syntax:

```
INSERT INTO TABLENAME (COLUMN1, COLUMN2, COLUMN3)
VALUES (VALUE A1, VALUE B1, VALUE C1),
       (VALUE A2, VALUE B2, VALUE C2),
       (VALUE A3, VALUE B3, VALUE C3);
```

Example: The below command adds a new row to the EMP table for columns EMP_ID, EMP_NAME, EMP_GENDER)

```
INSERT INTO [dbo].[EMP] (COLUMN1, COLUMN2, COLUMN3)
VALUES (101,'Ashok', 'M'),
       (102, 'Kumar', 'M'),
       (103, 'Ramya', 'F');
```

Insert Data from Another Table:

Syntax:

```
INSERT INTO DESTINATIONTABLENAME (COLUMN1, COLUMN2)
SELECT COLUMN1, COLUMN2
FROM SOURCETABLE
WHERE CONDITION;
```

Example: This copies rows from an existing table into another table. You can use a WHERE clause to filter the data being inserted.

```
INSERT INTO EMP_001 (COLUMN1, COLUMN2)
SELECT EMP_ID, EMP_NAME
FROM [dbo].[EMP]
```

Create a New Table and Insert Data:

Syntax:

```
SELECT * INTO NEWTABLE
FROM SOURCETABLE
WHERE CONDITION;
```

Example:

This statement creates a new table based on the result set of a SELECT query and inserts the data into it simultaneously. The new table will not have any indexes or constraints defined initially.

```
SELECT * INTO EMP_103 FROM [dbo].[EMP] WHERE 1=1
```

This statement creates a new table based on the existing table structure; data will not be copied to new table.

```
SELECT * INTO EMP_103 FROM [dbo].[EMP] WHERE 1 = 0
```

UPDATE

Modifies existing data within a table based on a specified condition.

Syntax:

```
UPDATE TABLENAME SET COLUMN NAME = VALUE
```

WHERE CONDITION

Example:

This updates existing data in a table. As depicted below, the UPDATE command updates **EMP_NAME** name with ID 110.

```
UPDATE [dbo].[EMP] SET [EMP_NAME] = 'Rahul'
WHERE [EMP_ID] = 110
```

The below update command updates **EMP_SAL** for multiple records.

```
UPDATE [dbo].[EMP] SET [EMP_SAL] = 35000
WHERE [EMP_ID] IN (110,111,112)
```

The below update command updates the column **EMP_MGR_ID with 118** for all the records.

```
UPDATE [dbo].[EMP] SET [EMP_MGR_ID] = 118
```

DELETE

Command used to remove one or more specific rows from a table.

Syntax:

```
DELETE FROM TABLENAME WHERE CONDITION
```

Example:

Selective Deletion: You can use a WHERE clause to specify a condition and delete only the rows that match that condition.

```
DELETE FROM [dbo].[EMP] WHERE [EMP_ID] = 110
```

```
DELETE FROM [dbo].[EMP] WHERE [EMP_LOC] = 'VIZ';
```

Deleting all rows: If you omit the WHERE clause, all rows in the table will be deleted.

```
DELETE FROM [dbo].[EMP]
```

DQL (Data Query Language) commands

DQL command is a subset of SQL commands specifically designed for querying and retrieving data from the database. DQL command (SELECT) performs specific tasks on the data within schema objects and extracts schema relations based on the query passed to it. It utilizes various clauses, functions, and keywords to filter and manipulate the data, thus enhancing its functionality.

SELECT (Retrieve Data)

This command retrieves the specified column (name) from the table:

Syntax:

```
SELECT * FROM TABLE NAME
SELECT COLUMN 1, COLUMN 2, COLUMN 3 FROM TABLE NAME
```

Example:

Select all columns: Use the asterisk (*) to retrieve all columns from a table.

```
SELECT * FROM [dbo].[EMP]
```

Select specific columns: List the desired column names, separated by commas, for clarity and performance.

```
SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL], [EMP_MAIL_ID] FROM [dbo].[EMP]
```

Use aliases: Assign temporary names to columns or tables for easier readability, especially with calculations or multiple tables.

```
SELECT SUM([EMP_SAL]) AS TOTAL_SAL FROM [dbo].[EMP]
```

```
SELECT COUNT(*) AS TOTAL_EMP FROM [dbo].[EMP]
```

However, using * is usually not recommended as it increases the amount of data transferred by including all columns, even those that aren't required. This can impact query performance. Instead, it's better to explicitly list the columns you need:

The SELECT statement is usually used with other clauses and functions like DISTINCT, AVG(), WHERE, ORDER BY, GROUP BY, and HAVING to extract data and aggregate, filter, sort, or group it to return one or more columns.

DCL (Data Control Language) commands:

DCL is the subset of SQL used to control access to data stored in a database. It deals with granting and revoking permissions (rights) that determine which users can perform which operations — such as SELECT, INSERT, UPDATE, or DELETE — on database objects like tables and views. DCL is primarily concerned with security and access control rather than with the data or structure itself.

Command	Purpose
GRANT	To give permissions on specific operations on a table to a user.
REVOKE	To remove permissions on specific operations on a table from a user.

Prerequisites: Creating a Login and a User

Before permissions can be granted, the target user must exist at both the server level (a Login) and the database level (a User) so that SQL Server can authenticate and authorize them.

Create a Login

Definition: A login is a server-level security principal that allows a user or application to connect to the SQL Server instance. It is authenticated at the server, not tied to any specific database.

Syntax:

```
CREATE LOGIN <login_name>  
WITH PASSWORD = '<password>'
```

Example:

```
CREATE LOGIN VDA8  
WITH PASSWORD = 'VDA@888'
```

Create a User for the Login

Definition: A database user is a database-level security principal mapped to a server login. It determines what the login is permitted to do inside a specific database and is the object that GRANT/REVOKE statements actually target.

Syntax:

```
CREATE USER <user_name>  
FOR LOGIN <login_name>
```

Example:

```
CREATE USER VDA8  
FOR LOGIN VDA8
```

GRANT:

Definition: GRANT is used to give a user specific permissions (such as SELECT, INSERT, UPDATE, DELETE) on a database object like a table, allowing that user to perform the specified operation(s).

Syntax:

```
GRANT <permission(s)> ON <table_name> TO <user_name>
```

Example:

```
GRANT SELECT ON EMP TO VDA8  
GRANT INSERT, UPDATE, DELETE ON EMP TO VDA8
```

Common Use Cases:

- ✓ Giving a reporting/analyst user read-only (SELECT) access to a table without allowing changes to the data.
- ✓ Allowing an application service account to INSERT, UPDATE, and DELETE records in an operational table.
- ✓ Onboarding a new team member with exactly the access their role requires, following the principle of least privilege.
- ✓ Enabling a specific role (e.g., HR) to manage only the tables relevant to their department.

REVOKE

Definition: REVOKE is used to remove permissions that were previously granted to a user, taking away their ability to perform the specified operation(s) on a database object.

Syntax:

```
REVOKE <permission(s)> ON <table_name> FROM <user_name>
```

Example:

```
REVOKE SELECT ON EMP FROM VDA8  
REVOKE INSERT, UPDATE, DELETE ON EMP FROM VDA8
```

Common Use Cases:

- ✓ Removing access immediately when an employee changes roles or leaves the organization.
- ✓ Temporarily restricting write access to a table during a maintenance window or audit.
- ✓ Correcting over-provisioned permissions that were granted in error.
- ✓ Enforcing periodic access reviews as part of a security/compliance policy.

TCL (Transaction Control Language) commands:

Transaction Control Language (TCL) commands in SQL are used to manage transactions and ensure data integrity by either permanently saving changes or undoing them:

TCL is the subset of SQL used to manage transactions within a database, giving control over when changes made by DML statements (INSERT, UPDATE, DELETE) are finalized or undone. TCL commands work hand-in-hand with the ACID properties described above.

Sample Setup Used for the Transaction Examples

The following table is used throughout this document (in the TCL section) to demonstrate transactions, COMMIT, ROLLBACK, and SAVE POINT in a way that illustrates ACID behavior in practice — a simple funds-transfer scenario between account holders.

Create and populate the table:

```
CREATE TABLE TRANS
```

```
(  
    NAME VARCHAR(50),  
    BALANCE INT  
)
```

```
INSERT INTO TRANS VALUES ('KUMAR', 100)
```

```
INSERT INTO TRANS VALUES ('RAJESH', 100)
```

```
INSERT INTO TRANS VALUES ('PRIYA', 100)
```

```
INSERT INTO TRANS VALUES ('RAMYA', 100)
```

Command	Purpose
BEGIN TRANSACTION	Marks the starting point of an explicit transaction.
COMMIT	Permanently saves all changes made during the transaction.
ROLLBACK	Undoes all changes made during the transaction (or back to a save point).
SAVE POINT	Marks an intermediate point within a transaction that a rollback can target.

BEGIN TRANSACTION: This command explicitly starts a new transaction in SQL Server. Changes made after this point are temporary until either committed or rolled back.

COMMIT:

Definition: COMMIT permanently saves all the changes made by the DML statements within the current transaction to the database. Once committed, the changes cannot be rolled back.

Syntax:

```
BEGIN TRANSACTION
```

```
<SQL statements>
```

```
COMMIT TRANSACTION
```

Example — transferring 50 from KUMAR to RAMYA:

```
BEGIN TRANSACTION
```

--> DEBIT 50 FROM KUMAR

UPDATE TRANS SET BALANCE = BALANCE - 50 WHERE NAME = 'KUMAR'

--> CREDIT 50 TO RAMYA

UPDATE TRANS SET BALANCE = BALANCE + 50 WHERE NAME = 'RAMYA'

COMMIT TRANSACTION

Common Use Cases:

- ✓ Finalizing a successful funds transfer once both the debit and credit updates succeed.
- ✓ Saving a multi-step order (order header + order lines + stock update) only once every step has succeeded.
- ✓ Persisting batch data loads after validation confirms the data is correct.

ROLLBACK

Definition: ROLLBACK undoes all the changes made by the DML statements within the current transaction, returning the database to the state it was in before the transaction began (or before the referenced savepoint)

Syntax:

BEGIN TRANSACTION

<SQL statements>

ROLLBACK TRANSACTION

Example — same transfer, but undone:

BEGIN TRANSACTION

--> DEBIT 50 FROM KUMAR

UPDATE TRANS SET BALANCE = BALANCE - 50 WHERE NAME = 'KUMAR'

--> CREDIT 50 TO RAMYA

UPDATE TRANS SET BALANCE = BALANCE + 50 WHERE NAME = 'RAMYA'

ROLLBACK TRANSACTION

Common Use Cases:

- ✓ Reversing a transaction when one of the steps fails (e.g., the credit update fails after the debit succeeded).
- ✓ Cancelling changes when a business rule validation fails mid-transaction.
- ✓ Undoing an entire batch of changes if any single record in the batch is invalid.

SAVE POINT

Definition: A savepoint marks an intermediate point inside a transaction. A subsequent ROLLBACK can target the savepoint name to undo only the statements executed after it, while keeping the earlier statements in the transaction intact (until the transaction is eventually committed or fully rolled back).

Syntax:

BEGIN TRANSACTION

<SQL statements>

SAVE TRANSACTION <savepoint_name>

<more SQL statements>

ROLLBACK TRANSACTION <savepoint_name>

Example — transfer 50 from RAMYA to KUMAR (kept), then a second transfer from KUMAR to PRIYA (undone via the savepoint):

BEGIN TRANSACTION

--> DEBIT 50 FROM RAMYA

UPDATE TRANS SET BALANCE = BALANCE - 50 WHERE NAME = 'RAMYA'

--> CREDIT 50 TO KUMAR

UPDATE TRANS SET BALANCE = BALANCE + 50 WHERE NAME = 'KUMAR'

SAVE TRANSACTION STAGE1

--> DEBIT 50 FROM KUMAR

UPDATE TRANS SET BALANCE = BALANCE - 50 WHERE NAME = 'KUMAR'

--> CREDIT 50 TO PRIYA

UPDATE TRANS SET BALANCE = BALANCE + 50 WHERE NAME = 'PRIYA'

ROLLBACK TRANSACTION STAGE1

Common Use Cases:

- ✓ Long, multi-stage transactions where only a later stage needs to be undone if it fails, without discarding earlier successful stages.
- ✓ Complex batch processes where partial rollback to a known-good checkpoint is preferable to restarting the whole transaction.
- ✓ Testing 'what-if' changes within a transaction, with the option to discard just the most recent set of statements.

TCL commands typically work in conjunction with DML (Data Manipulation Language) commands like INSERT, UPDATE, and DELETE. DDL commands (like CREATE or DROP) often trigger an automatic commit and cannot be rolled back using TCL commands

Differences Between SQL Command Types

Feature	DDL	DML	DQL	DCL	TCL
Full Form	Data Definition	Data Manipulation	Data Query	Data Control	Transaction Control
Purpose	Structure	Data changes	Data retrieval	Security	Transaction management
Commands	CREATE, ALTER, DROP	INSERT, UPDATE, DELETE	SELECT	GRANT, REVOKE	COMMIT, ROLLBACK
Works On	Schema	Data	Data	Permissions	Transactions
Rollback	✗ No	✓ Yes	✗ No	✗ No	✓ Yes
Auto Commit	✓ Yes	✗ No	✗ No	✓ Yes	✗ No

Important Differences between (DELETE, TRUNCATE, DROP)

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows	All rows	Entire table
WHERE clause	✓ Yes	✗ No	✗ No
Rollback	✓ Yes	✗ No	✗ No

Structure	Remains	Remains	Removed
-----------	---------	---------	---------

Introduction to Functions:

SQL Server functions are routines that accept parameters, perform calculations or data manipulations, and return a value. There are two main categories: **built-in (system) functions** and **user-defined functions (UDFs)**

Types of Functions

1. **System Functions (Built-in)**
 - Provided by SQL Server
 - Example: SUM(), GETDATE(), LEN()
2. **User-Defined Functions (UDFs)**
 - Created by users
 - Scalar or Table-valued functions

Aggregate Functions:

Aggregate functions are built-in functions that perform calculations on a set of values and return a single summarized result.

AVG(): Calculates the average.

COUNT(): Returns the number of rows.

MAX(): Returns the maximum value.

MIN(): Returns the minimum value.

SUM(): Calculates the sum of values.

COUNT:

Returns the **number of rows**

Count all rows

```
SELECT COUNT(*) AS TOTAL_EMPLOYEES FROM [dbo].[EMP];
```

Count non-null values

```
SELECT COUNT([EMP_SAL]) AS TOTAL_EMPLOYEES FROM [dbo].[EMP];
```

SUM:

Returns the **total sum of a numeric column**.

Example:

```
SELECT SUM([EMP_SAL]) AS TOTAL_SALARY FROM [dbo].[EMP];
```

AVG:

Returns the average value of a numeric column(non – NULL values).

Example:

```
SELECT AVG([EMP_SAL]) AS AVERAGE_SALARY FROM [dbo].[EMP];
```

MIN:

Returns the smallest value in a column.

Example:

```
SELECT MIN([EMP_SAL]) AS LOWEST_SALARY FROM [dbo].[EMP];
```

MAX:

Returns the largest value in a column.

Example:

```
SELECT MAX([EMP_SAL]) AS HIGHEST_SALARY FROM [dbo].[EMP];
```

Below SQL statement will return, Count of Employees, Minimum Salary, Maximum Salary, Average Salary and Total Salary in a row.

```
SELECT  
COUNT(*) AS COUNT_OF_EMPLOYEES,  
MIN([EMP_SAL]) AS LOWEST_SALARY,  
MAX([EMP_SAL]) AS HIGHEST_SALARY,  
AVG([EMP_SAL]) AS AVERAGE_SALARY,  
SUM([EMP_SAL]) AS TOTAL_SAL  
FROM [dbo].[EMP]
```

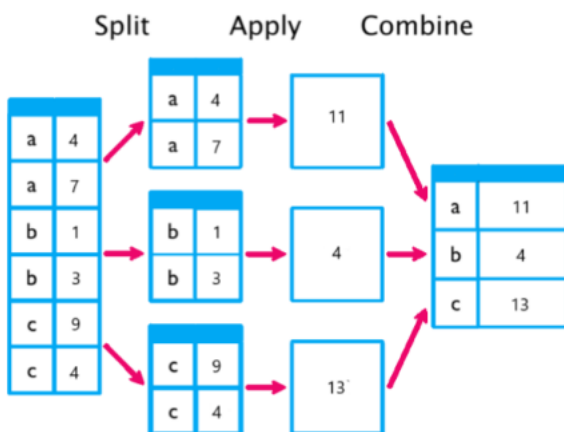
Using Aggregate Functions with **GROUP BY**

GROUP BY is used to arrange the identical data into groups so that aggregate functions can be applied to each group.

In simple words: "Split data into buckets → Apply calculations on each bucket"

The SQL Group By statement uses the split-apply-combine strategy.

- ✓ Split: The different groups are split with their values.
- ✓ Apply: The aggregate function is applied to the values of these groups.
- ✓ Combine: The values are combined in a single row.



The **SUM()** aggregate function, which results in the arithmetic sum of the rows' values, has been applied to the groups in the above illustration.

COUNT With GROUP BY:

Example: Below query will result No of employees for each location.

```
SELECT [EMP_LOC], COUNT(*) AS TOTAL_EMPLOYEES  
FROM [dbo].[EMP]  
GROUP BY [EMP_LOC];
```

Key Points:

- ✓ COUNT(*) → counts all rows
- ✓ COUNT(column) → ignores NULLs

SUM With GROUP BY:

Example: Below query will result Total Salary for each location

```
SELECT [EMP_LOC], SUM([EMP_SAL]) AS TOTAL_SALARY  
FROM [dbo].[EMP]
```

GROUP BY [EMP_LOC];

MIN With GROUP BY:

Example: Below query will result Minimum Salary for each location

```
SELECT [EMP_LOC], MIN([EMP_SAL]) AS MINIMUM_SALARY
FROM [dbo].[EMP]
GROUP BY [EMP_LOC];
```

MAX With GROUP BY:

Example: Below query will result Maximum Salary for each location

```
SELECT [EMP_LOC], MAX([EMP_SAL]) AS HIGHEST_SALARY
FROM [dbo].[EMP]
GROUP BY [EMP_LOC];
```

AVG With GROUP BY:

Example: Below query will result Average Salary for each location

```
SELECT [EMP_LOC], AVG([EMP_SAL]) AS AVERAGE_SALARY
FROM [dbo].[EMP]
GROUP BY [EMP_LOC];
```

Key Points:

- Columns in SELECT must be in GROUP BY
- More columns = more detailed output
- Fewer columns = higher-level summary
- Handle NULL's Properly
 - ✓ SUM, AVG → ignore NULL
 - ✓ COUNT(column) → ignores NULL
 - ✓ COUNT(*) → counts all rows

DATE FUNCTIONS

SQL Server provides a rich set of built-in date functions that allow you to extract, manipulate, and calculate date and time values. The functions covered in this section are: GETDATE, YEAR, MONTH, DAY, DATEPART, DATENAME, DATEADD, DATEDIFF, EOMONTH, and DATETRUNC.

GETDATE:

Definition: Returns the current system date and time of the SQL Server as a DATETIME value.

Syntax:

```
SELECT GETDATE()
```

Example:

```
SELECT GETDATE()
```

Result: 2026-03-23 17:26:34.973

GETDATE() is commonly used as a base reference for all other date calculations. It includes both date and time components.

YEAR() / MONTH() / DAY():

Definition: These three functions extract a specific numeric component from a given date value.

Syntax:

```
SELECT YEAR(date_value)
```

```
SELECT MONTH(date_value)
```

```
SELECT DAY(date_value)
```

Examples:

```
SELECT YEAR(GETDATE()) -- Result: 2026
```

```
SELECT MONTH(GETDATE()) -- Result: 3
```

```
SELECT DAY(GETDATE()) -- Result: 23
```

These are shorthand functions. They return integer values and are equivalent to using **DATEPART** with YYYY, MM, and DD respectively.

DATEPART():

Definition: Extracts a specific part of a date as an integer value. Supports a wide range of intervals including year, quarter, month, week, day, hour, minute, and more.

Syntax:

```
SELECT DATEPART(interval, date_value)
```

Examples:

```
SELECT DATEPART(YYYY, GETDATE()) -- Result: 2026 (Year)
```

```
SELECT DATEPART(MM, GETDATE()) -- Result: 3 (Month number)
```

```
SELECT DATEPART(QQ, GETDATE()) -- Result: 1 (Quarter number)
```

```
SELECT DATEPART(DD, GETDATE()) -- Result: 23 (Day of month)
```

```
SELECT DATEPART(DW, GETDATE()) -- Result: 2 (Day of week, 1=Sun)
```

```
SELECT DATEPART(WK, GETDATE()) -- Result: 13 (Week number)
```

```
SELECT DATEPART(DY, GETDATE()) -- Result: 82 (Day of year)
```

```
SELECT DATEPART(HH, GETDATE()) -- Result: 17 (Hour)
```

```
SELECT DATEPART(MI, GETDATE()) -- Result: 26 (Minute)
```

Available Interval Parts:

Interval Code	Full Name	Returns	Example Result
YYYY	Year	4-digit year	2026
MM	Month	Month number (1-12)	3
QQ	Quarter	Quarter number (1-4)	1
DD	Day	Day of month (1-31)	23
DW	Day of Week	Day of week (1=Sunday)	2
WK	Week	Week number of year	13
DY	Day of Year	Day of year (1-366)	82
HH	Hour	Hour (0-23)	17
MI	Minute	Minute (0-59)	26
SS	Second	Second (0-59)	34
MS	Millisecond	Millisecond (0-999)	973

DATENAME():

Definition: Similar to **DATEPART** but returns the name of the date part as a character string instead of an integer. Most useful for returning month names and day names.

Syntax

`SELECT DATENAME(interval, date_value)`

Examples

`SELECT DATENAME(MONTH, GETDATE())` -- Result: 'March'

`SELECT DATENAME(DW, GETDATE())` -- Result: 'Monday'

`SELECT DATEPART(MM, GETDATE())` -- Result: 3 (numeric - `DATEPART`)

`SELECT DATENAME(MM, GETDATE())` -- Result: 'March' (text - `DATENAME`)

Use `DATENAME` when you need labels for reports (e.g. 'March', 'Monday') instead of numbers.

DATEADD():

Definition: Adds or subtracts a specified number of date/time intervals to a given date. Use a negative number to go back in time.

Syntax:

`SELECT DATEADD(interval, number, date_value)`

Examples:

`SELECT DATEADD(DAY, 1, GETDATE())` -- Result: Tomorrow

`SELECT DATEADD(DAY, -1, GETDATE())` -- Result: Yesterday

`SELECT DATEADD(MONTH, 1, GETDATE())` -- Result: One month from now

`SELECT DATEADD(MONTH, -1, GETDATE())` -- Result: One month ago

`SELECT DATEADD(MONTH, -1, '2026-03-30')` -- Result: 2026-02-28

`SELECT DATEADD(YEAR, 1, GETDATE())` -- Result: Same day next year

Note: When adding months to a date like 2026-03-30, SQL Server automatically adjusts to the last valid day of the resulting month (e.g. 2026-02-28).

DATEDIFF():

Definition: Calculates the difference between two date values for a specified interval. Returns an integer representing the count of interval boundaries crossed.

Syntax:

`SELECT DATEDIFF(interval, start_date, end_date)`

Examples:

`SELECT DATEDIFF(YYYY, '2005-10-12', GETDATE())` -- Result: 20 (years)

`SELECT DATEDIFF(MM, '2025-01-01', GETDATE())` -- Result: 14 (months)

`SELECT DATEDIFF(DD, '2026-01-01', GETDATE())` -- Result: 81 (days)

`DATEDIFF` is perfect for calculating age, tenure, number of days between orders, time since last login, and more.

EOMONTH():

Definition: Returns the last day of the month for a given date. An optional offset parameter allows you to shift forward or backward by a number of months.

Syntax:

`SELECT EOMONTH(date_value, month_offset)`

Examples:

`SELECT EOMONTH(GETDATE(), 0)` -- Result: 2026-03-31 (end of current month)

`SELECT EOMONTH(GETDATE(), 1)` -- Result: 2026-04-30 (end of next month)

`SELECT EOMONTH(GETDATE(), -1)` -- Result: 2026-02-28 (end of previous month)

Practical Use Cases:

Start of current month:

`SELECT DATEADD(DAY, 1, EOMONTH(GETDATE(), -1))`

Result: 2026-03-01

End of current month:

```
SELECT EOMONTH(GETDATE(), 0)
```

Result: 2026-03-31

Start of current year:

```
SELECT DATEADD(DAY, 1, EOMONTH(GETDATE(), -MONTH(GETDATE())))
```

Result: 2026-01-01

End of current year:

```
SELECT EOMONTH(GETDATE(), 12 - MONTH(GETDATE()))
```

Result: 2026-12-31

Start of current week (Sunday):

```
SELECT DATEADD(DAY, -(DATEPART(DW, GETDATE()) - 1), GETDATE())
```

Result: 2026-03-22

End of current week (Saturday):

```
SELECT DATEADD(DAY, (7 - DATEPART(DW, GETDATE())) - 1, GETDATE())
```

Result: 2026-03-28

DATETRUNC():

Definition: Truncates a date to the beginning of the specified interval. For example, truncating to MONTH returns the first day of that month, truncating to YEAR returns January 1st of that year.

Syntax:

```
SELECT DATETRUNC(interval, date_value)
```

Examples:

```
SELECT DATETRUNC(YYYY, GETDATE())      -- Result: 2026-01-01 (start of year)
SELECT DATETRUNC(QQ, GETDATE())         -- Result: 2026-01-01 (start of quarter Q1)
SELECT DATETRUNC(QQ, '2026-06-10')      -- Result: 2026-04-01 (start of Q2)
SELECT DATETRUNC(MM, GETDATE())         -- Result: 2026-03-01 (start of month)
SELECT DATETRUNC(WEEK, GETDATE())       -- Result: 2026-03-22 (start of week)
```

Deriving Period Start and End Dates Using **DATETRUNC**

DATETRUNC combined with **DATEADD** gives you clean period boundary logic:

Year Start Date (via EOMONTH):

```
SELECT DATEADD(DAY, 1, EOMONTH(GETDATE(), -MONTH(GETDATE())))
```

Year End Date:

```
SELECT DATEADD(DAY, -1, DATETRUNC(YYYY, DATEADD(YEAR, 1, GETDATE())))
```

Result: 2026-12-31

Quarter End Date:

```
SELECT DATEADD(DAY, -1, DATETRUNC(QQ, DATEADD(QQ, 1, GETDATE())))
```

Result: 2026-03-31

Month End Date:

```
SELECT DATEADD(DAY, -1, DATETRUNC(MM, DATEADD(MM, 1, GETDATE())))
```

Result: 2026-03-31

Week End Date:

```
SELECT DATEADD(DAY, -1, DATETRUNC(WEEK, DATEADD(WEEK, 1, GETDATE())))
```

Result: 2026-03-28

Common Date Format Patterns for Reporting:

Target Format	SQL Expression	Example Output
'2026-01'	<code>CONCAT(YEAR([Order Date]), '-', RIGHT('0'+CAST(MONTH([Order Date]) AS VARCHAR),2))</code>	2026-01
'2026-March'	<code>CONCAT(YEAR([Order Date]), '-', DATENAME(MONTH, [Order Date]))</code>	2026-March
'2026-Mar'	<code>CONCAT(YEAR([Order Date]), '-', LEFT(DATENAME(MONTH,[Order Date]),3))</code>	2026-Mar
'2026-Q1'	<code>CONCAT(YEAR([Order Date]), '-Q', DATEPART(QQ, [Order Date]))</code>	2026-Q1

STRING FUNCTIONS:

SQL Server string functions let you manipulate, search, and transform text data. The functions covered in this section are: LEFT, RIGHT, SUBSTRING, CHARINDEX, LEN, UPPER, LOWER, REPLACE, STUFF, LTRIM, and RTRIM.

All examples below use the sample string: 'Samsung Galaxy Note3'

Character Position (Index) Reference — 'Samsung Galaxy Note3'

Position (Index)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Character	S	a	m	s	u	n	G		G	a	l	a	x	y		N	o	t	e	3

LEFT():

Definition: Extracts a specified number of characters from the LEFT (beginning) of a string.

Syntax:

`SELECT LEFT(string, number_of_characters)`

Example:

`SELECT LEFT('Samsung Galaxy Note3', 10)`

Result: 'Samsung Ga'

`SELECT LEFT('Samsung Galaxy Note3', 7)`

Result: 'Samsung'

RIGHT()

Definition: Extracts a specified number of characters from the RIGHT (end) of a string.

Syntax:

`SELECT RIGHT(string, number_of_characters)`

Example:

`SELECT RIGHT('Samsung Galaxy Note3', 5)`

Result: 'Note3'

`SELECT RIGHT('Samsung Galaxy Note3', 1)`

Result: '3'

LEN()

Definition: Returns the total number of characters in a string, excluding trailing spaces.

Syntax:

`SELECT LEN(string)`

Example:

`SELECT LEN('Samsung Galaxy Note3')`

Result: 20

`SELECT LEN('Hello ') -- Trailing spaces are ignored`

Result: 5

CHARINDEX():

Definition: Returns the starting position (integer) of a search string within a main string. Returns 0 if not found.

Syntax:

CHARINDEX(search_string, main_string, start_position)

Example 1: Find space position in a product name

```
SELECT CHARINDEX(' ', 'Samsung Galaxy Note3', 1)
```

Example 2: Extract email parts from [dbo].[EMP]

```
SELECT
```

```
[EMP_MAIL],
```

```
CHARINDEX('@', [EMP_MAIL], 1) AS [@POSITION1],      -- position of @
```

```
CHARINDEX('.', [EMP_MAIL], 1) AS [.POSITION2],      -- position of first '.'
```

```
LEN([EMP_MAIL]) AS LENGTH,
```

```
LEFT([EMP_MAIL], 11) AS E_NAME,
```

```
-- Dynamic extraction using CHARINDEX:
```

```
LEFT([EMP_MAIL], CHARINDEX('@', [EMP_MAIL], 1) - 1) AS E_NAME1,
```

```
SUBSTRING([EMP_MAIL], CHARINDEX('@', [EMP_MAIL], 1) + 1, (CHARINDEX('.', [EMP_MAIL], 1) - CHARINDEX('@', [EMP_MAIL], 1)) - 1) AS Company
```

```
FROM [dbo].[EMP]
```

SUBSTRING()

Definition: Extracts a portion of a string starting at a specified position for a given length. This is the most flexible of the text extraction functions.

Syntax:

```
SELECT SUBSTRING(string, start_position, number_of_characters)
```

Examples:

```
SELECT SUBSTRING('Samsung Galaxy Note3', 9, 6)
```

Starts at position 9 ('G'), extracts 6 chars

Result: 'Galaxy'

```
SELECT SUBSTRING('Samsung Galaxy Note3', 1, 7)
```

Result: 'Samsung'

```
SELECT SUBSTRING('Samsung Galaxy Note3', 16, 5)
```

Result: 'Note3'

UPPER() & LOWER()

Definition: UPPER converts all characters in a string to uppercase. LOWER converts all characters to lowercase.

Syntax:

```
SELECT UPPER(string)
```

```
SELECT LOWER(string)
```

Examples:

```
SELECT UPPER('Samsung Galaxy Note3')
```

Result: 'SAMSUNG GALAXY NOTE3'

```
SELECT LOWER('Samsung Galaxy Note3')
```

Result: 'samsung galaxy note3'

REPLACE():

Definition: Replaces all occurrences of a specified substring within a string with a new substring.

Syntax:

```
REPLACE(original_string, search_string, replacement_string)
```

Examples:

-- Replace model number

```
SELECT REPLACE('Samsung Galaxy Note3', 'Note3', 'S24')
```

-- Remove all spaces

```
SELECT REPLACE('Samsung Galaxy Note3', ' ', '')
```

STUFF():

Definition: Deletes a specified number of characters from a string and inserts a new substring at the specified start position.

Use Case: Surgical replacement where you know the exact character position and length to replace — for example, overwriting a known segment of a product name.

Syntax

```
STUFF(original_string, start_position, length_to_delete, new_string)
```

Examples

-- Delete 4 chars from position 16, insert 'S24'

```
SELECT STUFF('Samsung Galaxy Note3', 16, 4, 'S24')
```

-- Delete 5 chars from position 16, insert 'S24'

```
SELECT STUFF('Samsung Galaxy Note3', 16, 5, 'S24')
```

REPLICATE():

Definition: Repeats a string expression a specified number of times.

Syntax

```
REPLICATE(string_expression, integer_expression)
```

Example

```
SELECT REPLICATE('PBI', 3)
```

LTRIM(), RTRIM(), TRIM():

Definition: Remove leading spaces (LTRIM), trailing spaces (RTRIM), or both (TRIM) from a string. Note: TRIM removes only leading and trailing spaces; it does NOT remove internal whitespace.

Syntax

```
LTRIM(character_expression)
```

```
RTRIM(character_expression)
```

TRIM(character_expression)

Examples

```
SELECT LTRIM(' Samsung Galaxy Note3')
```

```
SELECT RTRIM('Samsung Galaxy Note3 ')
```

```
SELECT TRIM(' Samsung Galaxy Note3 ')
```

Note: **TRIM** removes leading/trailing spaces only internal spaces remain untouched.

These are commonly used to normalize text data before comparisons, as SQL comparisons can be case-sensitive depending on collation settings.

String functions quick reference.

Function	Purpose	Syntax	Example Output
LEFT()	Extract from start	LEFT(str, n)	LEFT('Samsung...', 7) = 'Samsung'
RIGHT()	Extract from end	RIGHT(str, n)	RIGHT('Samsung...', 5) = 'Note3'
LEN()	Count characters	LEN(str)	LEN('Samsung...') = 20
SUBSTRING()	Extract from middle	SUBSTRING(str, pos, n)	SUBSTRING(..., 9, 6) = 'Galaxy'
UPPER()	Convert to uppercase	UPPER(str)	UPPER('hello') = 'HELLO'
LOWER()	Convert to lowercase	LOWER(str)	LOWER('HELLO') = 'hello'
REPLACE()	Replace occurrences	REPLACE(str, old, new)	REPLACE('abc','b','x') = 'axc'
STUFF()	Insert/delete in string	STUFF(str, pos, len, new)	STUFF('abcde',2,3,'XY') = 'aXYe'
CHARINDEX()	Find position	CHARINDEX(find, str)	CHARINDEX('G','Samsung...') = 9
LTRIM()	Remove leading spaces	LTRIM(str)	LTRIM(' Hi') = 'Hi'
RTRIM()	Remove trailing spaces	RTRIM(str)	RTRIM('Hi ') = 'Hi'
TRIM()	Remove leading & trailing spaces	RTRIM(str)	RTRIM(' Hi ') = 'Hi'

Data Type Conversion: CAST and CONVERT

CAST and CONVERT both change a value from one data type to another (e.g. an INT to a VARCHAR, or a string to a DATE). They're needed constantly in SQL because operations like string concatenation (+) require every operand to be a compatible type — you cannot directly concatenate a number or a date onto a string without converting it first.

CAST:

Definition:

Converts an expression from one data type to another. CAST is part of the ANSI SQL standard, so it works the same way across SQL Server, PostgreSQL, MySQL, Oracle, etc.

Syntax:

CAST(value **AS** target_data_type)

Example — building a "Year-Quarter" label such as "2026-Q3" from the current date:

```
SELECT CAST(YEAR(GETDATE()) AS VARCHAR(10)) + '-Q'  
      + CAST(DATEPART(QQ, GETDATE()) AS VARCHAR(10))
```

Logic: GETDATE() returns today's date/time. YEAR(...) extracts the year as an INT, and DATEPART(QQ, ...) extracts the quarter number (1-4) as an INT. Both are numeric, but they need to be joined onto text with '-Q' using the + operator, which SQL Server only allows between matching types — so each numeric value is CAST to VARCHAR(10) first, then concatenated.

CONVERT

Definition:

Converts an expression from one data type to another, but is SQL Server-specific (T-SQL only) and adds an optional third argument — a style code — that controls the exact text format used for date/time and numeric conversions.

Syntax:

```
CONVERT(target_data_type, value [, style])
```

Example — the same "Year-Quarter" label, built with CONVERT instead of CAST:

```
SELECT CONVERT(VARCHAR(10), YEAR(GETDATE())) + '-Q'  
      + CONVERT(VARCHAR(10), DATEPART(QQ, GETDATE()))
```

Applied to a real table column instead of GETDATE(), alongside a CONCAT-based equivalent, both approaches produce the same result:

```
SELECT [Order_Date],  
      CONCAT(YEAR([Order_Date]), '-Q', DATEPART(QQ, [Order_Date])),  
      CONVERT(VARCHAR(10), YEAR([Order_Date])) + '-Q'  
      + CONVERT(VARCHAR(10), DATEPART(QQ, [Order_Date]))  
FROM [dbo].[Orders]
```

Logic: CONCAT() automatically converts each argument to text and joins them, so it needs no explicit CAST/CONVERT at all — it's the simplest option when you don't need to control the exact output format. The CONVERT version does the same job more explicitly. Both are shown side by side to demonstrate that they return an identical "YYYY-Qn" string; CONCAT is shorter to write, while CONVERT gives more control (via style codes) when formatting matters.

CONVERT's Style Code — Reformatting Dates

CONVERT's real advantage over CAST is the style code, used to reformat dates. Style 103 means British/French format: dd/mm/yyyy.

```
SELECT GETDATE()           -- default: YYYY-MM-DD
```

```
SELECT CONVERT(VARCHAR(20), GETDATE(), 103) -- DD/MM/YYYY
```

```
SELECT EMP_DOJ, CONVERT(VARCHAR(20), EMP_DOJ, 103) FROM EMP
```

```
SELECT EMP_DOJ, CONVERT(DATE, EMP_DOJ, 103) FROM EMP
```

Logic: GETDATE() and most SQL Server DATE columns display by default in an ISO-like YYYY-MM-DD format. Reporting or region-specific requirements often call for DD/MM/YYYY instead. Converting to VARCHAR with style 103 reformats the text representation for display; converting to DATE with style 103 tells SQL Server how to interpret an

incoming date string (e.g. one stored as text in DD/MM/YYYY order) so it parses correctly into a true DATE value rather than being misread.

CAST vs CONVERT — Which to Use

Aspect	CAST	CONVERT
Standard	ANSI SQL — portable across databases	T-SQL specific — SQL Server / Sybase only
Style / format control	Not supported	Optional 3rd argument (style code) controls date/number formatting
Typical use	Simple type changes; portable code	Date/number reformatting; SQL Server-only scripts
Error on bad data	Throws an error and stops the query	Throws an error and stops the query

TRY_CAST and TRY_CONVERT(Safe Conversion)

CAST and CONVERT both throw a hard error and stop the query if a value can't be converted (e.g. converting the text "N/A" to an INT). This is a real problem on messy, real-world data where a handful of bad rows would otherwise stop an entire query or ETL job. TRY_CAST and TRY_CONVERT solve this: they attempt the same conversion, but return NULL instead of raising an error when the value can't be converted, letting the rest of the query run.

Sample Setup Used for the TRY_CAST and TRY_CONVERT Examples

The following table is used to explain TRY_CAST and TRY_CONVERT.

Create and populate the table:

-- Create Table

```
CREATE TABLE EMP_DETAILS
```

```
(  
    NAME VARCHAR(50),  
    AGE  VARCHAR(10),  
    J_DATE VARCHAR(20)  
);
```

-- Insert Data

```
INSERT INTO EMP_DETAILS (NAME, AGE, J_DATE)
```

```
VALUES
```

```
('RAKESH', '30', '2026-04-16'),  
( 'RAJESH', '31', '2026-01-16'),  
( 'PRIYA', '26', '2026-04-09'),  
( 'LALITHA', '32', 'NA'),  
( 'KIRAN', '24', '2026-04-13'),  
( 'PRANATHI', '29', '2026-04-15'),  
( 'TRISHA', '21', 'TEST'),  
( 'ASHOK', '27', '2026-03-24'),  
( 'KUMAR', 'AA', 'AA');
```

TRY_CAST

Definition:

Works exactly like CAST, but returns NULL instead of an error if the conversion fails.

Syntax:

```
TRY_CAST(value AS target_data_type)
```

TRY_CONVERT

Definition:

Works exactly like CONVERT (including the style-code argument), but returns NULL instead of an error if the conversion fails.

Syntax:

TRY_CONVERT(target_data_type, value [, style])

Example — cleaning up a travel details table with inconsistent AGE and J_DATE columns

```
SELECT *,
    TRY_CONVERT(INT, [AGE]),
    TRY_CONVERT(DATE, [J_DATE], 103),
    TRY_CAST([AGE] AS INT),
    TRY_CAST([J_DATE] AS DATE)
FROM [dbo].[TRAVEL_DET]
```

Logic: AGE and J_DATE are stored as text (VARCHAR), but likely contain a mix of clean values and bad/blank entries typed in by hand. Using plain CAST or CONVERT here would cause the entire query to fail the moment it hits one bad row. TRY_CONVERT / TRY_CAST instead convert what they can and quietly return NULL for anything that doesn't fit (e.g. a non-numeric AGE, or a J_DATE that isn't a real date), so the query completes and the bad rows are visible as NULLs to investigate afterward — rather than blocking the whole result set.

A second confirming example, showing TRY_CONVERT applied directly to a known-good value (GETDATE()) as a sanity check that the function behaves identically to CONVERT when the data is valid:

```
SELECT TRY_CONVERT(DATE, GETDATE(), 103)
```

A further use of TRY_CAST/TRY_CONVERT is wrapping them in another function, such as YEAR(), so downstream calculations don't error out either:

```
SELECT *,
    TRY_CAST(J_DATE AS DATE),
    TRY_CONVERT(DATE, J_DATE),
    YEAR(TRY_CAST(J_DATE AS DATE)),
    YEAR(TRY_CONVERT(DATE, J_DATE)),
    TRY_CAST(AGE AS INT)
FROM [dbo].[TRAVEL_DET]
```

Logic: YEAR() requires a valid DATE input. If J_DATE contains an unparseable value, TRY_CAST/TRY_CONVERT return NULL, and YEAR(NULL) safely returns NULL rather than erroring — so the whole SELECT still runs across every row in the table.

Convert Formatting Style:

Style	Format	Example Output	Use Case
100	Mon dd yyyy hh:miAM	Jun 25 2026 12:30PM	Default US datetime format
101	mm/dd/yyyy	06/25/2026	US date format
102	yyyy.mm.dd	2026.06.25	ISO with dots
103	dd/mm/yyyy	25-06-2026	Indian/UK format
104	dd.mm.yyyy	25.06.2026	European format
105	dd-mm-yyyy	25-06-2026	Common reporting format
106	dd Mon yyyy	25-Jun-26	Readable report format
107	Mon dd, yyyy	Jun 25, 2026	Business reports
108	hh:mm:ss	14:35:50	Time only (24-hour)
109	Mon dd yyyy hh:mi:ss:mmmAM	Jun 25 2026 02:35:50:123PM	Includes milliseconds
110	mm-dd-yyyy	06-25-2026	US format with hyphens
111	yyyy/mm/dd	25-06-2026	ISO with slashes
112	yyyymmdd	20260625	Sorting, file names
113	dd Mon yyyy hh:mi:ss:mmm	25 Jun 2026 14:35:50:123	Reporting
114	hh:mi:ss:mmm	14:35:50:123	Time with milliseconds
120	yyyy-mm-dd hh:mi:ss	25-06-2026 14:35	ODBC canonical
121	yyyy-mm-dd hh:mi:ss:mmm	35:50.1	ODBC with milliseconds
126	yyyy-mm-ddThh:mi:ss:mmm	2026-06-25T14:35:50.123	ISO 8601
127	yyyy-mm-ddThh:mm:ss.fffZ	2026-06-25T14:35:50.123Z	UTC timestamp
130	dd mon yyyy hh:mi:ss:mmmAM	02 Muh 1448 02:35:50:123PM	Hijri date
131	dd/mm/yyyy hh:mi:ss:mmmAM	02/01/1448 02:35:50:123PM	Hijri format

NULL Functions:**ISNULL, COALESCE, NULLIF**

Sample Setup Used for the ISNULL, COALESCE and NULLIF

Examples

The following table is used to explain ISNULL, COALESCE and NULLIF.

Create and populate the table:**-- Create Table**

```
CREATE TABLE EMP_VALUES
```

```
(
  FIRST_NAME VARCHAR(50) NULL,
  LAST_NAME  VARCHAR(50) NULL,
  VALUE      INT NULL
);
```

Insert Data:

```
INSERT INTO EMP_VALUES (FIRST_NAME, LAST_NAME, VALUE)
```

```
VALUES
```

```
('RAKESH', 'REDDY', 100),
('RAJESH', 'KUMAR', 200),
(NULL,     'REDDY', 200),
```

```
('PRIYA', 'REDDY', 300),  
( 'LALITHA', 'KUMARI', NULL),  
( 'KIRAN', 'KUMAR', 100),  
(NULL, NULL, 100),  
( 'PRANATHI', 'YADAV', 300),  
( 'TRISHA', NULL, 300),  
( 'ASHOK', NULL, 200);
```

ISNULL

Definition: Replaces a NULL value with a specified default value. ISNULL only checks and handles a single column/expression at a time.

Syntax:

```
ISNULL(column, default_value)
```

Example — replacing missing first names with 'NA':

```
SELECT FIRST_NAME,  
       ISNULL(FIRST_NAME, 'NA')  
FROM CUSTOMERS_INFO
```

Logic: wherever FIRST_NAME is NULL, ISNULL substitutes the literal text 'NA'; every non-NULL FIRST_NAME passes through unchanged. This guarantees the output column never contains a blank/NULL value, which is useful for reports and exports where NULL looks broken to an end user.

COALESCE

Definition:

Returns the first non-NULL value from a list of two or more expressions. Unlike ISNULL, COALESCE can check across multiple columns in priority order, falling back through each one until it finds a value that isn't NULL — with a final literal as the ultimate fallback.

Syntax:

```
COALESCE(column1, column2, ..., default_value)
```

Example — filling in a missing first name from the last name, and vice versa:

```
SELECT FIRST_NAME, LAST_NAME,  
       COALESCE(FIRST_NAME, LAST_NAME, 'NA') AS F_NAME,  
       COALESCE(LAST_NAME, FIRST_NAME, 'NA') AS L_NAME  
FROM CUSTOMERS_INFO
```

Logic: F_NAME uses FIRST_NAME if it exists; if FIRST_NAME is NULL, it falls back to LAST_NAME; if both are NULL, it falls back to 'NA'. L_NAME does the mirror image, preferring LAST_NAME and falling back to FIRST_NAME, then 'NA'. This is the key difference from ISNULL — COALESCE lets a NULL in one column be recovered from a value in a different, related column, not just replaced with a fixed constant.

NULLIF

Definition:

Compares two expressions and returns NULL if they are equal; otherwise it returns the first expression. It is effectively the reverse operation of ISNULL/COALESCE — instead of replacing NULL with a value, it replaces a specific value with NULL.

Syntax:

```
NULLIF(expression1, expression2)
```

Example — treating a placeholder value of 200 as if it were missing data:

```
SELECT *,  
    NULLIF(VALUE, 200)  
FROM CUSTOMERS_INFO
```

Logic: whenever VALUE equals 200 (for instance, a sentinel/placeholder number used to mean "unknown" instead of a real NULL), NULLIF converts it to a true NULL so it can be handled consistently with the rest of the missing data (e.g. excluded from AVG(), or later passed through ISNULL/COALESCE). If VALUE is anything other than 200, it is returned unchanged.

WHERE:

The WHERE clause is used to filter rows from a table based on a specified condition. It retrieves only those records that satisfy the given condition. It is applied before any grouping or aggregation.

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

-- Get employees with salary greater than 25000

```
SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL]  
FROM [dbo].[EMP]  
WHERE [EMP_SAL] > 25000;
```

-- Multiple conditions

```
SELECT * FROM [dbo].[EMP]  
WHERE [EMP_GENDER] = 'M' AND [EMP_LOC] = 'HYD'
```

Key points:

- ✓ Filters data **at the row level**
- ✓ Executes **before** GROUP BY
- ✓ **Cannot** be used with aggregate functions (SUM, COUNT, AVG, etc.)
- ✓ Supports operators: =, !=, >, <, >=, <=, BETWEEN, IN, LIKE, IS NULL
- ✓ Multiple conditions can be combined using AND / OR

1. = (EQUAL TO)

Get employee with EMP_ID = 101

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_ID = 101;
```

Get all Male employees

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_GENDER = 'M';
```

Get employees located in HYD

```
SELECT EMP_ID, EMP_NAME, EMP_LOC FROM [dbo].[EMP]  
WHERE EMP_LOC = 'HYD';
```

Get employee with exact salary

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL = 35000;
```

2. != (NOT EQUAL TO)**Get all employees except M**

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_GENDER != 'M';
```

Get employees NOT in HYD

```
SELECT EMP_NAME, EMP_LOC FROM [dbo].[EMP]  
WHERE EMP_LOC != 'HYD';
```

Get employees whose salary is not 29000

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL != 29000;
```

Using <> (alternate syntax for !=)

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_LOC <> 'DLH';
```

3. > (GREATER THAN)**Get employees with salary greater than 25000**

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL > 25000;
```

Get employees who joined after 2022

```
SELECT EMP_NAME, EMP_DOJ FROM [dbo].[EMP]  
WHERE EMP_DOJ > '2022-01-01';
```

Get employees with EMP_ID greater than 104

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_ID > 104;
```

4. < (LESS THAN)**Get employees with salary less than 23000**

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL < 23000;
```

Get employees who joined before 2024

```
SELECT EMP_NAME, EMP_DOJ FROM [dbo].[EMP]  
WHERE EMP_DOJ < '2024-01-01';
```

Get employees with EMP_ID less than 105

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_ID < 105;
```

5. >= (GREATER THAN OR EQUAL TO)

Get employees with salary >= 22000

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL >= 22000;
```

Get employees who joined on or after 2022

```
SELECT EMP_NAME, EMP_DOJ FROM [dbo].[EMP]  
WHERE EMP_DOJ >= '2022-01-01';
```

Get employees with EMP_ID >= 105

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_ID >= 105;
```

6. <= (LESS THAN OR EQUAL TO)

Get employees with salary <= 33000

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL <= 33000;
```

Get employees who joined on or before 2023

```
SELECT EMP_NAME, EMP_DOJ FROM [dbo].[EMP]  
WHERE EMP_DOJ <= '2023-12-31';
```

Get employees with EMP_ID <= 103

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_ID <= 103;
```

7. BETWEEN (RANGE CHECK - INCLUSIVE)

Get employees with salary between 20000 and 30000

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL BETWEEN 20000 AND 30000;
```

Get employees who joined between 2022 and 2024

```
SELECT EMP_NAME, EMP_DOJ FROM [dbo].[EMP]  
WHERE EMP_DOJ BETWEEN '2022-01-01' AND '2024-12-31';
```

Get employees with EMP_ID between 102 and 106

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_ID BETWEEN 102 AND 106;
```

NOT BETWEEN - outside salary range

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_SAL NOT BETWEEN 20000 AND 40000;
```

8. IN (MATCH ANY VALUE IN LIST)

Get employees located in HYD, MI or DLH

```
SELECT EMP_NAME, EMP_LOC FROM [dbo].[EMP]
WHERE EMP_LOC IN ('HYD', 'MI', 'DLH');
```

Get employees with specific EMP_IDs

```
SELECT * FROM [dbo].[EMP]
WHERE EMP_ID IN (101, 103, 105, 107);
```

Get employees under specific managers

```
SELECT EMP_NAME, MANAGER_ID FROM [dbo].[EMP]
WHERE MANAGER_ID IN (102, 114);
```

NOT IN - exclude specific locations

```
SELECT EMP_NAME, EMP_LOC FROM [dbo].[EMP]
WHERE EMP_LOC NOT IN ('Chennai', 'MI');
```

9. LIKE (PATTERN MATCHING)

Names starting with 'R'

```
SELECT EMP_NAME FROM [dbo].[EMP]
WHERE EMP_NAME LIKE 'R%';
```

Names ending with 'a'

```
SELECT EMP_NAME FROM [dbo].[EMP]
WHERE EMP_NAME LIKE '%a';
```

Names containing 'Kumar' anywhere

```
SELECT EMP_NAME FROM [dbo].[EMP]
WHERE EMP_NAME LIKE '%Kumar%';
```

Email ending with '@gmail.com'

```
SELECT EMP_NAME, EMP_MAIL FROM [dbo].[EMP]
WHERE EMP_MAIL LIKE '%@gmail.com';
```

Mobile numbers starting with '98'

```
SELECT EMP_NAME, EMP_MOBILE FROM [dbo].[EMP]
WHERE EMP_MOBILE LIKE '98%';
```

NOT LIKE - emails not from gmail

```
SELECT EMP_NAME, EMP_MAIL FROM [dbo].[EMP]
WHERE EMP_MAIL NOT LIKE '%@gmail.com';
```

LIKE Wildcards Reference:

% -> Matches zero or more characters
_ -> Matches exactly one character

10. IS NULL / IS NOT NULL

Employees with no Manager (top-level)

```
SELECT EMP_NAME, MANAGER_ID FROM [dbo].[EMP]  
WHERE MANAGER_ID IS NULL;
```

Employees whose Email is not filled

```
SELECT EMP_NAME, EMP_MAIL FROM [dbo].[EMP]  
WHERE EMP_MAIL IS NULL;
```

Employees whose Mobile number is missing

```
SELECT EMP_NAME, EMP_MOBILE FROM [dbo].[EMP]  
WHERE EMP_MOBILE IS NULL;
```

IS NOT NULL - Employees who HAVE a manager

```
SELECT EMP_NAME, MANAGER_ID FROM [dbo].[EMP]  
WHERE MANAGER_ID IS NOT NULL;
```

Employees who have provided their email

```
SELECT EMP_NAME, EMP_MAIL FROM [dbo].[EMP]  
WHERE EMP_MAIL IS NOT NULL;
```

11. MULTIPLE CONDITIONS - AND**Female employees in HYD**

```
SELECT * FROM [dbo].[EMP]  
WHERE EMP_GENDER = 'F' AND EMP_LOC = 'HYD';
```

Male employees with salary > 25000

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_GENDER = 'M' AND EMP_SAL > 25000;
```

Employees in DLH who joined after 2022 with salary >= 30000

```
SELECT EMP_NAME, EMP_LOC, EMP_SAL FROM [dbo].[EMP]  
WHERE EMP_LOC = 'DLH'  
AND EMP_DOJ > '2022-01-01'  
AND EMP_SAL >= 30000;
```

Employees under manager 115 with gmail email

```
SELECT EMP_NAME, MANAGER_ID FROM [dbo].[EMP]  
WHERE MANAGER_ID = 115 AND EMP_MAIL LIKE '%@gmail.com';
```

12. MULTIPLE CONDITIONS - OR**Employees in HYD OR Chennai**

```
SELECT EMP_NAME, EMP_LOC FROM [dbo].[EMP]  
WHERE EMP_LOC = 'HYD' OR EMP_LOC = 'Chennai';
```

Employees with salary < 15000 OR salary > 350000

```
SELECT EMP_NAME, EMP_SAL FROM [dbo].[EMP]
```

WHERE EMP_SAL < 15000 OR EMP_SAL > 35000;

Employees who are Female OR located in MI

```
SELECT EMP_NAME, EMP_GENDER, EMP_LOC FROM [dbo].[EMP]
WHERE EMP_GENDER = 'F' OR EMP_LOC = 'MI';
```

13. COMBINED AND + OR (WITH PARENTHESES)

Female employees in HYD OR Male employees in DLH

```
SELECT EMP_NAME, EMP_GENDER, EMP_LOC FROM [dbo].[EMP]
WHERE (EMP_GENDER = 'F' AND EMP_LOC = 'HYD')
OR (EMP_GENDER = 'M' AND EMP_LOC = 'DLH');
```

Employees with salary between 20000-30000 AND in HYD or MI

```
SELECT EMP_NAME, EMP_SAL, EMP_LOC FROM [dbo].[EMP]
WHERE EMP_SAL BETWEEN 20000 AND 30000
AND EMP_LOC IN ('HYD', 'MI');
```

Employees with no mobile OR no email, AND joined before 2022

```
SELECT EMP_NAME, EMP_MOBILE, EMP_MAIL FROM [dbo].[EMP]
WHERE (EMP_MOBILE IS NULL OR EMP_MAIL IS NULL)
AND EMP_DOJ < '2022-01-01';
```

Complex condition combining all types

```
SELECT EMP_NAME, EMP_SAL, EMP_LOC, EMP_GENDER
FROM [dbo].[EMP]
WHERE EMP_LOC IN ('HYD', 'DLH')
AND EMP_SAL BETWEEN 50000 AND 100000
AND EMP_GENDER = 'F'
AND EMP_MAIL LIKE '%@gmail.com'
AND MANAGER_ID IS NOT NULL;
```

Quick Reference Summary

Operator	Use Case	Example
=	Exact match	EMP_LOC = 'HYD'
!= or <>	Not equal	EMP_GENDER != 'M'
>	Greater than	EMP_SAL > 35000
<	Less than	EMP_SAL < 35000
>=	Greater than or equal	EMP_DOJ >= '2023-01-01'
<=	Less than or equal	EMP_SAL <= 35000
BETWEEN	Range (inclusive)	EMP_SAL BETWEEN 20000 AND 40000
IN	Match from list	EMP_LOC IN ('DLH', 'MI')
LIKE	Pattern match	EMP_MAIL LIKE '%@gmail.com'
IS NULL	Check NULL value	MANAGER_ID IS NULL
IS NOT NULL	Check not NULL	EMP_MOBILE IS NOT NULL

AND	All conditions must match	EMP_GENDER='F' AND EMP_SAL>50000
OR	Any condition can match	EMP_LOC='DLH' OR EMP_LOC='MI'

Key Points:

- ✓ Every non-aggregated column in SELECT must appear in GROUP BY.
- ✓ NULL values are grouped together as a single group in SQL Server.
- ✓ Multiple columns can be specified in GROUP BY for multi-level aggregation (e.g., GROUP BY Customer Name, City).

GROUP BY:

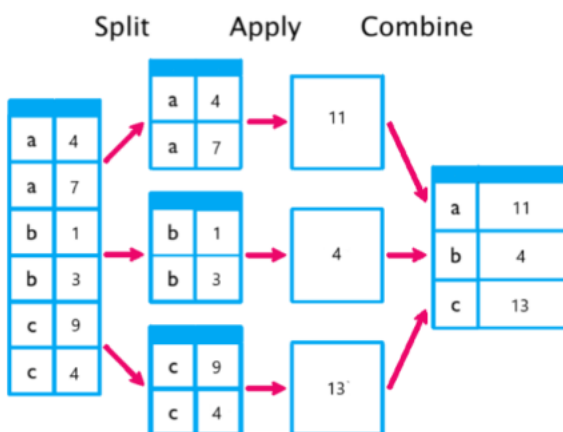
Is used to arrange identical data into groups so that aggregate functions can be applied to each group.

Aggregates rows that share the same values in one or more columns into summary rows. Must be used with an aggregate function in the SELECT list.

In simple words: "Split data into buckets → Apply calculations on each bucket"

The SQL Group By statement uses the split-apply-combine strategy.

- ✓ Split: The different groups are split with their values.
- ✓ Apply: The aggregate function is applied to the values of these groups.
- ✓ Combine: The values are combined in a single row.



The **SUM()** aggregate function, which results in the arithmetic sum of the rows' values, has been applied to the groups in the above illustration.

Syntax

```
SELECT column1, AGGREGATE_FUNCTION(column2)
FROM table_name
GROUP BY column1
```

1. Employee count per location

```
SELECT [EMP_LOC], COUNT(*) AS NO_OF_EMP
FROM [dbo].[EMP]
GROUP BY [EMP_LOC]
```

2. Total salary per location

```
SELECT [EMP_LOC], SUM([EMP_SAL]) AS TOTAL_SAL
FROM [dbo].[EMP]
GROUP BY [EMP_LOC]
```

3. Count employees by location

```
SELECT [EMP_LOC], COUNT(*) AS EMP_COUNT  
FROM [dbo].[EMP]  
GROUP BY [EMP_LOC];
```

4. Total salary by gender

```
SELECT [EMP_GENDER], SUM([EMP_SAL]) AS TOTAL_SAL  
FROM [dbo].[EMP]  
GROUP BY [EMP_GENDER];
```

5. Avg salary by manager

```
SELECT [MANAGER_ID], AVG([EMP_SAL]) AS AVG_SAL  
FROM [dbo].[EMP]  
GROUP BY [MANAGER_ID];
```

6. DOJ year wise count

```
SELECT YEAR([EMP_DOJ]) AS DOJ_YEAR, COUNT(*) AS EMP_COUNT  
FROM [dbo].[EMP]  
GROUP BY YEAR([EMP_DOJ]);
```

7. Location and gender split

```
SELECT [EMP_LOC], [EMP_GENDER], COUNT(*) AS CNT  
FROM [dbo].[EMP]  
GROUP BY [EMP_LOC], [EMP_GENDER];
```

Key Points:

- ✓ Every non-aggregated column in SELECT must appear in GROUP BY.
- ✓ NULL values are grouped together as a single group in SQL Server.
- ✓ Multiple columns can be specified in GROUP BY for multi-level aggregation (e.g., GROUP BY Customer Name, City).

HAVING

Definition: Filters grouped results based on a condition applied to an aggregate function. HAVING operates after GROUP BY, whereas WHERE filters individual rows before grouping.

Syntax

```
SELECT column1, AGG_FUNC(col2)  
FROM table_name  
GROUP BY column1  
HAVING AGG_FUNC(col2) > value
```

1. Locations with more than 3 employees

```
SELECT [EMP_LOC], COUNT(*) AS NO_OF_EMP  
FROM [dbo].[EMP]  
GROUP BY [EMP_LOC]  
HAVING COUNT(*) > 3
```

2. Customers with 50+ orders, excluding Germany

```
SELECT [Customer Name], COUNT(*) AS NO_OF_ORDERS  
FROM [dbo].[Orders]  
WHERE [Country] <> 'Germany'  
GROUP BY [Customer Name]  
HAVING COUNT(*) >= 50
```

3. Locations with more than 3 employees

```
SELECT [EMP_LOC], COUNT(*) AS CNT
FROM [dbo].[EMP]
GROUP BY [EMP_LOC]
HAVING COUNT(*) > 3;
```

4. Managers with high team salary

```
SELECT [MANAGER_ID], SUM([EMP_SAL]) AS TOTAL_SAL
FROM [dbo].[EMP]
GROUP BY [MANAGER_ID]
HAVING SUM([EMP_SAL]) > 30000;
```

5. Gender groups with avg salary > 25000

```
SELECT [EMP_GENDER], AVG([EMP_SAL]) AS AVG_SAL
FROM [dbo].[EMP]
GROUP BY [EMP_GENDER]
HAVING AVG([EMP_SAL]) > 25000;
```

6. DOJ years with multiple joiners

```
SELECT YEAR([EMP_DOJ]) AS DOJ_YEAR, COUNT(*) AS CNT
FROM [dbo].[EMP]
GROUP BY YEAR([EMP_DOJ])
HAVING COUNT(*) >= 2;
```

7. Locations with max salary > 25000

```
SELECT [EMP_LOC], MAX([EMP_SAL]) AS MAX_SAL
FROM [dbo].[EMP]
GROUP BY [EMP_LOC]
HAVING MAX([EMP_SAL]) > 25000;
```

Key Points:

- ✓ HAVING cannot reference column aliases defined in the SELECT clause — repeat the aggregate expression.
- ✓ WHERE filters rows before aggregation; HAVING filters groups after aggregation.
- ✓ Both WHERE and HAVING can coexist in the same query (see Example 2 above).

ORDER BY:

Definition: Sorts the final result set in ascending (ASC) or descending (DESC) order. It is always the last clause evaluated.

Use Case: Display location-wise employee counts ranked from highest to lowest; identify top-N customers by order volume using TOP + ORDER BY.

Syntax

```
SELECT column1, AGG_FUNC(col2)
FROM table_name
GROUP BY column1
ORDER BY column1 ASC | DESC
```

1. Salary low to high

```
SELECT [EMP_NAME], [EMP_SAL]
FROM [dbo].[EMP]
ORDER BY [EMP_SAL] ASC;
```

2. Salary high to low

```
SELECT [EMP_NAME], [EMP_SAL]
FROM [dbo].[EMP]
ORDER BY [EMP_SAL] DESC;
```

3. Name alphabetical order

```
SELECT [EMP_NAME]
FROM [dbo].[EMP]
ORDER BY [EMP_NAME];
```

4. Latest joiners first

```
SELECT [EMP_NAME], [EMP_DOJ]
FROM [dbo].[EMP]
ORDER BY [EMP_DOJ] DESC;
```

5. Location then salary

```
SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL]
FROM [dbo].[EMP]
ORDER BY [EMP_LOC], [EMP_SAL] DESC;
```

6. Order alphabetically by location

```
SELECT [EMP_LOC], COUNT(*) AS NO_OF_EMP
FROM [dbo].[EMP]
GROUP BY [EMP_LOC]
ORDER BY [EMP_LOC] DESC
```

7. Top 2 locations by employee count (ascending)

```
SELECT TOP 2 [EMP_LOC], COUNT(*) AS NO_OF_EMP
FROM [dbo].[EMP]
GROUP BY [EMP_LOC]
ORDER BY NO_OF_EMP ASC
```

Key Points:

- ✓ ORDER BY is the only clause that can reference SELECT aliases (e.g., ORDER BY NO_OF_EMP).
- ✓ Without ORDER BY, SQL Server does not guarantee any sort order — never rely on implicit ordering.
- ✓ GROUP BY must precede ORDER BY. You cannot place ORDER BY between GROUP BY and HAVING.

Below are the examples covering **TOP, WHERE, GROUP BY, HAVING & ORDER BY**.

Top 5 locations with highest employee count

```
SELECT TOP 5
EMP_LOC,
COUNT(*) AS EMP_COUNT
FROM dbo.EMP
WHERE EMP_LOC IS NOT NULL
GROUP BY EMP_LOC
HAVING COUNT(*) > 0
ORDER BY EMP_COUNT DESC;
```

Find top 3 email domains used by employees

```
SELECT TOP 3
RIGHT(EMP_MAIL, LEN(EMP_MAIL) - CHARINDEX('@', EMP_MAIL)) AS EMAIL_DOMAIN,
COUNT(*) AS TOTAL_USERS
FROM dbo.EMP
```

```

WHERE EMP_MAIL IS NOT NULL
GROUP BY RIGHT(EMP_MAIL, LEN(EMP_MAIL) - CHARINDEX('@', EMP_MAIL))
HAVING COUNT(*) > 1
ORDER BY TOTAL_USERS DESC;

```

Top 3 years with highest employee hiring

```

SELECT TOP 3
YEAR(EMP_DOJ) AS JOIN_YEAR,
COUNT(*) AS TOTAL_HIRES
FROM dbo.EMP
WHERE EMP_DOJ IS NOT NULL
GROUP BY YEAR(EMP_DOJ)
HAVING COUNT(*) > 0
ORDER BY TOTAL_HIRES DESC;

```

Top 3 months with highest joining count

```

SELECT TOP 3
DATENAME(MONTH, EMP_DOJ) AS JOIN_MONTH,
COUNT(*) AS EMP_COUNT
FROM dbo.EMP
WHERE EMP_DOJ IS NOT NULL
GROUP BY DATENAME(MONTH, EMP_DOJ)
HAVING COUNT(*) > 0
ORDER BY EMP_COUNT DESC;

```

Top 5 employee name prefixes with highest occurrences

```

SELECT TOP 5
LEFT(EMP_NAME, 1) AS NAME_PREFIX,
COUNT(*) AS CNT
FROM dbo.EMP
WHERE EMP_NAME IS NOT NULL
GROUP BY LEFT(EMP_NAME, 1)
HAVING COUNT(*) > 1
ORDER BY CNT DESC;

```

Top 5 highest average salaries by location

```

SELECT TOP 5
EMP_LOC,
AVG(EMP_SAL) AS AVG_SALARY
FROM dbo.EMP
WHERE EMP_SAL IS NOT NULL
GROUP BY EMP_LOC
HAVING AVG(EMP_SAL) > 0
ORDER BY AVG_SALARY DESC;

```

Find top 5 oldest employees based on DOJ

```

SELECT TOP 5
EMP_ID,
EMP_NAME,
DATEDIFF(YEAR, EMP_DOJ, GETDATE()) AS EXPERIENCE_YEARS
FROM dbo.EMP

```

```
WHERE EMP_DOJ IS NOT NULL
ORDER BY EXPERIENCE_YEARS DESC;
```

Top 3 locations with highest total salary payout

```
SELECT TOP 3
EMP_LOC,
SUM(EMP_SAL) AS TOTAL_SALARY
FROM dbo.EMP
WHERE EMP_SAL IS NOT NULL
GROUP BY EMP_LOC
HAVING SUM(EMP_SAL) > 100000
ORDER BY TOTAL_SALARY DESC;
```

Top 5 mobile number prefixes with most employees

```
SELECT TOP 5
LEFT(EMP_MOBILE, 3) AS MOBILE_PREFIX,
COUNT(*) AS TOTAL_EMP
FROM dbo.EMP
WHERE EMP_MOBILE IS NOT NULL
GROUP BY LEFT(EMP_MOBILE, 3)
HAVING COUNT(*) > 1
ORDER BY TOTAL_EMP DESC;
```

SUB QUERIES:

A Subquery (also called a nested query or inner query) is a SQL query embedded inside another SQL query. The outer query is called the main query, and the inner query provides data or conditions to it. Subqueries are enclosed in parentheses and can appear in SELECT, FROM, WHERE, or HAVING clauses.

Use Cases:

- ✓ Finding records based on aggregate values (e.g., highest, lowest salary)
- ✓ Filtering rows using conditions derived from another table or query
- ✓ Finding the Nth highest or Nth lowest value in a column
- ✓ Replacing complex JOINS with readable nested logic
- ✓ Comparing values row-by-row with another result set

Syntax:

General syntax of a subquery in a WHERE clause:

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT aggregate_function(column_name) FROM table_name);
```

Subquery used in the FROM clause (Derived Table):

```
SELECT column1, column2
FROM (SELECT column1, column2 FROM table_name WHERE condition) AS alias;
```

Limitations:

- ✓ Subqueries can be difficult to read and debug when deeply nested (3+ levels)
- ✓ Performance may degrade for large datasets because the inner query runs for each outer row (correlated subquery)

- ✓ Cannot use ORDER BY inside a subquery unless TOP is also used (SQL Server)
- ✓ A subquery in SELECT can return only a single value (scalar subquery)

Examples:

Find the employee(s) earning the highest salary:

```
SELECT [EMP_NAME] FROM [dbo].[EMP]
WHERE [EMP_SAL] = (SELECT MAX([EMP_SAL]) FROM [dbo].[EMP]);
```

Find the employee(s) with the 2nd highest salary by excluding the maximum:

```
SELECT [EMP_NAME] FROM [dbo].[EMP]
WHERE [EMP_SAL] = (
    SELECT MAX([EMP_SAL]) FROM [dbo].[EMP]
    WHERE [EMP_SAL] <
        (SELECT MAX([EMP_SAL]) FROM [dbo].[EMP])
);
```

Find the employee with Nth Highest Salary (Step-by-Step)

Step 1: Get distinct salary values in descending order:

```
SELECT DISTINCT [EMP_SAL] FROM [dbo].[EMP] ORDER BY [EMP_SAL] DESC
```

Step 2: Get the top N (e.g., 5) distinct salaries:

```
SELECT DISTINCT TOP 5 [EMP_SAL] FROM [dbo].[EMP] ORDER BY [EMP_SAL] DESC
```

Step 3: Get the minimum of those top N values (= Nth highest):

```
SELECT MIN([EMP_SAL]) FROM [dbo].[EMP]
WHERE [EMP_SAL] IN (SELECT DISTINCT TOP 5 [EMP_SAL] FROM [dbo].[EMP] ORDER BY [EMP_SAL] DESC)
```

Step 4: Return employees at that Nth highest salary:

```
SELECT [EMP_NAME] FROM [dbo].[EMP]
WHERE [EMP_SAL] = (
    SELECT MIN([EMP_SAL]) FROM [dbo].[EMP]
    WHERE [EMP_SAL] IN (
        SELECT DISTINCT TOP 3 [EMP_SAL] FROM [dbo].[EMP] ORDER BY [EMP_SAL] DESC
    )
);
```

Subquery in FROM clause (Derived Table)

Aggregate city sales, filter, and get top 10 results:

```
SELECT TOP 10 * FROM
(
    SELECT [City], SUM([Net_Sales]) AS TOTAL_SALES,
    DATEADD(DD, 5, [Order_Date]) AS EXPECTED_DELIVERY_DATE
    FROM [dbo].[RetailSales]
    WHERE [Age] >= 25
    GROUP BY [City], DATEADD(DD, 5, [Order_Date])
    HAVING SUM([Net_Sales]) > 200
) AS T1
ORDER BY TOTAL_SALES DESC;
```

Nested Aggregation with Subquery

Get total salary per location using a derived table:

```
SELECT [EMP_LOC], SUM(TOTAL_SAL) AS TOT_SAL
FROM (
    SELECT [EMP_LOC], [EMP_GENDER],
    SUM([EMP_SAL]) AS TOTAL_SAL, COUNT(*) AS NO_OF_EMP
    FROM [dbo].[EMP]
    GROUP BY [EMP_LOC], [EMP_GENDER]
) AS A
GROUP BY [EMP_LOC];
```

Common Table Expressions (CTEs):

A Common Table Expression (CTE) is a temporary, named result set defined at the beginning of a SQL query using the WITH keyword. It exists only for the duration of the query and can be referenced like a regular table or view. CTEs make complex queries more readable and maintainable by breaking them into logical, named steps.

Use Cases:

- ✓ Simplifying complex nested subqueries for better readability
- ✓ Breaking multi-step logic into clearly named stages
- ✓ Finding highest/Nth highest salary in a clean and reusable way
- ✓ Filtering grouped/aggregated results before further processing
- ✓ Recursive queries (hierarchical data like org charts, categories)
- ✓ Reusing the same subquery result multiple times in a single query

Syntax:

```
WITH cte_name AS
(
    -- SQL query goes here
    SELECT column1, column2 FROM table_name WHERE condition
)
SELECT * FROM cte_name;
```

Multiple CTEs can be chained together:

```
WITH CTE1 AS
(
    SELECT ... FROM table1
),
CTE2 AS
(
    SELECT ... FROM CTE1
)
SELECT * FROM CTE2;
```

Limitations:

- ✓ A CTE is not persisted — it exists only for the duration of a single query
- ✓ Cannot use ORDER BY inside a CTE unless TOP is also specified
- ✓ CTEs cannot be indexed; for repeated use, consider temp tables or views
- ✓ Non-recursive CTEs do not provide performance benefits over subqueries — they are primarily a readability tool

Example 1 – Highest Salary using CTE

Equivalent to subquery approach but more readable:

```
WITH HIGHESTSAL AS
(
    SELECT MAX([EMP_SAL]) AS HIGHEST_SAL FROM [dbo].[EMP]
)
SELECT [EMP_NAME] FROM [dbo].[EMP]
WHERE [EMP_SAL] = (SELECT * FROM HIGHESTSAL);
```

Example 2 – 2nd Highest Salary using CTE

```
WITH SECONDHIGHSAL AS
(
    SELECT MAX([EMP_SAL]) AS SECOND_HIGH FROM [dbo].[EMP]
    WHERE [EMP_SAL] < (SELECT MAX([EMP_SAL]) FROM [dbo].[EMP])
)
SELECT [EMP_NAME] FROM [dbo].[EMP]
WHERE [EMP_SAL] = (SELECT * FROM SECONDHIGHSAL);
```

Example 3 – City-wise Sales Analysis using CTE

Filter employees in HYD, then group by gender and sum salaries:

```
WITH HYD_LOC AS
(
    SELECT * FROM [dbo].[EMP]
    WHERE [EMP_LOC] = 'HYD'
)
SELECT [EMP_GENDER], SUM([EMP_SAL]) AS TOTAL_SAL
FROM HYD_LOC
GROUP BY [EMP_GENDER];
```

Example 4 – Retail Sales CTE with All Clauses

CTE combining WHERE, GROUP BY, HAVING, DATEADD, and then TOP N retrieval:

```
WITH CITY_SALES_1 AS
(
    SELECT [City], SUM([Net_Sales]) AS TOTAL_SALES,
        DATEADD(DD, 5, [Order_Date]) AS EXPECTED_DELIVERY_DATE
    FROM [dbo].[RetailSales]
    WHERE [Age] >= 25
    GROUP BY [City], DATEADD(DD, 5, [Order_Date])
    HAVING SUM([Net_Sales]) > 200
)
SELECT TOP 10 * FROM CITY_SALES_1
ORDER BY TOTAL_SALES DESC;
```

Example 5 – Two-Level Aggregation with CTE

First group by location and gender; then group again by location only:

```
WITH EMP_CTE AS
(
    SELECT [EMP_LOC], [EMP_GENDER],
        SUM([EMP_SAL]) AS TOTAL_SAL, COUNT(*) AS NO_OF_EMP
    FROM [dbo].[EMP]
    GROUP BY [EMP_LOC], [EMP_GENDER]
```

```
)
SELECT [EMP_LOC], SUM(TOTAL_SAL) AS TOT_SAL
FROM EMP_CTE
GROUP BY [EMP_LOC];
```

Example 6 – CTE with DENSE_RANK for Nth Highest per Group

```
WITH CTE_101 AS
(
    SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL],
    DENSE_RANK() OVER(PARTITION BY [EMP_LOC] ORDER BY [EMP_SAL] DESC) AS RNK
    FROM [dbo].[EMP]
)
SELECT * FROM CTE_101 WHERE RNK = 2;
```

WINDOW FUNCTIONS

Window Functions perform calculations across a set of table rows that are somehow related to the current row, without collapsing rows the way GROUP BY does. They use the OVER() clause to define the 'window' (set of rows) on which the function operates. Results are added as new columns alongside existing data, not summarized

ROW_NUMBER()	Assigns a unique sequential integer to each row within the result set or partition. No duplicates — every row gets a distinct number.
RANK()	Assigns a rank based on the ORDER BY value. Rows with the same value receive the same rank, and the next rank is skipped (1, 1, 3).
DENSE_RANK()	Same as RANK() but does NOT skip ranks. Duplicate values get the same rank, and the next rank follows consecutively (1, 1, 2).
LAG()	Returns a value from a previous row in the result set (without using a self-join). Useful for comparing current and past values.
LEAD()	Returns a value from a subsequent (next) row in the result set. Useful for comparing current and future values.



WINDOW
FUNCTIONS.xlsx

General syntax: (ROW_NUMBER, RANK, DENSE_RANK)

```
function_name() OVER ([PARTITION BY column_name] [ORDER BY column_name ASC|DESC])
function_name() OVER ([PARTITION BY column_name] [ORDER BY column_name ASC|DESC])
```

LAG and LEAD syntax:

```
LAG(column_name, offset, default_value) OVER (PARTITION BY col ORDER BY col)
LEAD(column_name, offset, default_value) OVER (PARTITION BY col ORDER BY col)
```

Limitations:

- ✓ Window functions cannot be used directly in WHERE or HAVING clauses — wrap them in a subquery or CTE first
- ✓ They may add processing overhead on very large datasets

- ✓ RANK() skips ranks after ties, which can confuse when filtering by exact rank number — use DENSE_RANK() to avoid this
- ✓ LAG/LEAD require careful ordering to ensure the correct previous/next row is selected

Example 1 – ROW_NUMBER, RANK, and DENSE_RANK Together

Demonstrate the difference between the three ranking functions on salary:

```
SELECT *,
    ROW_NUMBER() OVER(ORDER BY [EMP_SAL] DESC) AS R_NO,
    RANK() OVER(ORDER BY [EMP_SAL] DESC) AS RNK,
    DENSE_RANK() OVER(ORDER BY [EMP_SAL] DESC) AS D_RNK
FROM [dbo].[EMP];
```

Example 2 – PARTITION BY (Ranking within Groups)

Rank employees by salary within each location separately:

```
SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL],
    ROW_NUMBER() OVER(PARTITION BY [EMP_LOC] ORDER BY [EMP_SAL] DESC) AS R_NO,
    RANK() OVER(PARTITION BY [EMP_LOC] ORDER BY [EMP_SAL] DESC) AS RNK,
    DENSE_RANK() OVER(PARTITION BY [EMP_LOC] ORDER BY [EMP_SAL] DESC) AS D_RNK
FROM [dbo].[EMP];
```

Example 3 – 2nd Highest Salary per Location (Subquery + DENSE_RANK)

Filter to get only the 2nd highest earner in each location:

```
SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL], RNK
FROM (
    SELECT [EMP_NAME], [EMP_LOC], [EMP_SAL],
        DENSE_RANK() OVER(PARTITION BY [EMP_LOC] ORDER BY [EMP_SAL] DESC) AS RNK
    FROM [dbo].[EMP]
) AS T
WHERE RNK = 2;
```

Example 4 – Top 5 States per Region by Order Count

Rank states within each region by number of orders and filter top 5:

```
SELECT [Region], [State], ORDERS, ORDER_RNK
FROM (
    SELECT [Region], [State], COUNT(*) AS ORDERS,
        DENSE_RANK() OVER(PARTITION BY [Region] ORDER BY COUNT(*) DESC) AS ORDER_RNK
    FROM [dbo].[Orders_]
    GROUP BY [Region], [State]
) AS T1
WHERE ORDER_RNK <= 5;
```

Example 5 – LAG Function (Previous Quarter Value)

Get the previous quarter's orders alongside each row for year-over-year comparison:

```
SELECT [YEAR_], [QTR], [ORDERS],
    LAG([ORDERS], 1, null) OVER(PARTITION BY [YEAR_] ORDER BY [QTR]) AS PRV_VALUE
FROM [dbo].[RTL_QTR]
ORDER BY [YEAR_], [QTR] ASC;
```

Example 6 – LAG Function with Quarter-over-Quarter Growth

Use lag to get the previous period value and then compute QoQ growth percentage:

```

WITH CTE_QTR AS (
    SELECT [YEAR_], [QTR], [ORDERS],
           LAG([ORDERS], 1, NULL) OVER(PARTITION BY [YEAR_] ORDER BY [QTR] DESC) AS PRV_VALUE
    FROM [dbo].[RTL_QTR]
)
SELECT [YEAR_], [QTR], [ORDERS],
       CASE
           WHEN PRV_VALUE = 0 THEN NULL
           ELSE (([ORDERS] - PRV_VALUE) * 100.0 / PRV_VALUE)
       END AS QOQ_PERCENT
FROM CTE_QTR;

```

Below is the query to create a new table (Use Retail Sales Table)

---> YEAR, QUARTER(2020-Q1), NO OF ORDERS, GROSS SALES, NET SALES, TOTAL QTY, UNIT PRICE, SHIPPING COST, PROFIT, AVERAGE DELIVERY DAYS

```

SELECT * INTO RTL_QTR FROM
(
    SELECT
    YEAR([Order_Date]) AS YEAR_,
    CONVERT(VARCHAR(10),YEAR([Order_Date]))+'-Q'+ CONVERT(VARCHAR(10),DATEPART(QQ, [Order_Date])) AS QTR,
    COUNT(*) AS ORDERS,
    ROUND(SUM([Gross_Sales]),0) AS GROSS_SALES,
    ROUND(SUM([Net_Sales]),0) AS NET_SALES,
    ROUND(SUM([Quantity]),0) AS QTY,
    ROUND(SUM([Unit_Price]),0) AS UNIT_PRICE,
    ROUND(SUM([Shipping_Cost]),0) AS SHIPPING_COST,
    ROUND(SUM([Profit]),0) AS PROFIT,
    AVG([Delivery_Days]) AS AVG_DELIVERY
    FROM [dbo].[RetailSales]
    GROUP BY YEAR([Order_Date]),
    CONVERT(VARCHAR(10),YEAR([Order_Date]))+'-Q'+ CONVERT(VARCHAR(10),DATEPART(QQ, [Order_Date]))
) AS T1
WHERE 1= 1

```

---> YEAR, MONTH(2020-01), NO OF ORDERS, GROSS SALES, NET SALES, TOTAL QTY, UNIT PRICE, SHIPPING COST, PROFIT, AVERAGE DELIVERY DAYS

```

SELECT * INTO RTL_MONTH FROM
(
    SELECT
    YEAR([Order_Date]) AS YEAR_,
    FORMAT([Order_Date], 'yyyy-MM') AS MONTH_,
    COUNT(*) AS ORDERS,
    ROUND(SUM([Gross_Sales]),0) AS GROSS_SALES,
    ROUND(SUM([Net_Sales]),0) AS NET_SALES,
    ROUND(SUM([Quantity]),0) AS QTY,
    ROUND(SUM([Unit_Price]),0) AS UNIT_PRICE,
    ROUND(SUM([Shipping_Cost]),0) AS SHIPPING_COST,
    ROUND(SUM([Profit]),0) AS PROFIT,
    AVG([Delivery_Days]) AS AVG_DELIVERY

```

```
FROM [dbo].[RetailSales]
GROUP BY YEAR([Order_Date]),
FORMAT([Order_Date], 'yyyy-MM')
) AS T1
WHERE 1 = 1
```

Note: Use the below dataset to create tables explained in query above.



RetailSales.txt

Example: Running totals of Orders for each month in a year

```
SELECT [YEAR_], [MONTH_],[QTY],
SUM([QTY]) OVER(PARTITION BY [YEAR_] ORDER BY [MONTH_] ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS RUNNING_TOTALS
FROM [dbo].[RTL_MONTH]
```

Example: 3 Months Rolling average of orders in each year

```
SELECT [YEAR_], [MONTH_],[QTY],
AVG([QTY]) OVER(PARTITION BY [YEAR_] ORDER BY [MONTH_] ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS RUNNING_TOTALS
FROM [dbo].[RTL_MONTH]
```

UNBOUNDED PRECEDING

Definition: This sets the start of your window frame to the very first row of the partition.

Usage: Used to calculate values from the start of the data set up to the current row.

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

From first row → up to current row

UNBOUNDED FOLLOWING

Definition: This sets the end of your window frame to the very last row of the partition.

Usage: Used to calculate values from the current row through the end of the data set.

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING

From current row → till last row

Example: Current row + previous 2 rows

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

Example: Current row + next 2 rows

ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING

Introduction to Joins:

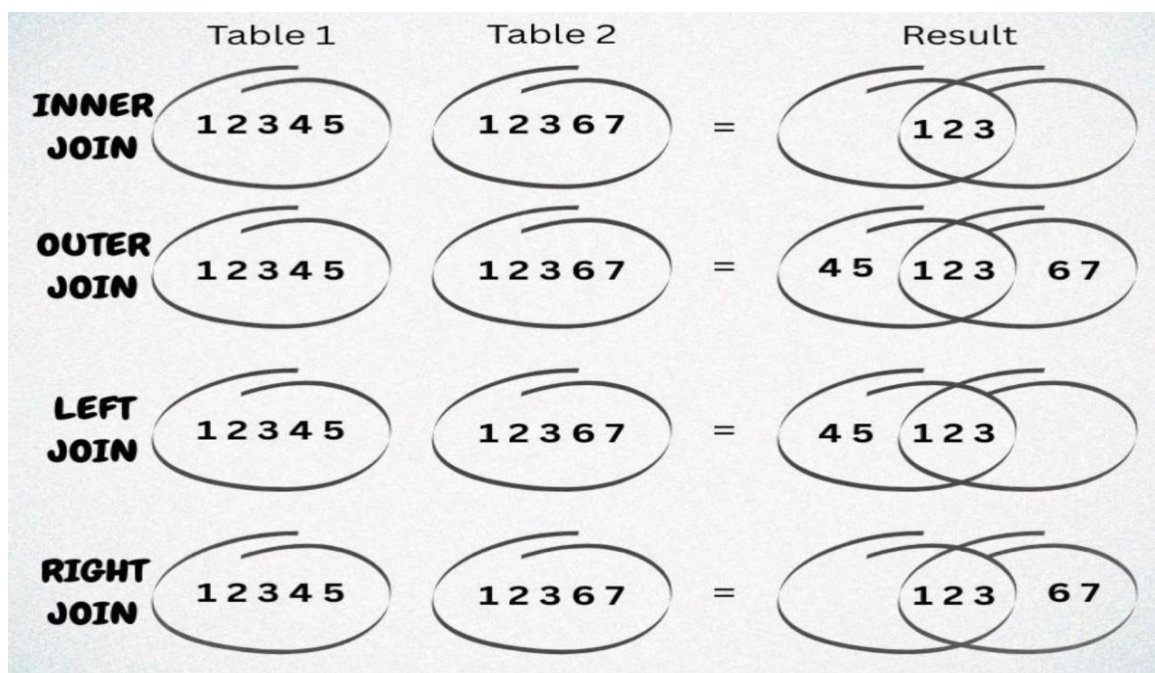
SQL Server joins are the backbone of combining data across tables. If your data model has normalized tables (which it usually should), joins are unavoidable—and using the right one matters for both correctness and performance.

Joins are the cornerstone of relational database querying. They allow you to combine rows from two or more tables based on related columns, enabling you to work with normalized data spread across multiple tables as if it were a single unified result set. SQL Server supports a rich set of join types — each designed for a specific data retrieval pattern.

This guide covers all eight join types available in SQL Server, providing definitions, syntax, real-world use cases, example queries, sample result sets, performance considerations, and key notes for each.

Types of Joins:

Join Type	Returns	NULL-safe?	Common Use Case
INNER JOIN	Only matching rows from both tables	No	Fetching related records
LEFT JOIN	All left rows + matched right (NULL if no match)	Yes	Optional relationships
RIGHT JOIN	All right rows + matched left (NULL if no match)	Yes	Rarely used; prefer LEFT
FULL OUTER JOIN	All rows from both, NULLs where no match	Yes	Finding all gaps/orphans
CROSS JOIN	Every combination (Cartesian product)	N/A	Test data, combinations
SELF JOIN	Table joined to itself	No	Hierarchical/recursive data
CROSS APPLY	Like INNER JOIN for table-valued functions	No	Row-by-row TVF calls
OUTER APPLY	Like LEFT JOIN for table-valued functions	Yes	Optional TVF results



Inner Join:

INNER JOIN returns only the rows where there is a matching value in both the left and right tables based on the join condition. Rows with no match in either table are excluded from the result set. It is the most commonly used join in SQL Server

Syntax:

SELECT columns

FROM TableA AS A

INNER JOIN TableB AS B

ON A.KeyColumn = B.KeyColumn;

Uses

- ✓ Retrieve records that exist in both tables (e.g., orders with a valid customer).
- ✓ Combine normalized tables to produce meaningful output.
- ✓ Filter out unmatched/orphan rows automatically.
- ✓ Build reports that require all foreign-key relationships to be satisfied.

Prerequisites

- ✓ Both tables must share a logically related column (primary key / foreign key or a business key).
- ✓ The join column data types must be compatible or explicitly cast.
- ✓ Indexes on join columns significantly improve performance.

Considerations

- ✓ Excludes any row from either table that does not satisfy the ON condition.
- ✓ If the join column contains NULLs, those rows are never matched (NULL ≠ NULL).
- ✓ Produces a Cartesian product multiplied by matching rows — be cautious with one-to-many relationships.
- ✓ Use query execution plans to verify index usage on large tables.

Note: INNER JOIN is the default join type in SQL Server; writing JOIN without a keyword is equivalent to INNER JOIN.

LEFT OUTER JOIN:

Definition:

LEFT JOIN returns all rows from the left (first) table and the matched rows from the right table. If there is no match in the right table, NULLs are returned for all right-table columns. No left-side row is ever omitted from the result.

Syntax

SELECT columns

FROM TableA AS A

LEFT JOIN TableB AS B -- or LEFT OUTER JOIN

ON A.KeyColumn = B.KeyColumn;

Uses

Find all records in the left table, including those with no related records on the right.

Detect orphan rows: WHERE B.KeyColumn IS NULL.

Optional relationship lookups (e.g., orders may or may not have a shipment).

Preserve the complete left dataset while enriching with optional right-side data.

Prerequisites

A clear understanding of which table is the 'driving' (left) table.

Compatible join column data types.

Awareness that NULL values on the right side indicate no match — not missing data in the source.

Considerations

NULLs on right-side columns indicate absence of a match, not actual NULL values in the data.

Adding a WHERE B.Column IS NULL effectively converts it to an anti-join (find unmatched rows).

Performance: large left tables with few matches still return all rows — ensure proper indexing.
Do not filter right-side columns in the WHERE clause (use the ON clause instead) to avoid inadvertently converting to an INNER JOIN.

Note LEFT OUTER JOIN and LEFT JOIN are synonymous in SQL Server. The OUTER keyword is optional.

RIGHT OUTER JOIN:

Definition:

RIGHT JOIN returns all rows from the right (second) table and the matched rows from the left table. If there is no match in the left table, NULLs are returned for all left-table columns. It is the mirror image of a LEFT JOIN.

Syntax:

```
SELECT columns  
FROM TableA AS A  
RIGHT JOIN TableB AS B -- or RIGHT OUTER JOIN  
ON A.KeyColumn = B.KeyColumn;
```

Uses

- ✓ Preserve all rows from the right table while optionally matching the left table.
- ✓ Identify right-table rows with no corresponding left-table records.
- ✓ Used when the right table is the primary/authoritative dataset.
- ✓ Can always be rewritten as a LEFT JOIN by swapping table order (preferred for readability).

Prerequisites

- ✓ Understand which table must be fully preserved (the right table in this case).
- ✓ Compatible join column data types.
- ✓ Typically avoided in favor of LEFT JOIN for consistency in query readability.

Considerations

- ✓ Most developers rewrite RIGHT JOINS as LEFT JOINS (swap table order) for consistency.
- ✓ Mixing LEFT and RIGHT JOINS in a single query reduces readability.
- ✓ NULLs on left-side columns indicate no matching row in the left table.
- ✓ Adding WHERE A.Column IS NULL finds right-table rows absent from the left.

Note The query 'FROM A RIGHT JOIN B ON ...' is exactly equivalent to 'FROM B LEFT JOIN A ON ...' — use LEFT JOIN for all outer joins to keep code uniform.

FULL OUTER JOIN (FULL JOIN):

Definition

FULL OUTER JOIN returns all rows from both the left and right tables. Where a match exists, columns from both sides are populated. Where no match exists on either side, the non-matching side's columns are returned as NULL. It is the union of LEFT JOIN and RIGHT JOIN results.

Syntax

```
SELECT columns  
FROM TableA AS A  
FULL OUTER JOIN TableB AS B -- or FULL JOIN  
ON A.KeyColumn = B.KeyColumn;
```

Uses

- ✓ Find all rows in both tables, including unmatched rows on either side.
- ✓ Data reconciliation: compare two datasets and highlight discrepancies.
- ✓ Detect orphan records in either table simultaneously.

- ✓ Merge or synchronize tables where either side may have extra rows.

Prerequisites

- ✓ Compatible join column data types.
- ✓ Understanding that NULLs will appear for unmatched columns on both sides.
- ✓ Consider using UNION of LEFT + RIGHT JOINS when FULL OUTER JOIN is not supported (e.g., MySQL).

Considerations

- ✓ Can generate very large result sets if both tables are large with few matches.
- ✓ Filter with COALESCE or ISNULL to handle NULLs in output columns.
- ✓ WHERE A.Key IS NULL OR B.Key IS NULL narrows results to unmatched rows only.
- ✓ Not supported in all databases (e.g., MySQL requires a UNION workaround).
- ✓ Index both join columns for acceptable performance on large tables.

Note FULL OUTER JOIN and FULL JOIN are synonymous. Use COALESCE(A.ID, B.ID) to create a non-NULL key column in the output.

CROSS JOIN:

Definition

CROSS JOIN produces the Cartesian product of two tables — every row in the left table is combined with every row in the right table. There is no ON condition. If the left table has M rows and the right has N rows, the result has $M \times N$ rows.

Syntax

```
SELECT columns
FROM TableA AS A
CROSS JOIN TableB AS B;
```

-- Equivalent implicit syntax (avoid in modern SQL):

```
SELECT columns FROM TableA, TableB;
```

Uses

- ✓ Generate all possible combinations (e.g., size × color for a product catalog).
- ✓ Create test/seed data by multiplying a small dataset.
- ✓ Build date-dimension grids or scheduling matrices.
- ✓ Pair every element with every other element for comparison purposes.

Prerequisites

- ✓ Full understanding of the row explosion: even modest tables create huge results.
- ✓ Ensure no accidental CROSS JOIN from a missing ON clause.
- ✓ Typically used on small tables or with additional filtering.

Considerations

- ✓ Row count explodes multiplicatively — $1,000 \times 1,000 = 1,000,000$ rows.
- ✓ Never use CROSS JOIN on large tables without a WHERE clause filter.
- ✓ An accidental CROSS JOIN from a missing ON condition causes serious performance issues.
- ✓ Verify intent: a WHERE clause after CROSS JOIN effectively turns it into an INNER JOIN.

Note An INNER JOIN with a condition of $1=1$ is functionally equivalent to a CROSS JOIN but should never be used — always use explicit CROSS JOIN syntax for clarity.

SELF JOIN:

Definition

A SELF JOIN is a join of a table with itself. It is not a distinct SQL keyword — it is achieved by aliasing the same table with two different names and then joining the aliases. It is used to compare rows within the same table or to traverse hierarchical/recursive data structures.

Syntax

```
SELECT A.Column1, B.Column2
```

```
FROM TableA AS A
```

```
INNER JOIN TableA AS B -- same table, different alias
```

```
ON A.RelatedColumn = B.PrimaryKey;
```

Uses

- ✓ Traverse employee-manager hierarchies stored in a single table.
- ✓ Find duplicate rows by comparing a table to itself.
- ✓ Build organizational charts or bill-of-materials structures.
- ✓ Compare adjacent rows (e.g., consecutive periods) within the same dataset.

Prerequisites

- ✓ The table must contain a self-referencing column (e.g., ManagerID referencing EmployeeID).
- ✓ Two distinct aliases are mandatory to differentiate the two 'copies' of the table.
- ✓ Deep or infinite recursion is better handled with recursive CTEs.

Considerations

- ✓ Use LEFT JOIN (not INNER JOIN) so the top-level record (no manager) is not excluded.
- ✓ For unlimited hierarchy depth, use a recursive CTE (WITH RECURSIVE).
- ✓ Ensure the self-referencing foreign key column is indexed.
- ✓ Avoid creating circular references in the data, which cause infinite loops.

Note: SELF JOIN is a logical concept, not a SQL Server keyword. Always assign meaningful aliases (e.g., 'E' for Employee, 'M' for Manager) for readability.

General Best Practices & Recommendations

Performance

- ✓ Always index join columns — both primary key and foreign key sides.
- ✓ Avoid functions on join column expressions (e.g., CAST, YEAR()) as they prevent index seeks.
- ✓ Use query execution plans (Ctrl+M in SSMS) to identify missing indexes and costly joins.
- ✓ Prefer set-based joins over cursor-based row-by-row processing.
- ✓ Filter rows as early as possible (in the WHERE clause or subquery) before joining.

Readability & Maintainability

- ✓ Use meaningful table aliases consistently throughout the query.
- ✓ Always use explicit JOIN syntax — avoid implicit comma-separated table lists.
- ✓ Prefer LEFT JOIN over RIGHT JOIN; swap table order instead for consistency.
- ✓ Place JOIN conditions in the ON clause; reserve WHERE for final result filtering.
- ✓ Comment complex joins explaining business logic, not just syntax.

Correctness

- ✓ Account for NULL values — NULLs never match in join conditions.
- ✓ Test with small datasets first to verify row counts before running on production.
- ✓ Beware of fan-out (duplicate rows) when joining on non-unique keys.

- ✓ Use DISTINCT or GROUP BY only when necessary — it often masks underlying data issues.
- ✓ Validate FULL OUTER JOIN results by checking both IS NULL conditions separately.

Examples: Use the below insert script, and practice below scenario to get clear idea.

```
CREATE TABLE TBL_1 (ID INT);
CREATE TABLE TBL_2 (ID INT);
```

```
INSERT INTO TBL_1 (ID) VALUES
(1),
(0),
(NULL),
(1),
(2),
(3);
```

```
INSERT INTO TBL_2 (ID) VALUES
(1),
(0),
(1),
(2),
(4),
(5),
(NULL);
```

INNER JOIN:

```
SELECT T1.[ID] AS T1_ID, T2.[ID] AS T2_ID
FROM [dbo].[TBL_01] AS T1
INNER JOIN
[dbo].[TBL_02] AS T2
ON T1.ID = T2.ID
```

T1_ID	T2_ID
1	1
1	1
0	0
1	1
1	1
2	2

LEFT OUTER JOIN:

```
SELECT T1.[ID] AS T1_ID, T2.[ID] AS T2_ID
FROM [dbo].[TBL_01] AS T1
LEFT JOIN
[dbo].[TBL_02] AS T2
ON T1.ID = T2.ID
```

T1_ID	T2_ID
1	1
1	1
0	0
NULL	NULL
1	1
1	1
2	2
3	NULL

RIGHT OUTER JOIN:

```
SELECT T1.[ID] AS T1_ID, T2.[ID] AS T2_ID
FROM [dbo].[TBL_01] AS T1
RIGHT JOIN
[dbo].[TBL_02] AS T2
ON T1.ID = T2.ID
```

T1_ID	T2_ID
1	1
1	1
0	0
1	1
1	1
2	2
NULL	4
NULL	5
NULL	NULL

FULL OUTER JOIN:

```
SELECT T1.[ID] AS T1_ID, T2.[ID] AS T2_ID
FROM [dbo].[TBL_01] AS T1
FULL OUTER JOIN
[dbo].[TBL_02] AS T2
ON T1.ID = T2.ID
```

T1_ID	T2_ID
1	1
1	1
0	0
NULL	NULL
1	1
1	1
2	2
3	NULL

NULL	4
NULL	5
NULL	NULL

SELF JOIN:

```
SELECT T1.[ID] AS T1_ID, T2.[ID] AS T2_ID
FROM [dbo].[TBL_01] AS T1
JOIN
[dbo].[TBL_02] AS T2
ON T1.ID = T2.ID
```

T1_ID	T2_ID
1	1
1	1
0	0
1	1
1	1
2	2

CROSS JOIN:

```
SELECT T1.[ID] AS T1_ID, T2.[ID] AS T2_ID
FROM [dbo].[TBL_01] AS T1
CROSS JOIN
[dbo].[TBL_02] AS T2
```

T1_ID	T2_ID
1	1
1	0
1	1
1	2
1	4
1	5
1	NULL
0	1
0	0
0	1
0	2
0	4
0	5
0	NULL
NULL	1
NULL	0
NULL	1
NULL	2
NULL	4

NULL	5
NULL	NULL
1	1
1	0
1	1
1	2
1	4
1	5
1	NULL
2	1
2	0
2	1
2	2
2	4
2	5
2	NULL
3	1
3	0
3	1
3	2
3	4
3	5
3	NULL

SET OPERATORS:

Defination:

Set Operators combine the result sets of two or more SELECT statements into a single result set.

The columns in each SELECT must match in number and have compatible data types.

Column names in the final result are taken from the first SELECT statement.

Operator	Returns	Duplicates?	Typical Use
UNION	All rows from both queries	No (removes duplicates)	Merge distinct records
UNION ALL	All rows from both queries	Yes (keeps duplicates)	Merge with performance priority
INTERSECT	Only rows common to BOTH queries	No	Find shared records
EXCEPT	Rows in first query NOT in second	No	Find exclusive records

UNION:

Definition

UNION combines the results of two or more queries and automatically removes duplicate records.

It performs a DISTINCT operation internally.

Syntax

SELECT COLUMN_NAMES

FROM TABLE1

UNION

SELECT COLUMN_NAMES

FROM TABLE2

Note:

If duplicate employee records exist, SQL Server removes duplicates automatically.
Both SELECT statements must have the same number of columns.
Columns must have compatible (matching or implicitly convertible) data types.
Column aliases from the FIRST SELECT are used in the output.

Use Case:

Combine results from two queries and eliminate duplicate rows.
Commonly used to merge data from similar tables or different filter conditions.

Example:

Retrieve all employees located in HYD or VIZ without any duplicates.

```
SELECT * FROM [dbo].[EMP]
WHERE [EMP_LOC] = 'HYD'
UNION
SELECT * FROM [dbo].[EMP]
WHERE [EMP_LOC] = 'VIZ'
```

Example:

Employee with highest AND lowest salary

```
SELECT [EMP_NAME], [EMP_SAL]
FROM [dbo].[EMP]
WHERE [EMP_SAL] = (SELECT MAX([EMP_SAL]) FROM [dbo].[EMP])
UNION
SELECT [EMP_NAME], [EMP_SAL]
FROM [dbo].[EMP]
WHERE [EMP_SAL] = (SELECT MIN([EMP_SAL]) FROM [dbo].[EMP]);
```

UNION ALL:

UNION ALL combines the results of multiple queries including duplicate records.
Unlike UNION, it does NOT remove duplicates.

Use Case:

Combine result sets including ALL duplicate rows.
Faster than UNION because no deduplication step is performed.

Example:

Retrieve employees from HYD and VIZ — intentionally keeping duplicates (e.g., an employee transferred between locations appearing in both result sets).

```
SELECT * FROM [dbo].[EMP]
WHERE [EMP_LOC] = 'HYD'
UNION ALL
SELECT * FROM [dbo].[EMP]
WHERE [EMP_LOC] = 'VIZ'
```

INTERSECT

Definition:

INTERSECT returns only the common records available in both query results.

Ideal for finding employees who are also managers, or customers who placed multiple order types.

Example:

Find employees who are also managers — EMP_ID must exist in both EMP_ID and MANAGER_ID columns.

```
SELECT DISTINCT [EMP_ID]
FROM [dbo].[EMP]
INTERSECT
SELECT DISTINCT [MANAGER_ID]
FROM [dbo].[EMP]
```

Only IDs that exist in both columns (i.e., managers)

EXCEPT**Definition:**

EXCEPT returns records from the first query that do not exist in the second query.

Useful for finding records exclusive to one dataset — e.g., employees who are NOT managers.

Example:

Identify employees who are NOT managing anyone — their EMP_ID does not appear as a MANAGER_ID.

```
SELECT DISTINCT [EMP_ID]
FROM [dbo].[EMP]
EXCEPT
SELECT DISTINCT [MANAGER_ID]
FROM [dbo].[EMP]
```

Iterative Statements:**Definition:**

Iterative Statements control the flow of execution in SQL Server.

They help SQL Server make decisions or repeatedly execute logic.

Main iterative/control statements:

- IF
- IF...ELSE
- ELSE IF
- WHILE

Iterative (control-flow) statements let you build conditional logic and loops inside T-SQL scripts, stored procedures, and triggers.

SQL Server supports: IF, IF...ELSE, IF...ELSE IF...ELSE, and WHILE loops.

Code blocks are enclosed in BEGIN...END and can be nested to any depth.

Prerequisites**Requirement Explanation**

Variables	Usually used with DECLARE
Conditions	Boolean expressions required
BEGIN...END	Used for multiple statements

IF Statements:**Definition**

Executes code only when a condition evaluates to TRUE.

Or

Execute a block of code only when a condition is TRUE.

Note:

Variables must be declared with DECLARE before use.

BEGIN...END is required when the IF block contains more than one statement

Example:

Check whether a person is eligible to vote based on their age.

```
DECLARE @AGE INT = 17
```

```
IF @AGE > 18
```

```
BEGIN
```

```
    PRINT 'ELIGIBLE TO VOTE'
```

```
END
```

IF ELSE Statement

Definition:

Used when two different outcomes are possible.

Evaluate multiple conditions sequentially and execute the first matching branch. Replaces nested IFs with cleaner, readable code.

Note:

Conditions are evaluated top-down. Put the MOST RESTRICTIVE condition first — otherwise lower conditions will incorrectly match.

Example:

Determine discount percentage based on purchase amount (30% > 5000, 20% > 3000, 15% > 2000, else 10%).

Determine bonus percentage for an employee based on their salary.

```
DECLARE @AMT INT = 200
```

```
IF @AMT > 5000
```

```
    BEGIN
```

```
        PRINT 'ELIGIBLE FOR 30% DISCOUNT'
```

```
    END
```

```
ELSE IF @AMT > 3000
```

```
    BEGIN
```

```
        PRINT 'ELIGIBLE FOR 20% DISCOUNT'
```

```
    END
```

```
ELSE IF @AMT > 2000
```

```
    BEGIN
```

```
        PRINT 'ELIGIBLE FOR 15% DISCOUNT'
```

```
    END
```

```
ELSE
```

```
    BEGIN
```

```
        PRINT 'ELIGIBLE FOR 10% DISCOUNT'
```

```
    END
```

Example:

Determine the percentage of Bonus for each Employee Based on Salary.

```
DECLARE @EID INT = 105
```

```
DECLARE @ESAL INT = (SELECT [EMP_SAL] FROM [dbo].[EMP]
                      WHERE [EMP_ID] = @EID)
```

```
IF @ESAL > 35000
    BEGIN
        PRINT 'ELIGIBLE FOR 35% BONUS'
    END
ELSE IF @ESAL > 30000
    BEGIN
        PRINT 'ELIGIBLE FOR 30% BONUS'
    END
ELSE IF @ESAL > 25000
    BEGIN
        PRINT 'ELIGIBLE FOR 25% BONUS'
    END
ELSE IF @ESAL > 20000
    BEGIN
        PRINT 'ELIGIBLE FOR 20% BONUS'
    END
ELSE
    BEGIN
        PRINT 'ELIGIBLE FOR 10% BONUS'
    BEGIN
```

WHILE Loop**Definition**

Repeats a block of statements until condition becomes FALSE.

Repeatedly execute a block of code as long as a condition remains TRUE.

Common in batch processing, generating test data, iterating over result sets.

Example:

Print an increasing pattern of 'A' characters — one A, then AA, then AAA, up to AAAAA.

```
DECLARE @I INT = 1
```

```
WHILE @I <= 5
    BEGIN
        PRINT REPLICATE('A', @I)
        SET @I = @I + 1
    END
```

Output

```
A
AA
AAA
AAAA
AAAAA
```

Views:

Definition:

A VIEW is a virtual (logical) table based on a stored SELECT query.

Views do NOT store data physically (except Indexed/Materialized Views).

They act as a mirror image of the underlying table(s) — data reflects real-time changes.

Views simplify complex queries, enforce security, and enable reusability across reports.

Use Case:

Hide complex joins from end users — expose a simple SELECT on the view instead. Restrict column access: give a user access to a view that excludes sensitive columns (e.g., salary).

Pre-aggregate data for dashboards and reports that run repeatedly.

Note:

Simple views are updatable (INSERT/UPDATE/DELETE) if they reference a single base table.

Views with GROUP BY, DISTINCT, or JOINS are generally NOT directly updatable.

Drop a view with: DROP VIEW [schema].[view_name]

Alter an existing view with: ALTER VIEW — avoids breaking dependent permissions.

Types of views:

Type	Data Storage	Updatable?	Performance	When to Use
Simple View	No (virtual)	Yes (single table)	Standard	Basic filtering / column masking
Complex View	No (virtual)	Generally No	Standard	Multi-table JOINS, aggregations
Indexed View (Materialized)	YES (physical)	No	Very Fast	Heavy aggregations repeated daily

Simple View:

Definition

A View created from a single table without aggregates or GROUP BY.

Example: Filter Hyderabad Employees.

Create a simple view:

```
CREATE VIEW vw_EMP_HYD
AS
SELECT *
FROM [dbo].[EMP]
WHERE [EMP_LOC] = 'HYD'
```

Query the view like a table:

```
SELECT * FROM [dbo].[vw_HYD_Employees];
```

Modify an existing view:

```
ALTER VIEW vw_HYD_Employees
AS
SELECT [EMP_ID], [EMP_NAME], [EMP_LOC], [EMP_SAL], [EMP_DOJ]
FROM [dbo].[EMP]
WHERE [EMP_LOC] = 'HYD';
```

Complex View:

Definition

A View created using:

- Joins
- Aggregations
- GROUP BY
- UNION
- Subqueries

Example:

City-wise Customer and Sales Metrics

```
CREATE VIEW vw_CITY_METRICS
AS
SELECT
DISTINCT C.[City],
COUNT(C.[CustomerID]) AS NO_OF_CUST,
COUNT(O.[OrderID]) AS CNT,
SUM(OI.[LineAmount]) AS TOTAL_LINE_AMT,
AVG(O.[OrderAmount]) AS AVG_ORDER_AMOUNT
FROM [dbo].[OrderItems] AS OI
LEFT JOIN [dbo].[Orders_] AS O
ON OI.[OrderID] = O.[OrderID]
LEFT JOIN [dbo].[Customers_] AS C
ON O.[CustomerID] = C.[CustomerID]
GROUP BY C.[City]
```

Example: Top highest salary earning employees for each department.

```
CREATE VIEW vwTOP3EMP_DEP
AS
(
SELECT T.[DEP_NAME], T.[EMP_NAME], T.[EMP_SAL]
FROM
(
SELECT D.[DEP_NAME], E.[EMP_NAME],E.[EMP_SAL],
DENSE_RANK() OVER( PARTITION BY D.[DEP_NAME] ORDER BY E.EMP_SAL DESC) AS RNK
FROM [dbo].[Dep] AS D
LEFT JOIN
[dbo].[EMP_001] AS E
ON D.DEP_ID = E.DEP_ID) AS T
WHERE T.RNK <= 3
```

)

Example:

Management dashboard showing:

- City performance
- Sales trends
- Customer metrics

```
CREATE VIEW vwSalesMetrics
AS
(
SELECT
YEAR([Order_Date]) AS YEAR,
CONVERT(VARCHAR(10), YEAR([Order_Date])) + '-Q' + CONVERT(VARCHAR(10), DATEPART(QQ, [Order_Date])) AS QTR,
FORMAT([Order_Date], 'yyyy-MMM') AS MONTH,
COUNT(*) AS NO_ORDERS,
ROUND(SUM([Gross_Sales]),0) AS TOTAL_GROSS_SALES,
ROUND(SUM([Net_Sales]),0) AS TOTAL_NET_SALES,
ROUND(SUM([Unit_Price]),0) AS TOTAL_UNIT_PRICE,
ROUND(SUM([Quantity]),0) AS TOTAL_QUANTIRY,
ROUND(SUM([Cost]),0) AS TOTAL_COST,
ROUND(SUM([Shipping_Cost]),0) AS TOTAL_SHIPPING_COST
FROM [dbo].[RetailSales]
GROUP BY
YEAR([Order_Date]),
CONVERT(VARCHAR(10), YEAR([Order_Date])) + '-Q' + CONVERT(VARCHAR(10), DATEPART(QQ, [Order_Date])),
FORMAT([Order_Date], 'yyyy-MMM')
)
```

Indexed (Materialized) Views:

Definition

An Indexed View stores data physically using a clustered index. Also called Materialized View.

Note:

View must be created WITH SCHEMABINDING (prevents underlying table changes without updating the view).

Must use COUNT_BIG(*) instead of COUNT(*) for row counting.

Requires a UNIQUE CLUSTERED INDEX on a key column.

Cannot use SELECT *, subqueries, derived tables, or OUTER JOINs.

Example:

Indexed Order Summary View

Step 1:

Create view WITH SCHEMABINDING

```
CREATE VIEW vw_Orders_Summary
WITH SCHEMABINDING
AS
SELECT
[Order_ID],
COUNT_BIG(*) AS ORDERS,      -- COUNT_BIG is mandatory
```

```
SUM([Sales]) AS TOTAL_SALES
FROM [dbo].[ORDERS_TEXT]
GROUP BY [Order_ID];
```

Step 2:

Create UNIQUE CLUSTERED INDEX (materializes the view)

```
CREATE UNIQUE CLUSTERED INDEX UCI_ORDER_ID
ON [dbo].[vw_Orders_Summary] ([Order_ID]);
```

Step 3:

Query it like a normal table — blazing fast!

```
SELECT * FROM [dbo].[vw_Orders_Summary];
```

Note: Indexed Views: Data is stored physically — you CANNOT drop the base table without first dropping the indexed view. Schema changes require dropping and recreating the view.

Views Comparison

Type	Stores Data?	DML (INSERT/UPDATE)?	SCHEMABINDING?	Best For
Simple	No (virtual)	Yes	No	Single table queries with filters
Complex	No (virtual)	No	No	Multi-table JOINS and aggregations
Indexed/Materialized	Yes (physical)	No	Mandatory	Heavy read workloads, 100M+ rows

Common View Commands:

Command	Description
CREATE VIEW vw_name AS ...	Create a new view
ALTER VIEW vw_name AS ...	Modify an existing view
DROP VIEW vw_name	Delete a view
SELECT * FROM vw_name	Query a view like a table
WITH SCHEMABINDING	Prevents changes to underlying table columns

Stored Procedure:

A Stored Procedure (SP) is a pre-compiled, named collection of T-SQL statements stored in the database.

SPs accept INPUT parameters (values passed IN) and OUTPUT parameters (values passed OUT to the caller).

Unlike functions, stored procedures can include DML (INSERT/UPDATE/DELETE), DDL, PRINT, and complex control-flow logic.

They are executed with the EXEC (or EXECUTE) keyword

Syntax:

```
CREATE PROCEDURE procedure_name
```

```
(
    @param1 DataType,          -- input parameter (default)
    @param2 DataType OUTPUT    -- output parameter
)
AS
BEGIN
    -- SQL statements
END
-- Execute
EXEC procedure_name @param1 = value
```

Use Case:

Encapsulate business logic that multiple applications or reports reuse.

ATM banking transactions: validate account, PIN, balance, then credit or debit — all in one procedure.

Age eligibility checker: pass a date of birth, get back eligibility status.

Parameterized reports: pass date ranges, department IDs, etc

Example 1:

Input Parameter: Age Eligibility Check

```
CREATE PROCEDURE sp_CHECKAGE
(
    @DOB DATE
)
AS
BEGIN
    DECLARE @AGE INT
    SET @AGE = DATEDIFF(YYYY, @DOB, GETDATE())

    IF @AGE >= 18
    BEGIN
        PRINT 'ELIGIBLE TO VOTE'
    END
    ELSE
    BEGIN
        PRINT 'NOT ELIGIBLE TO VOTE, AGE IS '
        + CONVERT(VARCHAR(10), @AGE)
    END
END

-- Execute
EXEC [dbo].[sp_CHECKAGE] @DOB = '2015-10-20'
```

Example 2:

Stored Procedure with No Parameters

The simplest form of a stored procedure takes no parameters and simply runs a fixed query whenever it is executed.

Example — aggregate employee metrics per location:

```
CREATE PROCEDURE sp_EMPMETRICS
AS
BEGIN
    SELECT EMP_LOC,
        SUM(EMP_SAL) AS SALARY,
```

```

COUNT(*) AS EMPLOYEES,
AVG(EMP_SAL) AS AVG_SALARY
FROM EMP
GROUP BY EMP_LOC
END

```

```
EXEC sp_EMPMETRICS
```

Example 3:

Stored Procedure with an Input Parameter

Input parameters let the same procedure logic run against different values each time it is called, without rewriting the SQL.

Example — check voting eligibility from a date of birth:

```

CREATE PROCEDURE sp_CHECKAGE
(
    @DOB DATE
)
AS
BEGIN
    DECLARE @AGE INT
    SET @AGE = DATEDIFF(YYYY, @DOB, GETDATE())

    IF @AGE >= 18
    BEGIN
        PRINT 'ELIGIBLE TO VOTE'
    END
    ELSE
    BEGIN
        PRINT CONCAT('NOT ELIGIBLE TO VOTE ', 'AGE IS ', @AGE)
    END
END

EXEC sp_CHECKAGE @DOB = '2008-10-17'

```

Example :4

Stored Procedure with OUTPUT Parameter

Get Employee Salary via Output Parameter

```

-- Create SP with an OUTPUT parameter
CREATE PROCEDURE sp_GetSalary
(
    @EMP_ID INT,      -- Input parameter
    @EMP_SAL INT OUTPUT -- Output parameter
)
AS
BEGIN
    SELECT @EMP_SAL = [EMP_SAL]
    FROM [dbo].[EMP]

```

```

WHERE [EMP_ID] = @EMP_ID;
END;

-- Execute and capture the OUTPUT value
DECLARE @MySalary INT;

EXEC [dbo].[sp_GetSalary]
    @EMP_ID = 105,
    @EMP_SAL = @MySalary OUTPUT;

PRINT 'Salary of Employee 105: ' + CONVERT(VARCHAR, @MySalary);

```

Example 5:

Stored Procedure with Input and Output Parameters

Output parameters allow a procedure to return computed values back to the caller, in addition to (or instead of) a result set. This is useful when the calling code needs specific scalar values, such as before/after amounts, rather than a full row set.

Requirement demonstrated: given an employee ID and a salary increment percentage, update the employee's salary and return both the old and new salary as output parameters.

Example:

```

CREATE PROCEDURE sp_EMPSAL
(
    @EID INT,
    @SALINC INT,
    @OSAL INT OUTPUT,
    @NSAL INT OUTPUT
)
AS
BEGIN
    SET @OSAL = (SELECT EMP_SAL FROM EMP WHERE EMP_ID = @EID)
    SET @NSAL = @OSAL + (@OSAL * @SALINC / 100)
    UPDATE EMP SET EMP_SAL = @NSAL WHERE EMP_ID = @EID
END

DECLARE @OLDSAL INT
DECLARE @NEWSAL INT

EXEC sp_EMPSAL
    @EID = 1002,
    @SALINC = 20,
    @OSAL = @OLDSAL OUTPUT,
    @NSAL = @NEWSAL OUTPUT

SELECT @OLDSAL AS OLD_SALARY, @NEWSAL AS NEW_SALARY

```

Example 5:

Multiple Input Parameters: ATM Banking Transaction

Simulate an ATM machine: validate account number, PIN, check sufficient balance for DEBIT, then update the balance and insert a transaction record.

Tables involved: ACCOUNT_INFO (stores balances), CUSTOMERS_BANK (customer details), TRANSACTION_TBL (transaction history).

```
CREATE PROCEDURE sp_TRANSACTIONS
(
    @ACNO  BIGINT,
    @PIN   INT,
    @T_TYPE VARCHAR(20),
    @T_AMT INT
)
AS
BEGIN
    -- Step 1: Validate account number
    IF (SELECT COUNT(1) FROM [dbo].[ACCOUNT_INFO] WHERE [AC_NO] = @ACNO) < 1
    BEGIN
        PRINT 'INCORRECT ACCOUNT NUMBER'; RETURN
    END

    -- Step 2: Validate PIN
    ELSE IF (SELECT COUNT(1) FROM [dbo].[ACCOUNT_INFO] WHERE [ATM_PIN] = @PIN) < 1
    BEGIN
        PRINT 'INCORRECT PIN NUMBER'; RETURN
    END

    -- Step 3: Check balance for DEBIT
    ELSE IF @T_TYPE = 'DEBIT'
        AND (SELECT MAX([BALANCE]) FROM [dbo].[ACCOUNT_INFO] WHERE [AC_NO] = @ACNO) < @T_AMT
    BEGIN
        PRINT 'INSUFFICIENT BALANCE IN YOUR ACCOUNT'; RETURN
    END

    ELSE
    BEGIN
        -- Step 4: Update balance
        IF @T_TYPE = 'DEBIT'
            UPDATE [dbo].[ACCOUNT_INFO]
            SET [BALANCE] = [BALANCE] - @T_AMT WHERE [AC_NO] = @ACNO

        IF @T_TYPE = 'CREDIT'
            UPDATE [dbo].[ACCOUNT_INFO]
            SET [BALANCE] = [BALANCE] + @T_AMT WHERE [AC_NO] = @ACNO

        -- Step 5: Generate transaction ID and log
        DECLARE @C_NAME  VARCHAR(100)
        DECLARE @T_DATE  DATETIME
        DECLARE @T_ID    VARCHAR(15)
        DECLARE @FINBALANCE INT

        SET @C_NAME = (SELECT C.[C_NAME]
                       FROM [dbo].[CUSTOMERS_BANK] AS C
                       INNER JOIN [dbo].[ACCOUNT_INFO] AS A ON C.[C_ID] = A.[C_ID])
```

```

        WHERE A.[AC_NO] = @ACNO)
SET @T_DATE = GETDATE()
SET @T_ID = UPPER(LEFT(@C_NAME,2)) + UPPER(RIGHT(@C_NAME,1))
        + CONVERT(VARCHAR(10), DATEPART(DAY, @T_DATE))
        + CONVERT(VARCHAR(10), DATEPART(HH, @T_DATE))
        + CONVERT(VARCHAR(10), DATEPART(MI, @T_DATE))
        + CONVERT(VARCHAR(10), DATEPART(SS, @T_DATE))
        + CONVERT(VARCHAR(10), DATEPART(MILLISECOND, @T_DATE))
SET @FINBALANCE = (SELECT MAX([BALANCE]) FROM [dbo].[ACCOUNT_INFO]
        WHERE [AC_NO] = @ACNO)

INSERT INTO [dbo].[TRANSACTION_TBL]
VALUES (@C_NAME, @ACNO, @T_DATE, @T_ID, @T_TYPE, @T_AMT, @FINBALANCE)

PRINT 'TRANSACTION COMPLETED SUCCESSFULLY!!!'
END
END

-- Execute (Credit Rs 3000)
EXEC [dbo].[sp_TRANSACTIONS]
    @ACNO = 36535468414,
    @PIN = 3086,
    @T_TYPE = 'CREDIT',
    @T_AMT = 3000

```

Stored Procedure with OUTPUT Parameter

Example:

Get Employee Salary via Output Parameter

```

-- Create SP with an OUTPUT parameter
CREATE PROCEDURE sp_GetSalary
(
    @EMP_ID INT,      -- Input parameter
    @EMP_SAL INT OUTPUT -- Output parameter
)
AS
BEGIN
    SELECT @EMP_SAL = [EMP_SAL]
    FROM [dbo].[EMP]
    WHERE [EMP_ID] = @EMP_ID;
END;

-- Execute and capture the OUTPUT value
DECLARE @MySalary INT;

EXEC [dbo].[sp_GetSalary]
    @EMP_ID = 105,
    @EMP_SAL = @MySalary OUTPUT;

PRINT 'Salary of Employee 105: ' + CONVERT(VARCHAR, @MySalary);

```

Common Use Cases of Stored Procedures:

- ✓ Encapsulating reusable business logic (e.g., salary increments, eligibility checks) at the database layer so multiple applications share the same logic consistently.
- ✓ Improving performance through pre-compilation and reduced network round-trips compared to sending raw ad-hoc SQL each time.
- ✓ Returning computed scalar results (via OUTPUT parameters) alongside or instead of full result sets, useful for confirmations like old/new values.
- ✓ Centralizing and securing data access — users can be granted EXECUTE rights on a procedure without direct table access.
- ✓ Simplifying scheduled/automated jobs (e.g., nightly metric aggregation) by wrapping the logic in a single callable unit.

Quick Comparison: SP vs View vs Trigger:

Feature	Stored Procedure	View	Trigger
Called by	EXEC keyword	SELECT statement	Automatically on DML
Accepts params?	Yes (IN and OUT)	No	No
Contains DML?	Yes	No	Yes
Stores data?	No (logic only)	No (Indexed: Yes)	No (logic only)
Common use	Business logic, ETL	Simplified queries	Auditing, validation

Stored Procedure Commands

Command	Description
CREATE PROCEDURE name (...) AS BEGIN...END	Define a new stored procedure
ALTER PROCEDURE name (...) AS BEGIN...END	Modify an existing stored procedure
DROP PROCEDURE name	Delete a stored procedure
EXEC / EXECUTE name @param=value	Run a stored procedure
@param DataType	Input parameter — value passed in
@param DataType OUTPUT	Output parameter — returns value to caller
RETURN	Exit the procedure immediately

Triggers:

Definition:

A TRIGGER is a special stored procedure that executes AUTOMATICALLY in response to a data modification event (INSERT, UPDATE, DELETE) on a table or view.

Triggers cannot be called manually — they fire implicitly when the triggering event occurs.

SQL Server provides two special virtual tables: INSERTED (holds new/updated rows) and DELETED (holds old/deleted rows).

Syntax:

CREATE TRIGGER trigger_name

ON table_name

AFTER INSERT | UPDATE | DELETE -- or INSTEAD OF

```

AS
BEGIN
    -- Use INSERTED and DELETED virtual tables
END

```

Types of Triggers

Trigger Type	When It Fires	Common Use
AFTER INSERT	After a row is inserted	Log new records, send notifications
AFTER UPDATE	After a row is updated	Audit trail of changes
AFTER DELETE	After a row is deleted	Log deleted records, archive data
INSTEAD OF INSERT	Replaces the INSERT operation	Custom insert logic on views
INSTEAD OF UPDATE	Replaces the UPDATE operation	Validate before updating
INSTEAD OF DELETE	Replaces the DELETE operation	Soft delete / archive instead

Magic Tables — INSERTED & DELETED

SQL Server provides two virtual tables inside every trigger:

Operation	INSERTED Table	DELETED Table
INSERT	Contains newly added rows	Empty
DELETE	Empty	Contains removed rows
UPDATE	Contains new (updated) values	Contains old (before-update) values

Use Case:

Audit trail: log every INSERT/UPDATE/DELETE automatically into a history/log table.

Enforce complex business rules that cannot be implemented using constraints.

Cascade custom actions: e.g., archive a record before deleting it.

Prevent invalid operations using INSTEAD OF triggers.

For Practice Create Below Tables:

-- Supporting tables

```

CREATE TABLE T_EMP (
    EMP_ID      INT,
    EMP_NAME    VARCHAR(100),
    EMP_LOC     VARCHAR(10),
    EMP_DOJ     DATE,
    EMP_SAL     INT,
    MANAGER_ID  INT
);

```

```

CREATE TABLE T_EMP_LOG (
    EMP_ID      INT,
    DATE_INSERTED DATETIME,
    DESCRIPTION  VARCHAR(100)
);

```

AFTER INSERT Trigger — Audit Log

Example:

Automatically log every new employee added to T_EMP

-- Create the AFTER INSERT trigger

```
CREATE TRIGGER tr_EMPLOG
ON [dbo].[T_EMP]
AFTER INSERT
AS
BEGIN
    INSERT INTO [dbo].[T_EMP_LOG] (EMP_ID, DATE_INSERTED, DESCRIPTION_)
    SELECT
        [EMP_ID],
        GETDATE(),
        'NEW EMPLOYEE ADDED: ' + CONVERT(VARCHAR(10), [EMP_ID])
    FROM INSERTED; -- 'INSERTED' holds the newly added rows
END;
```

-- Test: Insert fires the trigger automatically

```
INSERT INTO [dbo].[T_EMP]
VALUES (101, 'Lohit', 'HYD', '2026-01-10', 30000, 101);
```

-- Verify: Check the audit log

```
SELECT * FROM [dbo].[T_EMP_LOG];
```

AFTER DELETE Trigger — Deletion Log

Example:

Log every deleted employee record

```
CREATE TRIGGER tr_EMPLOG_D
ON [dbo].[T_EMP]
AFTER DELETE
AS
BEGIN
    INSERT INTO [dbo].[T_EMP_LOG] (EMP_ID, DATE_INSERTED, DESCRIPTION_)
    SELECT
        [EMP_ID],
        GETDATE(),
        'EMPLOYEE DELETED: ' + CONVERT(VARCHAR(10), [EMP_ID])
    FROM DELETED; -- 'DELETED' holds the rows that were removed
END;
```

-- Test: Delete triggers the log trigger

```
DELETE FROM [dbo].[T_EMP] WHERE EMP_ID = 101;
```

```
SELECT * FROM [dbo].[T_EMP_LOG]; -- Should show deletion record
```

AFTER UPDATE Trigger — Update Log

Example:**Log every updated employee record**

```

CREATE TRIGGER tr_emp_log_UPDATE
ON [dbo].[T_EMP]
AFTER UPDATE
AS
BEGIN

    INSERT INTO [dbo].[T_EMP_LOG]
    (
        EMP_ID,
        DATE_INSERTED,
        DESCRIPTION_
    )
    SELECT
        EMP_ID,
        GETDATE(),
        'OLD SAL - ' + CONVERT(VARCHAR(10), DELETED_SAL) +
        ' UPDATED SALARY - ' + CONVERT(VARCHAR(10), UPDATED_SAL) +
        ' EMPID - ' + CONVERT(VARCHAR(10), EMP_ID)
    FROM
    (
        SELECT
            D.EMP_ID,
            D.EMP_SAL AS DELETED_SAL,
            I.EMP_SAL AS UPDATED_SAL
        FROM DELETED AS D
        INNER JOIN INSERTED AS I
            ON D.EMP_ID = I.EMP_ID
    ) AS A;

END;

```

-- Test: Update triggers the log trigger

```
UPDATE [dbo].[T_EMP] SET EMP_SAL = 50000 WHERE EMP_ID = 101
```

```
SELECT * FROM [dbo].[T_EMP_LOG] -- Should show updated record
```

Indexes:**Definition:**

An index is a database object that speeds up data retrieval by creating a sorted pointer structure on one or more columns — similar to an index at the back of a book. Improves SELECT performance at the cost of slightly slower writes.

Types of Indexes**1. Clustered Index (CI)**

Definition: Physically sorts and stores the table data rows based on the key column(s). Only ONE clustered index is allowed per table.

Use Case: Primary key columns, columns used in range queries and ORDER BY.

Syntax:

```
CREATE CLUSTERED INDEX index_name  
ON table_name (column_name)
```

Example:

```
CREATE CLUSTERED INDEX CI_EMP_001  
ON [dbo].[EMP_001]([EMP_ID])
```

2. Non-Clustered Index (NCI)

Definition: Creates a separate structure with pointers to actual data rows. Multiple non-clustered indexes are allowed per table (up to 999).

Use Case: Frequently searched or filtered columns like email, phone, or name.

Syntax:

```
CREATE NONCLUSTERED INDEX index_name  
ON table_name (column_name)
```

Example:

```
CREATE NONCLUSTERED INDEX NCI_EMP_001  
ON [dbo].[EMP_001]([EMP_MAIL])
```

3. Unique Clustered Index (UCI)

Definition: A clustered index that also enforces uniqueness on the key column. Required for creating Indexed (Materialized) Views.

Syntax:

```
CREATE UNIQUE CLUSTERED INDEX index_name  
ON table_name (column_name)
```

Example (on an Indexed View):

```
CREATE UNIQUE CLUSTERED INDEX UCI_ORDER_ID  
ON [dbo].[vw_Orders1] ([Order_ID])
```

4. Clustered Columnstore Index (CCI)

Definition: Stores data by column rather than by row. Extremely fast for analytical and aggregate queries on large datasets.

Use Case: Data warehouse, reporting, and analytics workloads.

Syntax:

```
CREATE CLUSTERED COLUMNSTORE INDEX index_name  
ON table_name
```

Example:

```
CREATE CLUSTERED COLUMNSTORE INDEX CCI_EMP_001  
ON [dbo].[EMP_002]([EMP_ID])
```

Adding a Primary Key (Auto-creates Clustered Index)

-- Option 1: At table creation

```
CREATE TABLE T_EMP_001  
(
```

```

EMP_ID INT CONSTRAINT T_EMP_PK PRIMARY KEY,
EMP_NAME VARCHAR(100),
EMP_SAL INT
)

```

-- Option 2: Alter existing table

```

ALTER TABLE [dbo].[EMP]
ADD CONSTRAINT EMP_PK PRIMARY KEY ([EMP_ID])

```

Index Comparison

Index Type	Per Table	Sorts Rows Physically?	Allows Duplicates?	Best Used For
Clustered (CI)	1 only	Yes	Yes (unless UNIQUE)	Primary key, range queries
Non-Clustered (NCI)	Up to 999	No	Yes	Frequently filtered columns
Unique Clustered (UCI)	1 only	Yes	No (enforces uniqueness)	Indexed views, unique keys
Columnstore (CCI)	1 only	Column-based	Yes	Analytics, aggregations, DWH

Index Commands

Command	Description
CREATE [UNIQUE] [CLUSTERED] INDEX name ON table (col)	Create an index
DROP INDEX name ON table	Delete an index
ALTER INDEX name ON table REBUILD	Rebuild index (defragment fully)
ALTER INDEX name ON table REORGANIZE	Lightweight defragmentation
ALTER INDEX ALL ON table REBUILD	Rebuild all indexes on a table

SQL TOPICS & THEIR DEFINITIONS

What is RDBMS?

RDBMS (Relational Database Management System) is software that stores and manages data in the form of structured tables made up of rows and columns. Data across tables is linked through relationships using keys, enabling efficient querying and data integrity. Examples include SQL Server, MySQL, Oracle, and PostgreSQL.

What is SQL Server?

SQL Server is a relational database management system (RDBMS) developed by Microsoft that stores, retrieves, and manages structured data using the T-SQL (Transact-SQL) language. It provides features like security, transactions, indexing, and high availability, making it suitable for enterprise-scale applications and data warehousing.

What is a Database?

A database is an organized collection of structured data stored electronically, managed by a DBMS. In SQL Server, a database contains objects like tables, views, stored procedures, indexes, and functions. It provides a logical container that separates data from different applications or domains.

What is a Table?

A table is the fundamental storage unit in a relational database, organized into rows (records) and columns (fields). Each column has a specific data type and name, while each row represents a single data entry. Tables can be related to other tables using primary and foreign keys.

Database Objects

What is a View?

A view is a virtual table created by saving a SELECT query in the database. It does not store data physically (unless it is an indexed view) but presents data from one or more underlying tables. Views simplify complex queries, improve security by exposing only specific columns, and act as a mirror of the underlying data.

What is a Stored Procedure?

A stored procedure is a precompiled, named set of SQL statements stored in the database that can be executed repeatedly. It accepts input and output parameters, encapsulates business logic, and improves performance through execution plan caching. Examples include procedures for inserting records, validating data, or processing transactions.

What is an Index?

An index is a database object that creates a sorted pointer structure on one or more columns of a table to speed up data retrieval — similar to the index at the back of a book. While indexes significantly improve SELECT query performance, they add slight overhead to INSERT, UPDATE, and DELETE operations. Types include Clustered, Non-Clustered, and Columnstore indexes.

What is a Trigger?

A trigger is a special stored procedure that automatically executes (fires) in response to a specific DML event (INSERT, UPDATE, or DELETE) on a table or view. Triggers are used for auditing changes, enforcing business rules, and maintaining data consistency without requiring explicit calls. They have access to special virtual tables called INSERTED and DELETED.

Data Types

What is a Data Type?

A data type defines the kind of data a column or variable can store, such as numbers, text, or dates. SQL Server uses data types to allocate appropriate storage, validate input, and optimize query performance. Choosing the correct data type is critical for storage efficiency and data accuracy.

Numeric Data Types

What is the INT Data Type?

INT (Integer) stores whole numbers ranging from -2,147,483,648 to 2,147,483,647 and uses 4 bytes of storage. It is the most commonly used numeric data type for IDs, counts, and quantities. Use INT when values will not exceed approximately 2.1 billion.

What is the BIGINT Data Type?

BIGINT stores very large whole numbers ranging from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807, using 8 bytes. It is ideal for columns like account numbers, transaction IDs, or row counts in very large tables where INT would overflow.

What is the SMALLINT Data Type?

SMALLINT stores whole numbers from -32,768 to 32,767 using only 2 bytes of storage. It is suitable for columns with small numeric ranges such as age, month, or small category codes, saving storage space compared to INT.

What is the TINYINT Data Type?

TINYINT stores non-negative whole numbers from 0 to 255 using only 1 byte of storage. It is the smallest integer data type in SQL Server and is ideal for columns like status flags, boolean-like values, or small numeric codes.

What is the BIT Data Type?

BIT is a data type that stores 0, 1, or NULL, effectively representing a true/false or yes/no value. SQL Server optimizes storage by packing up to 8 BIT columns into a single byte. It is commonly used for boolean flags such as IsActive, IsDeleted, or IsApproved.

What is the DECIMAL Data Type?

DECIMAL stores fixed-precision and fixed-scale numbers with exact precision, defined as DECIMAL(precision, scale) — for example, DECIMAL(10, 2) stores up to 10 digits with 2 after the decimal point. It is the preferred type for financial and monetary values where precision is critical.

What is the NUMERIC Data Type?

NUMERIC is functionally identical to DECIMAL in SQL Server and also stores fixed-precision numbers. Both NUMERIC(p, s) and DECIMAL(p, s) behave the same way and can be used interchangeably. The distinction is historical — both are defined in the SQL standard.

What is the FLOAT Data Type?

FLOAT is an approximate numeric data type that stores floating-point numbers with up to 15 significant digits, using 4 or 8 bytes depending on the precision specified. Because it uses binary approximation, it may have minor rounding errors, making it unsuitable for exact financial calculations.

What is the REAL Data Type?

REAL is a single-precision floating-point data type that stores approximate numeric values using 4 bytes with up to 7 significant digits. It is equivalent to FLOAT(24) and is used for scientific or engineering calculations where minor imprecision is acceptable and storage efficiency matters.

Date & Time Data Types

What is the DATETIME Data Type?

DATETIME stores both date and time values ranging from January 1, 1753 to December 31, 9999, with accuracy to 3.33 milliseconds, using 8 bytes. It is widely used for timestamps, transaction dates, and event logs in SQL Server applications.

What is the DATE Data Type?

DATE stores only the date portion (year, month, day) with a range from 0001-01-01 to 9999-12-31, using just 3 bytes. It is the ideal choice when time information is not needed, such as for birthdates, hire dates, or order dates.

What is the SMALLDATETIME Data Type?

SMALLDATETIME stores date and time values from January 1, 1900 to June 6, 2079 with accuracy to the nearest minute, using 4 bytes. It uses less storage than DATETIME but sacrifices range and precision, suitable for scheduling data with minute-level accuracy.

What is the TIME Data Type?

TIME stores only the time of day (without a date) in the format HH:MM:SS.nnnnnnn, with precision up to 100 nanoseconds. It uses 3 to 5 bytes depending on the fractional seconds precision and is ideal for storing shift times, durations, or schedules.

What is the DATETIMEOFFSET Data Type?

DATETIMEOFFSET stores date and time combined with time zone offset information in the format YYYY-MM-DD HH:MM:SS +/-HH:MM. It is used in global applications where data needs to be stored and compared across multiple time zones consistently.

String Data Types

What is the CHAR Data Type?

CHAR is a fixed-length character data type that always stores exactly the number of characters specified, padding with spaces if the input is shorter — for example, CHAR(10) always uses 10 bytes. It is best suited for columns with consistent, fixed-length values like country codes, gender codes, or ZIP codes.

What is the VARCHAR Data Type?

VARCHAR is a variable-length character data type that stores only the actual number of characters entered, plus 2 bytes of overhead — for example, VARCHAR(100) uses only as much space as the data requires. It is the most commonly used string type for names, addresses, descriptions, and other variable-length text.

What is the VARCHAR(MAX) Data Type?

VARCHAR(MAX) is a variable-length string type that can store up to 2 GB (approximately 2.1 billion characters) of non-Unicode text. It is used for very large text fields such as document content, log entries, or JSON/XML data where the length is unpredictable and may exceed 8,000 characters.

What is the TEXT Data Type?

TEXT is a legacy data type that stores large non-Unicode character data up to 2 GB. It has been deprecated in modern SQL Server versions and should be replaced with VARCHAR(MAX). TEXT does not support many string functions and cannot be used in variables or most expressions directly.

What is the NCHAR Data Type?

NCHAR is a fixed-length Unicode character data type that stores multilingual text using 2 bytes per character — for example, NCHAR(10) always uses 20 bytes. The 'N' prefix stands for National Character Set, making it suitable for storing text in languages like Hindi, Arabic, Chinese, or Japanese.

What is the NVARCHAR Data Type?

NVARCHAR is a variable-length Unicode character data type that stores multilingual text using 2 bytes per character, only consuming space for the actual characters stored. It supports up to 4,000 characters (or 2 GB with NVARCHAR(MAX)) and is recommended for any column that may contain international or multilingual text.

What is the NTEXT Data Type?

NTEXT is a legacy data type for storing large Unicode text up to 2 GB, using 2 bytes per character. It has been deprecated and should be replaced with NVARCHAR(MAX) in modern SQL Server applications. Like TEXT, it has limited support for string functions and cannot be used in variables.

Constraints

What is a Constraint?

A constraint is a rule enforced on a column or table to maintain data accuracy, integrity, and consistency. SQL Server supports six types of constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT, and NOT NULL. Constraints prevent invalid data from being inserted or updated in the database.

What is a PRIMARY KEY Constraint?

A PRIMARY KEY constraint uniquely identifies each row in a table and ensures that no two rows have the same key value. A table can have only one primary key, which cannot contain NULL values. SQL Server automatically creates a clustered index on the primary key column by default.

What is a FOREIGN KEY Constraint?

A FOREIGN KEY constraint enforces a referential relationship between two tables by requiring that values in the foreign key column match values in the referenced primary key column of another table. It prevents orphan records and ensures data consistency across related tables. For example, an EMP table's DEPT_ID must match a valid DEPT_ID in the DEPT table.

What is a UNIQUE Constraint?

A UNIQUE constraint ensures that all values in a column (or combination of columns) are distinct across the table, preventing duplicate entries. Unlike a PRIMARY KEY, a UNIQUE constraint allows one NULL value. SQL Server automatically creates a non-clustered index to enforce uniqueness.

What is a CHECK Constraint?

A CHECK constraint defines a condition that every row must satisfy before data is inserted or updated. If the condition evaluates to FALSE, the operation is rejected. For example, CHECK(EMP_AGE >= 18) ensures no employee under 18 can be added to the table.

What is a DEFAULT Constraint?

A DEFAULT constraint automatically inserts a predefined value into a column when no value is provided during an INSERT operation. For example, DEFAULT(GETDATE()) will insert the current date and time if the user does not supply one. It helps maintain consistency without requiring explicit values every time.

What is a NOT NULL Constraint?

A NOT NULL constraint ensures that a column cannot store NULL values, making data entry mandatory for that field. It is applied at the column level and rejects any INSERT or UPDATE that tries to leave the column empty. For example, EMP_NAME VARCHAR(100) NOT NULL means every employee must have a name.

What is NULL in SQL Server?

NULL represents the absence of a value or unknown data in a column — it is not zero, not an empty string, and not a blank space. NULL is used when a value is missing, not applicable, or not yet known. Any arithmetic or comparison operation involving NULL returns NULL unless handled with ISNULL() or COALESCE().

SQL Command Categories

What is a SQL Command?

A SQL command is an instruction given to the database engine to perform a specific operation on data or database objects. SQL Server commands are grouped into five categories: DDL (Data Definition Language), DML (Data Manipulation Language), DQL (Data Query Language), DCL (Data Control Language), and TCL (Transaction Control Language).

What is DDL (Data Definition Language)?

DDL consists of SQL commands used to define, create, modify, and delete the structure of database objects like tables, views, and indexes. DDL commands are auto-committed, meaning changes take effect immediately and cannot be rolled back. Key DDL commands are CREATE, ALTER, DROP, and TRUNCATE.

What is DML (Data Manipulation Language)?

DML consists of commands used to insert, update, and delete data within existing database tables. Unlike DDL, DML commands are not automatically committed and can be rolled back within a transaction. The primary DML commands are INSERT, UPDATE, and DELETE.

What is DQL (Data Query Language)?

DQL refers to commands used to retrieve or query data from the database without modifying it. The SELECT statement is the only DQL command and is the most frequently used SQL statement. It allows filtering, sorting, joining, grouping, and aggregating data from one or more tables.

What is DCL (Data Control Language)?

DCL consists of commands used to control access and permissions on database objects. It defines who can read, modify, or execute specific objects within the database. The primary DCL commands are GRANT (to give permissions) and REVOKE (to remove permissions).

What is TCL (Transaction Control Language)?

TCL consists of commands used to manage and control database transactions — groups of SQL statements that are treated as a single unit of work. TCL ensures data consistency through the ACID properties. Key TCL commands are COMMIT, ROLLBACK, and SAVEPOINT.

DDL Commands

What is CREATE?

CREATE is a DDL command used to create new database objects such as tables, views, databases, indexes, stored procedures, and functions. Once executed, the object is immediately available in the database. Example: CREATE TABLE EMP (EMP_ID INT, EMP_NAME VARCHAR(100)) creates a new employee table.

What is ALTER?

ALTER is a DDL command used to modify the structure of an existing database object without dropping and recreating it. It can add, modify, or drop columns, change data types, add constraints, or rename objects. Example: ALTER TABLE EMP ADD EMP_EMAIL VARCHAR(200) adds a new column to an existing table.

What is DROP?

DROP is a DDL command that permanently deletes a database object — including all its data, structure, indexes, and constraints — from the database. It cannot be rolled back and is irreversible once committed. Example: DROP TABLE EMP permanently removes the EMP table and all its data.

What is TRUNCATE?

TRUNCATE is a DDL command that removes all rows from a table quickly by deallocating the data pages, but keeps the table structure intact for future use. It is faster than DELETE because it does not log individual row deletions and resets identity columns. TRUNCATE cannot be used when foreign key constraints reference the table.

DML Commands

What is INSERT?

INSERT is a DML command used to add new rows of data into a table. You can insert a single row, multiple rows at once, or data from another table using INSERT INTO ... SELECT. Example: INSERT INTO EMP VALUES (101, 'Ashok', 'HYD', 35000) adds one employee record to the EMP table.

What is UPDATE?

UPDATE is a DML command used to modify existing data in one or more rows of a table. It requires a SET clause to specify which columns to change and a WHERE clause to target specific rows. Without a WHERE clause, UPDATE will modify all rows in the table, so it must be used carefully.

What is DELETE?

DELETE is a DML command used to remove one or more specific rows from a table based on a condition in the WHERE clause. Unlike TRUNCATE, DELETE logs each row removal individually, making it slower but recoverable within a transaction. Example: DELETE FROM EMP WHERE EMP_ID = 101 removes a single employee record.

DQL Command

What is SELECT?

SELECT is the DQL command used to retrieve data from one or more tables or views in a database. It is the most powerful and commonly used SQL statement, supporting filtering (WHERE), sorting (ORDER BY), grouping (GROUP BY), joining multiple tables, and aggregating data. Example: SELECT * FROM EMP returns all rows and columns from the EMP table.

DCL Commands

What is GRANT?

GRANT is a DCL command used to give specific permissions (such as SELECT, INSERT, UPDATE, DELETE, or EXECUTE) on database objects to users or roles. It controls who can access or manipulate data and objects. Example: GRANT SELECT ON EMP TO UserA gives UserA read-only access to the EMP table.

What is REVOKE?

REVOKE is a DCL command used to remove previously granted permissions from a user or role. Once revoked, the user loses the ability to perform the specified action on the database object. Example: REVOKE SELECT ON EMP FROM UserA removes UserA's ability to query the EMP table.

TCL Commands

What is COMMIT?

COMMIT is a TCL command that permanently saves all changes made during the current transaction to the database. Once committed, the changes cannot be undone with ROLLBACK. It marks the successful end of a transaction and releases any locks held during the transaction.

What is ROLLBACK?

ROLLBACK is a TCL command that undoes all changes made during the current transaction, restoring the database to its state before the transaction began. It is used when an error occurs or when a transaction must be cancelled to maintain data integrity. ROLLBACK can also roll back to a specific SAVEPOINT.

What is SAVEPOINT?

SAVEPOINT is a TCL command that creates a named marker within a transaction, allowing partial rollback to that specific point rather than rolling back the entire transaction. It provides finer control over transactions. Example: SAVE TRANSACTION sp1 creates a savepoint, and ROLLBACK TRANSACTION sp1 reverts only the changes made after that point.

Aggregate Functions

What is a Function in SQL Server?

A function is a reusable named block of SQL code that accepts input parameters, performs a specific operation, and returns a result. SQL Server has built-in functions (aggregate, date, string, conversion) and supports user-defined functions (UDFs). Functions can be used inside SELECT statements, WHERE clauses, and expressions.

What is the SUM() Function?

SUM() is an aggregate function that calculates the total of all numeric values in a specified column across a set of rows. It ignores NULL values during calculation. Example: SELECT SUM(EMP_SAL) FROM EMP returns the total salary of all employees.

What is the AVG() Function?

AVG() is an aggregate function that computes the arithmetic mean (average) of all non-NULL numeric values in a column. It divides the sum of values by the count of non-NULL values. Example: SELECT AVG(EMP_SAL) FROM EMP returns the average salary across all employees.

What is the COUNT() Function?

COUNT() is an aggregate function that counts the number of rows or non-NULL values in a column. COUNT(*) counts all rows including NULLs, while COUNT(column_name) counts only non-NULL values in that column. Example: SELECT COUNT(EMP_ID) FROM EMP returns the total number of employees.

What is the MIN() Function?

MIN() is an aggregate function that returns the smallest value in a specified column, ignoring NULL values. It works with numeric, date, and string data types. Example: SELECT MIN(EMP_SAL) FROM EMP returns the lowest salary among all employees.

What is the MAX() Function?

MAX() is an aggregate function that returns the largest value in a specified column, ignoring NULL values. It works with numeric, date, and string data types. Example: SELECT MAX(EMP_SAL) FROM EMP returns the highest salary among all employees.

NULL Handling Functions

What is ISNULL()?

ISNULL() checks whether the given expression is NULL and replaces it with a specified substitute value. It takes two arguments: ISNULL(expression, replacement). Example: ISNULL(EMP_BONUS, 0) returns 0 if EMP_BONUS is NULL, preventing NULL from propagating in calculations.

What is NULLIF()?

NULLIF() compares two expressions and returns NULL if they are equal; otherwise, it returns the first expression. It takes the form NULLIF(expr1, expr2). Example: NULLIF(EMP_SAL, 0) returns NULL when salary is 0, useful for avoiding division-by-zero errors in calculations.

What is COALESCE()?

COALESCE() evaluates a list of expressions in order and returns the first non-NULL value found. It accepts multiple arguments: COALESCE(val1, val2, val3, ...). Example: COALESCE(EMP_MOBILE, EMP_EMAIL, 'No Contact') returns the first available contact detail for an employee.

Conversion Functions

What is the CAST() Function?

CAST() converts a value from one data type to another using the SQL standard syntax CAST(expression AS data_type). It is used when you need to change a data type for display, calculation, or comparison purposes. Example: CAST(EMP_SAL AS VARCHAR(10)) converts a numeric salary to a string for concatenation.

What is the CONVERT() Function?

CONVERT() is a SQL Server-specific function that converts a value to a specified data type, with an optional style parameter for date formatting: CONVERT(data_type, expression, style). Example: CONVERT(VARCHAR(10), GETDATE(), 103) formats the current date as DD/MM/YYYY. It is more flexible than CAST for date conversions.

Date & Time Functions

What is a Date Function in SQL Server?

Date functions are built-in functions that perform operations on date and time values, such as extracting parts of a date, adding or subtracting intervals, calculating differences, and formatting dates for display. They are essential for working with time-series data, calculating ages, tenure, or deadlines.

What is GETDATE()?

GETDATE() returns the current system date and time of the SQL Server instance as a DATETIME value. It takes no parameters and is widely used for timestamping records, calculating age or tenure, and setting default values. Example: SELECT GETDATE() returns something like 2026-05-16 14:35:22.123.

What is the YEAR() Function?

YEAR() extracts and returns the year component as an integer from a given date expression. Example: YEAR('2024-08-15') returns 2024. It is a shorthand equivalent to DATEPART(YEAR, date) and is commonly used in filters, groupings, and reporting.

What is the MONTH() Function?

MONTH() extracts and returns the month component (1 through 12) as an integer from a given date expression. Example: MONTH('2024-08-15') returns 8. It is useful for grouping data by month, calculating monthly metrics, or filtering records for a specific month.

What is the DAY() Function?

DAY() extracts and returns the day of the month as an integer from a given date expression. Example: DAY('2024-08-15') returns 15. It is equivalent to DATEPART(DAY, date) and is used in date arithmetic and grouping by day of month.

What is the FORMAT() Function?

FORMAT() converts a date, time, or numeric value to a formatted string using a .NET-style format pattern. Example: FORMAT(GETDATE(), 'dd-MMM-yyyy') returns '16-May-2026'. It is flexible for display purposes but is slower than CONVERT() and should be avoided for large result sets.

What is the DATEPART() Function?

DATEPART() returns a specific part of a date (such as year, quarter, month, day, hour, minute, second) as an integer. Syntax: DATEPART(datepart, date). Example: DATEPART(MONTH, '2024-08-15') returns 8. It supports many interval types including WEEK, WEEKDAY, and MILLISECOND.

What is the DATEADD() Function?

DATEADD() adds or subtracts a specified number of date/time intervals to a date value. Syntax: DATEADD(datepart, number, date). Example: DATEADD(MONTH, 3, '2024-01-01') returns '2024-04-01'. It is used to calculate future or past dates such as due dates or expiry dates.

What is the DATENAME() Function?

DATENAME() returns a specified part of a date as a character string (text) instead of a number. Syntax: DATENAME(datepart, date). Example: DATENAME(MONTH, '2024-08-15') returns 'August' instead of 8. It is useful when readable labels like day names or month names are needed.

What is the DATEDIFF() Function?

DATEDIFF() calculates the difference between two date values in terms of a specified interval. Syntax: DATEDIFF(datepart, startdate, enddate). Example: DATEDIFF(YEAR, '2000-05-01', GETDATE()) returns the age in years. It is widely used to calculate tenure, age, duration, or overdue days.

What is the EOMONTH() Function?

EOMONTH() returns the last day of the month for a given date, optionally adding a specified number of months offset. Syntax: EOMONTH(date, offset). Example: EOMONTH('2024-02-10') returns '2024-02-29' (accounts for leap years). It is useful for billing cycles, month-end reporting, and deadline calculations.

What is DATETRUNC()??

DATETRUNC() truncates a date or datetime value to the specified date part precision, setting smaller parts to their minimum values. Syntax: DATETRUNC(datepart, date). Example: DATETRUNC(MONTH, '2024-08-15') returns '2024-08-01'. It was introduced in SQL Server 2022 and is useful for time-based grouping and aggregation.

String Functions

What is a Text/String Function?

String functions are built-in SQL Server functions used to manipulate, format, extract, or search character data (text). They allow you to clean, split, combine, and transform string values in queries without using application-level code. Common examples include LEFT, RIGHT, LEN, SUBSTRING, REPLACE, and TRIM.

What is the LEFT() Function?

LEFT() extracts a specified number of characters from the left (beginning) of a string. Syntax: LEFT(string, number_of_characters). Example: LEFT('SQL Server', 3) returns 'SQL'. It is commonly used to extract prefixes, codes, or the first few characters of a string.

What is the RIGHT() Function?

RIGHT() extracts a specified number of characters from the right (end) of a string. Syntax: RIGHT(string, number_of_characters). Example: RIGHT('SQL Server', 6) returns 'Server'. It is useful for extracting suffixes, file extensions, or the last few digits of a code.

What is the LEN() Function?

LEN() returns the number of characters in a string, excluding trailing spaces. Syntax: LEN(string). Example: LEN('SQL Server') returns 10. It is commonly used to validate input length, check for empty strings, or apply conditions based on string length.

What is the SUBSTRING() Function?

SUBSTRING() extracts a portion of a string starting at a specified position for a given length. Syntax: SUBSTRING(string, start, length). Example: SUBSTRING('SQL Server', 5, 6) returns 'Server'. It is the most flexible string extraction function and works on any position within the string.

What is the CHARINDEX() Function?

CHARINDEX() searches for a substring within a string and returns the starting position (index) of its first occurrence. Syntax: CHARINDEX(substring, string, start). Example: CHARINDEX('Server', 'SQL Server') returns 5. If not found, it returns 0. Useful for locating separators or validating patterns.

What is the REPLACE() Function?

REPLACE() substitutes all occurrences of a specified substring within a string with a new substring. Syntax: REPLACE(string, old_value, new_value). Example: REPLACE('SQL Server 2019', '2019', '2022') returns 'SQL Server 2022'. It is used for data cleaning and text transformations.

What is the STUFF() Function?

STUFF() deletes a specified number of characters from a string starting at a given position and inserts a new string in their place. Syntax: STUFF(string, start, length, new_string). Example: STUFF('SQL Server', 4, 6, 'Database') returns 'SQL Database'. It is more powerful than REPLACE when position-based replacement is needed.

What is the REPLICATE() Function?

REPLICATE() repeats a given string a specified number of times. Syntax: REPLICATE(string, number). Example: REPLICATE('A', 5) returns 'AAAAA'. It is commonly used to generate padding, patterns, or repeated characters in output formatting.

What is the LTRIM() Function?

LTRIM() removes all leading (left-side) spaces from a string without affecting the content or trailing spaces. Syntax: LTRIM(string). Example: LTRIM(' SQL Server') returns 'SQL Server'. It is useful for cleaning user-input data where accidental spaces at the beginning may cause comparison issues.

What is the RTRIM() Function?

RTRIM() removes all trailing (right-side) spaces from a string. Syntax: RTRIM(string). Example: RTRIM('SQL Server ') returns 'SQL Server'. It is used to clean data imported from external sources where trailing whitespace may affect sorting or joining.

What is the TRIM() Function?

TRIM() removes both leading and trailing spaces (or a specified character) from a string in a single operation. Syntax: TRIM(string) or TRIM(character FROM string). Example: TRIM(' SQL Server ') returns 'SQL Server'. It was introduced in SQL Server 2017 as a cleaner alternative to combining LTRIM and RTRIM.

SQL Clauses

What is a Clause in SQL Server?

A clause is a component or keyword section of a SQL statement that defines a specific operation or filter on the data being queried. Clauses are the building blocks of SELECT statements. Common clauses include SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, and TOP.

What is the TOP Clause?

The TOP clause limits the number of rows returned by a SELECT query to a specified count or percentage. Syntax: SELECT TOP N column FROM table. Example: SELECT TOP 5 * FROM EMP ORDER BY EMP_SAL DESC returns the 5 highest-paid employees. It is SQL Server-specific and is commonly used for pagination and ranked results.

What is the WHERE Clause?

The WHERE clause filters rows returned by a SELECT, UPDATE, or DELETE statement based on a specified condition. Only rows where the condition evaluates to TRUE are included in the result. Example: SELECT * FROM EMP WHERE EMP_LOC = 'HYD' returns only employees based in Hyderabad.

What is the GROUP BY Clause?

GROUP BY groups rows that have the same values in specified columns into summary rows, typically used with aggregate functions like SUM, COUNT, AVG, MIN, and MAX. Example: SELECT EMP_LOC, COUNT(*) FROM EMP GROUP BY EMP_LOC returns the number of employees in each location.

What is the HAVING Clause?

HAVING filters groups produced by GROUP BY based on a condition applied to aggregate results. It is similar to WHERE but operates after grouping. Example: SELECT EMP_LOC, COUNT(*) FROM EMP GROUP BY EMP_LOC HAVING COUNT(*) > 5 returns only locations with more than 5 employees.

What is the ORDER BY Clause?

ORDER BY sorts the result set of a query in ascending (ASC) or descending (DESC) order based on one or more columns. By default, ORDER BY sorts in ascending order. Example: SELECT * FROM EMP ORDER BY EMP_SAL DESC returns employees sorted from highest to lowest salary.

Window Functions

What is a Window Function?

A window function performs a calculation across a defined set (window) of rows related to the current row without collapsing the result set like GROUP BY does. Each row retains its identity while also receiving an aggregate or ranking value. Window functions use the OVER() clause to define the window.

What is ROW_NUMBER()?

ROW_NUMBER() assigns a unique sequential integer to each row within a partition, starting from 1, ordered by a specified column. It does not allow ties — each row gets a distinct number. Example: ROW_NUMBER() OVER(PARTITION BY EMP_LOC ORDER BY EMP_SAL DESC) ranks employees within each location by salary.

What is RANK()?

RANK() assigns a rank to each row within a partition based on the ORDER BY column, but leaves gaps in ranking when there are ties. For example, if two employees share rank 2, the next rank will be 4 (not 3). It is useful when ties are meaningful and gaps in ranking are acceptable.

What is DENSE_RANK()?

DENSE_RANK() assigns ranks to rows similar to RANK() but without leaving gaps after ties. If two rows share rank 2, the next rank is 3 (not 4). It is preferred when you want consecutive ranking with no gaps, such as finding the 2nd or 3rd highest salary.

What is LAG()?

LAG() is a window function that accesses data from a previous row within the same result set partition without using a self-join. Syntax: LAG(column, offset, default) OVER(ORDER BY col). Example: LAG(EMP_SAL, 1, 0) returns the previous employee's salary in the ordered set, useful for comparing sequential values.

What is LEAD()?

LEAD() is a window function that accesses data from the next row within the same result set partition. Syntax: LEAD(column, offset, default) OVER(ORDER BY col). Example: LEAD(EMP_SAL, 1, 0) returns the next employee's salary, useful for comparing current values to future values in time-series or ordered data.

What is PARTITION BY?

PARTITION BY divides the result set into subsets (partitions) for a window function to operate on independently, similar to GROUP BY but without collapsing rows. Example: PARTITION BY EMP_LOC in ROW_NUMBER() resets the row number counter for each city independently. Without PARTITION BY, the window function operates on the entire result set.

What is ORDER BY in a Window Function?

ORDER BY within the OVER() clause defines the order in which rows are processed within each window partition for ranking or running calculations. It determines which row is 'first' or 'previous' in functions like ROW_NUMBER, LAG, and LEAD. It is separate from the ORDER BY at the end of the query that sorts the final output.

What is OVER() in a Window Function?

OVER() is a mandatory clause that defines the window (set of rows) on which a window function operates. It can optionally include PARTITION BY (to divide data into groups) and ORDER BY (to define row order within each group). Without OVER(), analytic functions like ROW_NUMBER() and RANK() cannot work.

What is PRECEDING in a Window Function?

PRECEDING defines the starting boundary of a window frame relative to the current row, specifying how many rows before the current row are included in the calculation. For example, ROWS BETWEEN 2 PRECEDING AND CURRENT ROW includes the current row and the 2 rows before it in a running calculation like a moving average.

What is UNBOUNDED PRECEDING?

UNBOUNDED PRECEDING sets the window frame boundary to the very first row of the partition — meaning the window starts from the beginning of the partition without any limit. It is commonly used with running totals: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW calculates the cumulative sum from the first row up to the current row.

What is FOLLOWING in a Window Function?

FOLLOWING defines the ending boundary of a window frame relative to the current row, specifying how many rows after the current row are included. Example: ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING includes the current row and the next 2 rows. UNBOUNDED FOLLOWING extends the frame to the last row of the partition.

Joins

What is a JOIN in SQL Server?

A JOIN is a clause used to combine rows from two or more tables based on a related column (usually a primary key and foreign key relationship). JOINS allow you to retrieve meaningful combined data from normalized database tables. SQL Server supports INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, SELF JOIN, and CROSS JOIN.

What is INNER JOIN?

INNER JOIN returns only the rows where there is a matching value in both tables based on the join condition. Non-matching rows from either table are excluded from the result. Example: Joining EMP and DEP_TBL on DEP_ID returns only employees who belong to an existing department.

What is LEFT JOIN (LEFT OUTER JOIN)?

LEFT JOIN returns all rows from the left (first) table and only the matching rows from the right table. Where no match exists in the right table, NULL values are returned for the right table's columns. It is used when you want to keep all records from the main table regardless of matches.

What is RIGHT JOIN (RIGHT OUTER JOIN)?

RIGHT JOIN returns all rows from the right (second) table and only the matching rows from the left table. Unmatched rows in the left table produce NULL values. It is the mirror of LEFT JOIN and can usually be rewritten as a LEFT JOIN by swapping the table order.

What is FULL OUTER JOIN?

FULL OUTER JOIN returns all rows from both tables, matching rows where the join condition is true, and NULLs where no match exists on either side. It effectively combines the results of LEFT JOIN and RIGHT JOIN. Useful for finding data that exists in one table but not the other.

What is SELF JOIN?

A SELF JOIN joins a table with itself using aliases to distinguish between the two instances. It is used to compare rows within the same table. Example: Joining EMP with itself on EMP_ID = MANAGER_ID retrieves each employee's manager name from the same table.

What is CROSS JOIN?

CROSS JOIN returns the Cartesian product of two tables — every row from the first table is combined with every row from the second table. If table A has 5 rows and table B has 4 rows, CROSS JOIN produces 20 rows. It is used for generating combinations but must be used carefully with large tables.

Subqueries, CTEs & Temp Tables

What is a Subquery?

A subquery is a SELECT statement nested inside another SQL statement (SELECT, INSERT, UPDATE, or DELETE) that provides a value or set of values to the outer query. Subqueries can be used in the WHERE, FROM, or SELECT clause. Example: SELECT * FROM EMP WHERE EMP_SAL = (SELECT MAX(EMP_SAL) FROM EMP) finds the highest-paid employee.

What is a CTE (Common Table Expression)?

A CTE is a temporary named result set defined using the WITH keyword that exists only for the duration of the query. It improves readability by breaking complex queries into named, reusable parts. CTEs are commonly used for recursive queries, ranking with ROW_NUMBER(), and deleting duplicate rows.

What is a Temp Table?

A temp table is a temporary table stored in the TempDB system database that exists only for the duration of a session or stored procedure. Prefixed with # (local) or ## (global), temp tables store intermediate results for multi-step operations. Example: SELECT * INTO #TempEMP FROM EMP WHERE EMP_LOC = 'HYD' creates a temporary copy of HYD employees.

Iterative Statements & Variables

What is an Iterative Statement?

An iterative statement is a programming construct in T-SQL that allows repeated or conditional execution of SQL code blocks. SQL Server supports IF, IF ELSE IF, and WHILE as control flow constructs for decision-making and looping within scripts, stored procedures, and triggers.

What is IF?

IF is a conditional statement that evaluates a Boolean expression and executes a SQL block (BEGIN...END) only when the condition is TRUE. If the condition is FALSE, the block is skipped entirely. Example: IF @AGE >= 18 BEGIN PRINT 'Eligible to vote' END evaluates the age and prints accordingly.

What is IF ELSE IF?

IF ELSE IF is a multi-branch conditional construct that evaluates multiple conditions in sequence and executes the first block whose condition is TRUE. It allows slab-based logic such as discount tiers or bonus slabs. Once a matching condition is found, remaining branches are skipped.

What is a WHILE Loop?

WHILE is a loop construct that repeatedly executes a block of SQL statements as long as the specified condition remains TRUE. It requires an increment or change inside the loop to eventually make the condition FALSE; otherwise, it results in an infinite loop.

Example:

A WHILE loop with REPLICATE() can print increasing patterns like A, AA, AAA.

What is DECLARE?

DECLARE is a T-SQL statement used to define a local variable with a specified data type within a script, stored procedure, or function. A variable declared with DECLARE holds a temporary value during execution and is not visible outside its scope.

Example:

DECLARE @EMP_SAL INT = 35000 creates an integer variable initialized to 35000.

What is a Parameter?

A parameter is a variable passed into or returned from a stored procedure or function to exchange data between the caller and the procedure. Input parameters supply values to the procedure, while OUTPUT parameters return results back to the caller.

Example:

@EMP_ID INT is an input parameter that accepts an employee ID when calling a procedure.

Set Operators

What is a Set Operator?

A set operator combines the result sets of two or more SELECT statements into a single result. Both queries must return the same number of columns with compatible data types. SQL Server supports four set operators: UNION, UNION ALL, INTERSECT, and EXCEPT.

What is UNION?

UNION combines the results of two SELECT queries into a single result set and automatically removes duplicate rows. It requires both queries to have the same number of columns with compatible data types.

Example:

Combining HYD employees and VIZ employees using UNION returns one unique list with no repeated records.

What is UNION ALL?

UNION ALL combines the results of two SELECT queries including all duplicate rows without deduplication. It is faster than UNION because it skips the sorting step required to remove duplicates. Use UNION ALL when duplicates are expected and acceptable, or when performance is critical.

What is INTERSECT?

INTERSECT returns only the rows that appear in the result sets of both SELECT queries. It acts like a mathematical intersection of two sets. Example: EMP_ID INTERSECT MANAGER_ID returns employee IDs that are also manager IDs — i.e., employees who manage others.

What is EXCEPT?

EXCEPT returns rows from the first SELECT query that do not appear in the second SELECT query's result. It acts like a mathematical set difference (A minus B). Example: EMP_ID EXCEPT MANAGER_ID returns employees who are not managers — present in EMP but not in the MANAGER_ID list.

Views (Detailed)

What is a View?

A view is a saved, named SELECT query stored in the database that behaves like a virtual table. It does not store data physically (except indexed views) but presents data from underlying tables dynamically whenever queried. Views are used for security (hiding sensitive columns), simplifying complex queries, and providing a consistent interface to data.

What is a Simple View?

A simple view is based on a single table with a straightforward SELECT query — no joins, aggregations, or GROUP BY. Because it directly maps to one table, it supports DML operations like INSERT, UPDATE, and DELETE through the view.

Example:

```
CREATE VIEW vw_HYD_EMP AS SELECT * FROM EMP WHERE EMP_LOC = 'HYD'.
```

What is a Complex View?

A complex view is built using multiple tables with JOINS, aggregate functions, GROUP BY, UNION, or subqueries. Because the data is derived from multiple sources or transformed, DML operations (INSERT, UPDATE, DELETE) are generally not supported through complex views. Example: A city metrics view joining Orders, Customers, and OrderItems.

What is an Indexed (Materialized) View?

An indexed view physically stores the view's result set on disk by creating a unique clustered index on it, unlike regular views that execute the query each time. It requires the WITH SCHEMABINDING option and uses COUNT_BIG() for aggregations. It dramatically improves read performance for large, frequently-queried aggregations at the cost of storage and write overhead.

Stored Procedures (Detailed)

What is a Stored Procedure?

A stored procedure is a named, precompiled collection of T-SQL statements stored in the database that can be executed by name using the EXEC command. It improves performance through cached execution plans, promotes code reuse, and enhances security by restricting direct table access. Stored procedures can accept input parameters, return output parameters, and return result sets.

What is an Input Parameter?

An input parameter is a variable declared in a stored procedure's parameter list that accepts a value from the caller at execution time. It allows the same procedure to be reused with different data inputs. Example: @EMP_ID INT in sp_GetEmployee accepts an employee ID so the procedure can look up that specific employee's details.

What is an Output Parameter?

An output parameter is a parameter declared with the OUTPUT keyword in a stored procedure that sends a value back to the calling code after the procedure executes. The caller must also use the OUTPUT keyword when executing the procedure to capture the returned value.

Example:

```
@TOTAL_SAL INT OUTPUT returns the calculated total salary to the calling script.
```

Triggers (Detailed)

What is a Trigger?

A trigger is a database object that automatically executes a predefined set of SQL statements in response to specific DML events (INSERT, UPDATE, DELETE) on a table or view. Triggers are invisible to the user and fire automatically, making them ideal for auditing, enforcing rules, and maintaining data integrity without requiring application-level code.

What is an AFTER INSERT Trigger?

An AFTER INSERT trigger fires automatically after a new row is successfully inserted into the target table. It has access to the INSERTED virtual table, which contains the newly added rows. Example: A trigger on T_EMP that logs the new EMP_ID and timestamp into T_EMP_LOG every time a new employee record is added.

What is an AFTER DELETE Trigger?

An AFTER DELETE trigger fires automatically after one or more rows are successfully deleted from the target table. It has access to the DELETED virtual table, which contains the removed rows before deletion. Example: A trigger that logs deleted employee IDs into an audit table with the deletion timestamp for compliance tracking.

What is an AFTER UPDATE Trigger?

An AFTER UPDATE trigger fires automatically after rows in the target table are modified by an UPDATE statement. It has access to both the INSERTED table (containing new values) and the DELETED table (containing old values before the update).

Example:

A trigger that records salary changes by logging the old and new salary whenever the EMP_SAL column is updated.

What is an INSTEAD OF UPDATE Trigger?

An INSTEAD OF UPDATE trigger intercepts an UPDATE operation and replaces it with custom logic defined inside the trigger body. The original UPDATE statement is not executed — instead, only the trigger's code runs. It is commonly used on views to implement custom update behavior when the underlying structure does not support direct updates.

What is an INSTEAD OF DELETE Trigger?

An INSTEAD OF DELETE trigger intercepts a DELETE operation and replaces it with custom logic, preventing the actual deletion from occurring. It is commonly used to implement soft deletes — instead of physically removing a row, the trigger sets an IsDeleted flag to 1 or moves the record to an archive table.

What is an INSTEAD OF INSERT Trigger?

An INSTEAD OF INSERT trigger intercepts an INSERT operation and replaces it with custom logic defined in the trigger. The original INSERT is not performed; only the trigger's code executes. It is typically used on views with complex underlying structures to route insert data to the correct underlying tables.

Indexes (Detailed)

What is an Index in SQL Server?

An index is a performance-optimization database object that creates a sorted data structure on one or more table columns to speed up data retrieval. Without an index, SQL Server performs a full table scan for every query. While indexes dramatically improve SELECT performance, they add overhead to INSERT, UPDATE, and DELETE operations and consume additional storage.

What is a Clustered Index?

A clustered index physically sorts and stores the table's data rows in the order of the index key. Because the data itself is reorganized, a table can have only one clustered index. SQL Server automatically creates a clustered index when a PRIMARY KEY constraint is defined.

Example:

CREATE CLUSTERED INDEX CI_EMP ON EMP(EMP_ID) stores employee rows in EMP_ID order on disk.

What is a Non-Clustered Index?

A non-clustered index creates a separate structure that contains the index key values and pointers (row locators) to the actual data rows, without physically reordering the table. A table can have up to 999 non-clustered indexes.

Example:

CREATE NONCLUSTERED INDEX NCI_EMP_MAIL ON EMP(EMP_MAIL) speeds up queries that search by email address.

What is a Columnstore Index?

A columnstore index stores data column-by-column rather than row-by-row, enabling extremely efficient compression and fast aggregate queries (SUM, AVG, COUNT) over large datasets. It is ideal for analytical and data warehouse workloads. The Clustered Columnstore Index (CCI) replaces the row-store structure entirely, while a Non-Clustered Columnstore Index coexists alongside it.

What is a Row Store Index?

A row store index is the traditional SQL Server index type (both clustered and non-clustered) that stores data row by row. It is optimized for OLTP (Online Transaction Processing) workloads involving frequent point lookups, range scans, and DML operations. Row store indexes are the default index type and contrast with columnstore indexes used for analytics.

What is a Filtered Index?

A filtered index is a non-clustered index created on a subset of rows in a table using a WHERE clause filter condition. It is smaller and more efficient than a full-table index when queries consistently filter on the same condition.

Example:

CREATE INDEX FI_ACTIVE_EMP ON EMP(EMP_ID) WHERE IsActive = 1 creates an index only for active employees, saving space and improving query speed.